LIVING DOCUMENT · UPDATED WITH EVERY CHANGE

FOX_Feature Implemented

Everything that has been built into FoxByte so far — for each feature: **what it does**, **how it works**, **how to use it**, **how to test it**, and its honest **status**. When a feature is added or changed, this document changes in the same commit.

Product: FoxByte — PostgreSQL with copy-on-write branching and a Blackbox

Last updated: 22 September 2026 · **Reflects:** `main` @ `2778762` (v1.0.1, exact repository path PR #1) + the FoxByte rename and brand indirection (`chore/foxbyte`)

License: AGPL-3.0 core · Apache-2.0 client SDKs

Source of truth: `docs/FOX_Feature_Implemented.html` → `make feature-doc` → this PDF

Supersedes: `docs/foxbyte-feature-reference` (25 Aug 2026, no longer maintained)

Method: every entry was taken from a read of the source code and git history, with file references. Status reflects the code; "Working" means the code path is complete, not that it was re-run for this edition unless the Test baseline section says so.

Change log

Newest first. Every change to FoxByte adds a row here.

Date	Feature(s)	What changed	Commit / PR
24 Sep 2026	A9, A1, M4	**The first signed release, checked end to end — and one thing it showed.** v0.4 (23 Sep) is the first release the new workflow published: `SHA256SUMS.sig` and the SPDX SBOM are attached, the signature verifies against the key built into fox (`releasesign verify`), and the merged `SHA256SUMS` carries the Windows distro's entry as well as the binaries' — the two publishing jobs raced and the re-signing held. `gh attestation verify` passes on a release file. `install.sh` run against it on macOS said "SHA256SUMS is signed with the FoxByte release key", verified both binaries and installed 0.4. The `postgres-walg` package is public: an anonymous manifest fetch answers 200, so installs pull the image instead of building it. **What it showed:** a build carrying another key refused v0.4 by name when the version was pinned, but plain `fox update --check` said "up to date" — a release that exists and does not verify was passed over in silence, which would hide a forged or broken one for ever. Resolve now returns that error even without a pin, while a genuine newer release is still offered (`TestResolveReportsABadSignature`); checked again against the real v0.4 both ways. **The console and the clients were checked against the release itself**, not only against a development build: in the throwaway VM the released `fox-linux-arm64` was verified by signature and checksum, reported `fox 0.4` and carries the console (only the Linux binaries are built with `-tags embedui` — on macOS and Windows the engine, and the console, run in the VM), and with it serving, `integration-ui` passed 5/5 and `integration-sdks` 21/21.	`fix/update-reports-bad-signature`
23 Sep 2026	Next steps	**The checklist's next steps, read against the audit's register.** Two were done and still listed (the VM suites in CI, and tests for the SDKs and the web UI), so they are gone; what is left is named plainly: cut the first signed release and check it end to end, make the image package public, watch tonight's nightly, the real-machine checks the VM cannot do, tests for the imports — the last area with none — gating release tags on the nightly, and a decision about how this document badges partial work (audit v2 G31: 34 features are badged done or manual while carrying an open sub-item). The seven gaps the audit raised and the owner left for later are listed together, with their ids, so nothing looks forgotten.	`chore/next-steps`
23 Sep 2026	M2, M1, K3, K4, L2–L6, F4, L4; known issues	**The suites run nightly in CI, the published clients and the web console have tests, and a bug they found is fixed.** Batch 5b of the 22 Sep audit (v2), the last. **M2 (G24):** `nightly.yml` runs each suite on a runner of its own — integration, ledger-v2, update, pg-upgrade, sdks, ui — installing ZFS and psql, building the engine (with the console embedded for the UI tests) and running it with `FOX_TEST_ALLOW_REAL_INSTALL` , which the guard allows because the runner is disposable; a failure keeps the log and the console's screenshots and traces for 14 days, and one suite can be run on demand. **K3, K4 (G26):** `scripts/integration_sdks.sh` drives each published client against a live engine — status, branches, SQL, the Blackbox's entries and verify, suspend, resume, the error a missing branch gives, delete — and the TypeScript client must	`feat/ci-suites-ui-tests`

		compile with its own tsconfig (21 checks). **L2–L6 (G26):** `scripts/integration_ui.sh` runs Playwright in Chromium against a real engine: sign in, a wrong password, a branch made in the dashboard, SQL in the console and that change in the Blackbox attributed to the signed-in account, and an API key shown once (5 tests). The test VM now installs nodejs and npm; both suites build in a copy, since the repository is read-only there. **F4 — a bug the SDK tests found:** a branch deleted and made again under the same name is a fresh clone, but this process still remembered provisioning its login roles, so the engine skipped creating an account's role there and Postgres reported the login as a wrong password. `forgetBranchRoles` drops those caches with the branch (`TestRecreatingABranchForgetsItsRoles`). **L4 — a limit the tests found:** the console's endpoint runs one statement per call; several in one string are refused by the extended protocol. Accepting them would mean the simple protocol, which changes how every value is rendered, so it is recorded rather than changed. Two suite failures were the suites themselves: the UI tests met services left running from a build without the console (they now restart it), and §1 cleared the account store but left the security log's anchors, which then rightly reported tampering (it clears them too). Verified in the throwaway VM: `make integration` 170/170, `integration-v2` 303/303, `integration-update` 51/51, `integration-pg-upgrade` 72/72, `integration-sdks` 21/21, `integration-ui` 5/5; `go test ./...` , `tsc` , `npm ci` . **The first nightly run (23 Sep, 02:17 UTC)** answered the open question: GitHub's runners do carry the ZFS module, and ledger-v2, update, pg-upgrade and sdks passed there. Two jobs failed, both on how CI built or looked things up rather than on the product. **ui:** the workflow ran `vite build` directly, while `make web-build` sets `VITE_API_URL=` — the console it built called `http://localhost:8080` instead of its own origin, and every request from the page failed ("Failed to fetch", in the screenshot the job kept). It now runs `make web-build` . **integration §11c:** the suite picks the MinIO client image out of `docker images` by tag, and an image pulled as tag@digest — which the engine does since 22 Sep — is listed with the tag `<none>`, so the reference was invalid and the test bucket was never made; it now takes the digest, and prints what the command said when it fails. The throwaway VM could not have caught either: its images carry both a tag and a digest, and its target builds the UI through the Makefile. Re-run there afterwards: `make integration` 170/170, `integration-ui` 5/5.	
22 Sep 2026	A1, A2, A9, M4, M6; known issues; next steps	**Releases are signed, and what they are built from is pinned by content.** Batch 5a of the 22 Sep audit (v2); the VM suites in CI and UI/SDK tests are 5b, by the owner's choice. **A9, A1 (G22):** anyone who could publish a release could replace a binary and its checksum together. SHA256SUMS is now signed with an Ed25519 release key the owner generated (`make release-key` , `cmd/releasesign`): the private half is the `FOX_RELEASE_SIGNING_KEY` Actions secret, the public half is built into fox and into `install.sh` , and a test keeps the two the same. `fox update` offers no release without a valid `SHA256SUMS.sig` ; `install.sh` checks it with OpenSSL 3 and refuses another key's or none (`FOX_ALLOW_UNSIGNED=1` for old releases), and where openssl is LibreSSL (macOS) says the signature was not checked unless `FOX_REQUIRE_SIGNATURE=1` . `install.ps1` checks checksums only: PowerShell has no Ed25519. **M4 (G22, G23):** every Action pinned by commit SHA;	`feat/supply-chain`

		the release fails without the signing secret; `release-checksums.sh` signs the merged SHA256SUMS and re-signs when the two publishing jobs race; `check-release-assets.sh` requires the signature; build attestations for every file and image digest, an SPDX SBOM, image provenance and SBOM; Dependabot for Actions, Go and npm. **M6 (G23):** the postgres base pinned by digest per major and wal-g checked by SHA-256 before unpacking (all four values confirmed: the digests against the registries and the VM's own images, the wal-g hashes against wal-g's published .sha256); MinIO pinned by digest, without the old fallback to `minio/minio:latest`; `scripts/check-image-digests.sh` reports a pin that has fallen behind its tag. The first update-suite run failed because OpenSSL cannot sign an empty file, and one fake release lists nothing; that release is no longer signed. Verified in the throwaway VM: `make integration` 170/170 (its image build now pulls the base by digest), `integration-v2` 303/303, `integration-update` 51/51 (fake releases signed with a test key its builds are given by `-ldflags -X`), `integration-pg-upgrade` 72/72; `go test ./...`, the four CI scripts (the installer's with OpenSSL 3 and with LibreSSL), `go vet` for macOS/Linux/Windows. The workflows themselves run for the first time on the next tag. Linux CI then failed one installer test: on Linux amd64 the host asset and the VM's engine are the same file, so the case that drops the host asset from SHA256SUMS left the file empty — which openssl cannot sign, so the installer stopped at the signature rather than the missing entry. That case now keeps a line, and the suite was re-run as a Linux amd64 machine (a uname stub): 11/11 there and on macOS.	
22 Sep 2026	D1, D2, D3, H10; known issues	**Backups can leave the machine, under Object Lock, and they are taken for you.** Batch 4 of the 22 Sep audit (v2). **D1 (G18):** the WAL archive and every base backup sat in the object store beside main, on the same disk. `fox backup target set s3://bucket[/prefix] --endpoint … --access-key …` (secret asked for, or `FOX_BACKUP_S3_SECRET_KEY`) now points wal-g at any S3-compatible bucket: it checks the bucket and reports its Object Lock, saves the target (0600), restarts main so its archive_command follows, and takes the first base backup there; `fox backup target local` goes back. Plain HTTP is refused without `--allow-http`. Every wal-g call now takes its keys from an env file (`s3.env`), and the local store's root credentials too (`minio.env`, `mc-local.env`), so no secret key is a command-line argument. Before building on Object Lock it was tried in the test VM: wal-g and mc both upload into a locked MinIO bucket, and a locked object's delete is refused ("WORM protected"). **H10 (G17, off-host):** with a remote target, every anchor — each branch's and the security log's — is copied to `<target>/anchors/`, retried until it lands. **D2 (G19):** `fox start` takes a first base backup when the cluster has none; the control plane takes one whenever the newest is older than `FOX_BACKUP_INTERVAL` (24h) and keeps `FOX_BACKUP_RETAIN` (7) with their WAL; `fox backup prune`; `fox status` and `/api/status` report the target and the newest backup's age, and warn. **G20:** branches are documented as disposable. A developer-machine finding: the engine's unit tests wrote secrets and env files into the real `~/.fox` of whoever ran `go test` (since `pg.env` in batch 2); they now run under a temporary home (`internal/branch/main_test.go`). The first suite run failed three checks. Two were the tests: one `psql -c` string ran the switch of WAL segment before its own commit, so a	`feat/remote-backups`

		restore correctly did not have the table; and MinIO words a refused delete "WORM protected". One was the new first backup doing its job: the upgrade suite's install on 16 now has three base backups, not two. Verified in the throwaway VM: `make integration` 170/170 (13 new in §11c, against a second MinIO with Object Lock), `integration-v2` 303/303, `integration-update` 51/51, `integration-pg-upgrade` 72/72; `go test ./...`, `go vet` for macOS/Linux/Windows. Not tested against AWS S3 itself.	
22 Sep 2026	H1, H2, H4, H10, H11, H12, H15; new G6; known issues	**The Blackbox now sees TRUNCATE and agents' data changes, its anchors are signed, and a security log records what happens at the doors.** Batch 3 of the 22 Sep audit (v2), scoped with the owner: TRUNCATE plus agents' DML, off-host anchors with the backup target in batch 4. **H1, H4 (G04):** the Blackbox's event triggers see DDL only, so `TRUNCATE orders` was neither refused nor recorded and an agent's `DELETE` left no trace. `datachanges.sql` gives every user table `bb_guard_truncate` (BEFORE TRUNCATE: blocked by default like DROP TABLE — `bb.policy` op `TRUNCATE`, same override, same error, a BLOCKED entry naming the table; allowed, recorded with risk `truncate`; imports and pipelines, with the guardrail off, may truncate and are recorded) and `bb_record_dml` (AFTER UPDATE OR DELETE per statement: recorded with table and statement when the actor is an agent; a person's are not, so ordinary writes stay fast). An event trigger adds both to every new table; dropping, disabling or renaming one is refused without the override. The first run found one regression, and §9 caught it: the install script revoked a helper's EXECUTE on every start, which the Blackbox recorded as an unattributed privilege change; the helper only adds these protective triggers, so the REVOKE is gone. **H10–H12 (G17):** every new anchor carries an Ed25519 signature over a fixed payload (specified in docs/ledger-anchor-format.md) by `~/.fox/anchor-signing.key` or `FOX_ANCHOR_KEY`; `fox blackbox anchor-key` prints the public key; `fox-verify --pubkey` refuses a mismatched signature, another key's, or an unsigned anchor after a signed one; `fox blackbox integrity` checks with the install's own key. README no longer implies the hash chain alone survives a superuser: it explains anchors, where to keep them and who holds the key. **G6 (G28):** a new security log in the account store — sign-ins and failures, throttling, sign-ups and refused setup tokens, refused OAuth, keys, passwords, deleted accounts, admin grants, owners changed, and every refused request at the API, the Agent API and the Gateway — hash-chained under SQLite's write lock, append-only by trigger, anchored and signed on the checkpoint schedule; `fox audit`, `fox audit verify`, `fox audit checkpoint`, `GET /api/audit` (admins). docs/policy-errors.md no longer calls TRUNCATE out of scope. Verified in the throwaway VM: `make integration` 157/157 (6 new: the security log), `integration-v2` 303/303 (18 new in §10, 4 in §3), `integration-update` 51/51, `integration-pg-upgrade` 72/72 (export, restore and upgrade carry the new triggers across); `go test ./...` (three new test files), `go vet` for macOS/Linux/Windows. The Windows CI job then failed `TestAnchorSignatures`: it checked the signing key is written 0600, and Windows has no Unix permission bits, so the file reads back 0666. The check now runs only where the bits exist; the key only ever lives on Linux, in the VM or on the host.	`feat/blackbox-data-changes`
22 Sep 2026	K1, K5, K6, F2, F4, E1, G2, G3, I11, L6; known issues	**Branches belong to the account that made them, every database role has its own password, and imports stay out**	`feat/per-user-access`

		of the engine's own network. Batch 2 of the 22 Sep audit (v2). **K1, K5, K6, F2 (G02):** every handler under `/api/branches/{name}` checked only that the caller was signed in, so any account could read, query, suspend or delete any other's branch; agents were listable and deletable by anyone; any key opened any branch through the Gateway. A branch made through the API, the Agent API, MCP, an import, a pipeline or branch-before now records its maker (`branch_owners`), and one rule, `internal/access`, is applied by all four surfaces: an admin (superuser or `db_admin` on main, remembered 30 s) may do anything, the owner may use and manage their branch, every account may use main, and any other branch answers 404, is left out of lists, and is "no branch" at the Gateway before it is woken. Deleting, cutover and checkpointing need owner or admin. A branch made at the CLI, before this, or whose owner was deleted is an admin's; `fox branch owner <name> [email]` shows or hands it over. An agent's scoped key now belongs to the account that asked for the agent, not the first account (N6). **E1, F4 (G10):** the install secret was the password of the superuser, `db_client` and every account's role, and was passed as `-e PGPASSWORD=…` on 28 `docker` command lines. Each login role now has its own HMAC-derived password (`ClientRolePassword`, `UserRolePassword`, next to the existing `AgentRolePassword`); the secret reaches docker through `~/.fox/pg.env` (0600, `--env-file`); a branch cloned before carries the old `db_client` password until `EnsureAppRole` sets it. `TestNoPasswordOnCommandLines` fails if a password is passed as an argument again. **I11 (G12):** imports and pipelines from loopback, private networks or container names are for admins; link-local and cloud-metadata addresses for nobody. **G2, L6 (G16):** password change (signs out other sessions; a form on the API keys page), `fox user list|passwd|delete`, `GET /api/users` and `DELETE /api/users/{id}` for admins (sessions, keys, OAuth sign-ins, pipelines and the admin grant go; branches become an admin's), and emails must be bare addresses of at most 63 bytes, the Postgres role-name limit that let two long emails share one role. OpenAPI documents the new routes. The first v2 run failed 12 checks, all one cause: its deliberately non-admin account worked on branches made at the CLI, which are now an admin's — the suite hands them over with `fox branch owner`. Verified in the throwaway VM: `make integration` 151/151 (23 new in §1d: two ordinary accounts against each other across the API, the Agent API, the Gateway, imports, passwords and accounts), `integration-v2` 281/281, `integration-update` 51/51, `integration-pg-upgrade` 72/72; `go test ./...` (five new test files), `go vet` for macOS/Linux/Windows, `tsc`.	
22 Sep 2026	A5, C1, C4, E2, E3, F2, G4, G5, J1, K1, K5, K6, L2, M2; known issues	**A fresh install is safe by default: nothing listens beyond the machine, the first account needs a setup token, and TLS no longer fails open.** Batch 1 of the 22 Sep audit (v2), after checking each finding against the code. **A5:** the control plane, Gateway and Agent API listened on every interface; they listen on `127.0.0.1` unless `FOX_LISTEN` says otherwise (the Lima and WSL forwarding reach loopback, so macOS and Windows are unchanged). **C4:** the object store (every WAL segment and base backup, a full copy of main), the restore container and debug ports are published on `127.0.0.1`; the HA standby publishes nothing (the Gateway routes to it by container address); `fox start` re-creates an older install's publicly published object store. **E3, G4:** whoever reached the	`feat/safe-defaults`

		sign-up page first became the admin, and sign-up stayed open. The first account over HTTP now needs a setup token that `fox start` prints (0600 in the state directory, `fox setup-token`, deleted with the first account; `fox user create` needs none); sign-up is closed unless `FOX_SIGNUP=open`; the first account is the lowest id, so two sign-ups at once cannot both be admin; ten failed sign-ins per address or email in 15 minutes hold further ones. **G5:** a provider sign-in created accounts with sign-up closed and joined any account with a matching email without asking whether the provider had verified it — an account takeover; now only a verified email joins or creates, and never while sign-up is closed; the state cookie is one-use, `Secure`, `SameSite=Lax`. **E2:** a service that cannot load its certificate refuses to start unless on loopback, and the Gateway refuses a plaintext client from another machine before asking for its key. **F2:** 30 s to authenticate, at most `FOX_GATEWAY_MAX_CONNS` (1000) connections. **J1:** a pipeline run deleted whatever branch had the pipeline's name, and a pipeline source could be a server path — the engine's secrets file included — loaded into a branch the caller could query; runs now replace only branches their pipeline made (`pipeline_targets`), and API sources must be connection strings. **C1, K1, K5:** branch names and agent ids reach Docker, ZFS and btrfs only if they are plain names (`checkName`); on btrfs `x/../main` would have been main's directory. CORS admits the configured origin (and its localhost aliases) only; `FOX_DEV_CORS=1` for the UI dev server. Request bodies 8 MiB, uploads `FOX_MAX_UPLOAD_MB`, header timeout 10 s. **K5, K6:** `FOX_AGENT_SUPERUSER` / `FOX_MCP_SUPERUSER` work only in `-tags insecure` builds. **M2:** CI runs gitleaks on the tree, govulncheck, `npm audit` and a UI type-check. G14's ignored `crypto/rand` error could not produce a weak token on Go 1.26; it is handled explicitly anyway. README gains a Security defaults section. OpenAPI 0.7. Verified in the throwaway VM: `make integration` 128/128 (17 new checks in §1 and §1c), `integration-v2` 281/281 (the MCP superuser switch is ignored by a release build), `integration-update` 51/51, `integration-pg-upgrade` 72/72; `go test ./...` (six new test files), `go vet` for macOS/Linux/Windows, `tsc`, the CI scripts. Not yet run: the new CI jobs themselves (gitleaks, govulncheck, npm audit) — their first run is this pull request.	
22 Sep 2026	Repository; next steps	**A private SSH key committed by mistake is out of the repository.** An external audit (v2, 22 Sep, gap G01) found `id_ed25520` and `id_ed25520.pub` at the repository root: an unencrypted OpenSSH key, not used by any code, added by accident with the 18 Sep repository-rename commit, in a public repository. Both files are no longer tracked, and `.gitignore` now refuses SSH keys and `*.pem` so this cannot recur by a `git add -A`. Untracking does not un-publish the key: it stays in history, so the fix that matters is removing its public half wherever it is trusted (GitHub SSH keys, servers, deploy keys); a history rewrite is optional and the owner's call. The `.gitignore` comment naming a `walg.env` that does not exist is corrected. The audit's other 34 gaps were checked against the code (31 true, 3 partly true, 1 not true in practice) and planned as five batches; the checklist's next steps list them.	`chore/remove-committed-key`
17 Sep 2026	H5, H15, H4, H2; known issues	**Blackbox rules and risk flags judge the statement that runs.** Both matched their patterns against `current_query()` — everything a client sent in one message — so `ALTER TABLE t ADD COLUMN note text`	`fix/blackbox-precision`

`DEFAULT 'drop column later'` was flagged and warned as dropping a column, a comment could trigger a rule, and in `ALTER TABLE a ADD x int; ALTER TABLE b DROP COLUMN y` the harmless first statement was warned or blocked for the second. Postgres gives an event trigger the command tag but not the statement's own text, so `ledger.sql` now has a tokenizer (`bb._sql_statements` : statements split at top-level semicolons, comments removed, standard, `E'…'` and dollar-quoted literals emptied, quoted identifiers kept; byte-based so multibyte text stays linear) and a per-backend position recording which statement is running, keyed by the query and `statement_timestamp()` (the same for every statement in one message, checked). The position lives in a table clients can't write and is moved by a function they can't execute — as a session setting it could have been set to line a blocked statement up with a harmless one. DDL run inside a function, a `DO` block or a trigger is recognised from `PG_CONTEXT` , which also names the statement Postgres actually ran, so it is matched by that text (closing an existing gap: a `DO` block that builds `'DR' || 'OP COLUMN'` is now caught) and never moves the top-level position. Anything uncertain falls back to the whole query, so nothing that matched before stops matching. Previews (`fox policy check` , REST, MCP) match the same way. **H4:** `flag` now records the command `FLAGGED` ; `bb.policy` refuses any action but block/flag/allow (a typo used to switch a block off silently). **H2:** every change to `bb.policy` is recorded in the append-only `bb.policy_history` . **A hang, found while testing and present on main:** a blocked command after a Blackbox write in the same transaction — `BEGIN; CREATE TABLE a(); DROP TABLE b;` — hung forever, because the dblink connection recording the attempt waited for the append lock its own session held, and Postgres can't see that deadlock. The recorders now check `pg_locks` first and skip that one `BLOCKED` entry with a WARNING; the block stands and the policy evaluation is still recorded (`blackbox_id` null, as the contract already allows). docs/policy-errors.md describes what a pattern is matched against. **Checked in the browser**, signed in, on a throwaway branch of the owner's install carrying the new SQL (main and the running servers untouched, branch deleted afterwards): the Policies preview reports `drop-column` for a literal on main but "no rule matches" on the branch, and still reports a real drop in a multi-statement text; with the rule set to block, a statement quoting the phrase ran, a real `DROP COLUMN` and a `DO` block building it from strings were refused with BBX01, a `DO` block creating a table then dropping one returned the guardrail error at once, and the Changes page showed the literal add APPLIED, the drop FLAGGED/drop-column and the refusals BLOCKED. That session found two console bugs, fixed here: **Allow & run again** sent the guardrail override for a policy-rule block, so it could never work — the query API takes `allow_rules` (`bb.policy_allow` , rule ids validated) and the console offers **Allow rule <id> & run again**, confirmed in the browser to drop the column while a guardrail block still offers the plain override — and a command with no result rows printed its success line twice. Verified in the test VM: integration-v2 251 / 251 (24 new §6b checks), integration 53 / 53, integration-update 51 / 51; `go test ./...` with `TestSchemaStatementMatching` , which fails if the revoke or the gate's statement matching is removed (both checked).

17 Sep 2026	M1, G1, H16; known issues	**The integration suites run in a throwaway VM, and running them there found two bugs.** The suites are destructive by design — `integration_test.sh` deletes main's whole Blackbox history to prove tampering is caught, restores main to an earlier point and fails HA over; all of them create accounts, keys and admin grants — and they ran in the same Lima VM as the developer's own install. That install lost its Blackbox history from before 16 Sep and was left with a dozen tables in main and 8 test accounts holding 70 working API keys. Cleaning up afterwards cannot undo a wiped ledger or a restore, and snapshotting main would leave the test run's segments in the real WAL archive, so the suites now run in their own VM. `scripts/test_vm.sh` creates `fox-test` (its own containers, storage, backups and accounts; Docker, ZFS, the test tools and Go checked against go.dev's digest), refuses the names `foxbyte` and `default` that fox would take for the real install, turns off port forwarding so the test stack never lands on the Mac's `localhost:8080`, and marks the VM; `scripts/lib/test_guard.sh`, sourced first by every suite, refuses to run without that mark and fails closed if the guard itself is missing. `make integration`, `integration-v2` and `integration-update` target it. On a fresh VM the first run gave 49 / 53, 226 / 227 and 51 / 51, and the five failures were two real bugs. **G1:** `fox start` launches three services that all create the auth store at once; on a new install two of them lost with `database is locked (SQLITE_BUSY)` and exited, so the control plane and the Agent API were down after the first start. Store creation now runs in `BEGIN IMMEDIATE` (which waits on the busy timeout) and retries a BUSY; `TestConcurrentFirstOpen` reproduced it (6 of 160 opens failed without the fix, none with it). **H16:** since agents connect through the Gateway (K5), the Gateway gave each connection a session of its own, overriding the session an agent branch is created with, so the agent's changes lost it; an agent's branch-scoped key now gets no Gateway session. Re-run on a fresh store: 53 / 53, 227 / 227, no first-start errors in the logs. With the owner's approval the leftovers were cleared from the real install: 70 test API keys revoked, a stale control plane on :8090 stopped, two stopped test containers and the pre-cleanup snapshot branch removed.	`fix/test-isolation`
21 Sep 2026	A3, C8; new M7; B1, D6, M1, M6, I12, index; next steps	**An install no longer dies on an image it cannot pull, and a re-run of `fox setup` finishes an interrupted one. C8:** the first install of v0.1 stopped on `unauthorized` — the image's package was private — with advice to set `FOX_IMAGE_CONTEXT`, which the Mac cannot pass into its VM. The image's build context (the Dockerfile and its entrypoint, 3.5 KB) is now embedded in `fox` (`docker/postgres/context.go`, byte-for-byte the files the release builds from, kept out of the image by a `.dockerignore`), so when a pull fails for any reason the engine builds the image itself; the error now says what that needs (Docker Hub and GitHub). **A3:** provisioning ran only when the VM was created, so an install cut short while apt worked left a VM every later setup called ready. Setup now checks on every run that the ZFS tools are there and Docker is installed and running, and provisions when not. Tried for real from a Mac against the throwaway VM with Docker removed and its reinstall killed mid-unpack: the first version failed and found two things — current Ubuntu's sudo refuses `-E`, and a package killed while unpacking is "reinstall required", which `apt-get install` calls already installed — so the script now passes its environment with `env`, runs `dpkg --`	`feat/install-resilience`

		`configure -a`, and reinstalls anything dpkg marks broken. The second run repaired the VM and brought PostgreSQL 18 up; a third skipped provisioning (1.7 s). **Checklist audit**, every open line checked against the code: M6 still listed the upgrade command and the portable backup as to-do (shipped as D6, D5); B1's `branch create --from` was a to-do though the suite tests it; seven index badges (H4, H5, K2, K5, L7, L8, L9) said Partial where the cards and checklist say Working; M1 did not list `integration-pg-upgrade`; B1 and D6 did not list the export, restore and `pg` commands. The brand system had no feature at all — it is now **M7**. The Import page claimed SQLite files could be imported with the CLI, which no loader supports (I12): the sentence is gone. The A3 card's VM name was the old slug (`foxbyte`; it is `fox`). "Suggested next steps" had gone stale and now lists what is really open. Verified in the throwaway VM: `make integration` 111/111 (§11b: an image tag on a `.invalid` registry, started away from the checkout, is built from the copy inside fox and runs main), `integration-v2` 281/281, `integration-update` 51/51, `integration-pg-upgrade` 72/72; `go test ./...` (five new tests), `go vet` for macOS/Linux/Windows, the CI scripts. The Windows CI job then failed one of the new tests: it checked that the embedded entrypoint is written executable, and Windows has no executable bit — a file written as 0755 reads back without it. The check now runs only where the bit exists; the context is only ever built on Linux, in the VM or the WSL distro.	
21 Sep 2026	H1, D5, D6; known issues	**A restart no longer writes to the Blackbox, and the upgrade's two bugs have tests of their own. H1:** every `fox start` re-runs the Blackbox's install script, and since GRANT/REVOKE capture was added (PR #13) that recorded the script's own nine `GRANT`/`REVOKE`/`ALTER DEFAULT PRIVILEGES` statements as schema changes with no actor — per running branch, per start — so the Blackbox page showed maintenance as if someone had changed privileges. The script's header already promised that re-installing never fires the triggers on its own DDL: they are dropped first. But they were re-created before the privilege block rather than after it. They are now created last. Capture of users' changes is unchanged, and no flag was added to switch capture off: a client can set custom settings, so one would have been a way to hide a change from the Blackbox. Entries recorded before the fix stay, because the table is append-only and its chain must keep verifying. Integration-v2 §9 checks a stop and start adds nothing to main's or a branch's Blackbox, and that a user's change afterwards is still recorded as that user. The upgrade suite now requires the Blackbox to be exactly the same after an upgrade and after a rollback — nothing lost and nothing added — where before it could only check that nothing was lost. **D5:** the retake-on-change logic of an export is its own function, `dumpAtStableHead`, and `TestDumpAtStableHead` drives it with a head that changes during the first dump, one that never settles, and failures on either side. **D6:** rollback after a branch was made on the upgraded install is covered by `TestClonesFirst` and the suite's rollback section. The H1 card's note that capture skips "schema `fox`" was a rename leftover; it is `bb`. One test fix: integration §1's "the first account may override the guardrail" assumed an empty account store but only cleared the config file, so after a run that was cut short — mine, stopped to pick up this change — the account it created was not the first and got no grant. §1 now clears the account store as well; seeded with a leftover account, the suite passes. Verified on the final	`feat/pg-upgrade`

		code in the throwaway VM: `make integration` 108/108, `integration-v2` 281/281, `integration-update` 51/51, `integration-pg-upgrade` 72/72; `go test ./...`, `go vet` for macOS/Linux/Windows, the CI scripts.	
21 Sep 2026	new D5, D6; M6, D1; known issues	**A portable export, and a supported way to move an install between PostgreSQL majors.** A fresh install runs 18, but an install made on 16 had only "uninstall and start again". **D5** `fox backup export` writes one verified archive to the user's disk — per branch a `pg_dump`, the branch's roles (each branch is its own cluster), and the Blackbox head, with a `SHA256SUMS` over everything — and `fox backup restore` puts a branch of it back into a new branch, of the same or a newer major, checking the Blackbox arrived exactly. **D6** `fox pg upgrade` shows the plan (every branch, what is lost and kept, and the PostgreSQL changes crossed — written from each release's migration notes, with probes that put the ones found in the user's own data first), refuses with what to do instead when HA is on or a Blackbox does not verify, asks, takes an export, then dumps every branch with the new `pg_dump` and reloads it into a new cluster. Nothing old is deleted: every branch moves untouched into a holding area outside the branch namespace, so any failure, and `--rollback`, is a move back; `--finalize` deletes it. **D1**: a new cluster restarts WAL numbering, so its segment names would collide with the old cluster's in the bucket and a rollback would find its own archive overwritten; each upgraded cluster now archives under its own prefix, recorded in main's dataset so it follows the data like the image does. The new suite, `make integration-pg-upgrade`, builds a real 16 install and passed 69 / 69. Its first run caught two bugs before they shipped: rollback destroyed the new main before a branch cloned from it, so `zfs destroy -R` took the clone and the next step failed on it — clones now go first; and an export read the Blackbox head after `pg_dump`, so a schema change landing mid-dump would have made a good export fail its own restore check — the head is now read on both sides of the dump. It also surfaced an older problem, fixed in the next row: every `fox start` appended nine unattributed `GRANT` / `REVOKE` entries to the Blackbox. Verified: `go test ./...` (22 new unit tests), `go vet` for macOS/Linux/Windows, and the suite in the throwaway VM.	`feat/pg-upgrade`
21 Sep 2026	M6	**PostgreSQL 18 passes every suite.** The move to 18 was held at Partial until the engine's unchecked dependencies on Postgres behaviour had run on a real 18 cluster. A new throwaway VM ran all three suites against `PostgreSQL 18.6`: `make integration` 108/108, `integration-v2` 277/277, `integration-update` 51/51 — 436 checks, no failures, no code changes needed. That covers the gate the plan listed: wal-g (v3.0.9, unchanged) archiving, base backups and fetches; the `recovery.signal` PITR protocol and branch-before's recovery reset; HA failover and failback, including the `pg_controldata` labels failback parses; the Blackbox's `PG_CONTEXT` matching, event-trigger tag lists, advisory-lock decoding, `dblink` recording of blocked attempts and the hash chain; and import cutover. The `:18` image was not yet published, so the engine built it from the Dockerfile with `PG_MAJOR=18` — the fallback path works as designed. The cluster has data checksums on and `uuidv7()`, and `pg_settings` still omits custom placeholder settings, as the 18 Sep reasoning assumed. M6 moves to Working.	`feat/postgres-18`

21 Sep 2026	M5, A2	**The Windows distro image is published zstd-compressed.** It is the one large download of a Windows install — about 1.7 GB of Ubuntu, Docker and the preloaded container images, 685 MB as gzip. It is now `zstd -19 --long=27`: the long window finds the libraries the rootfs and the Postgres image both carry, and zstd unpacks fast enough that the download saved is not spent again waiting. The build compresses the same bytes with gzip too and prints both sizes, so the saving is measured in every build log rather than claimed here; typical for this kind of content is a quarter to a third smaller. `wsl --import` and the `tar.exe` built into Windows are only dependable with gzip, so `fox.exe` unpacks the image itself with the pure-Go `klauspost/compress` decoder (BSD-licensed; the one new dependency) into a plain tar beside the download, imports that, and removes it — writing to `.partial` and renaming at the end, so an interrupted unpack cannot leave a truncated tar that the import would take for a whole distro. The installer tries the zstd image, then the gzip one older releases published, then the Ubuntu rootfs, and skips an image the release does not list rather than downloading a 404. Verified: five unit tests (a round trip through a 128 MiB window, a truncated download refused with nothing left behind, gzip passed through untouched, the build's `--long` held within the decoder's limit, zstd preferred), Pester updated, `go vet` for macOS/Linux/Windows, the CI scripts. **Not yet run on a real Windows machine**, which is where the unpack and the import meet for the first time. The Windows CI job caught one mistake in the new Pester check: its expected text was a double-quoted PowerShell string containing `$name`, which PowerShell interpolated (to nothing) before matching, so it looked for `foreach ( in …)`; it is single-quoted now, and no other assertion in either Pester file puts a variable inside double quotes.	`feat/postgres-18`
21 Sep 2026	M6, A2, A4, M5	**The PostgreSQL version is written down for users and maintainers, and `fox` is a command on Windows again.** The README now says a fresh install runs PostgreSQL 18, that an install keeps its own major, and that a base backup does not survive a major change while a `pg_dump` does. New `docs/postgres-versions.md` covers both readers: which major you run and how to check it, the manual route to a newer major until `fox pg upgrade` exists (and what does not come across), and the maintainer's checklist for moving FoxByte to the next major — change `PGMajor`, let `TestPostgresMajorIsInStepEverywhere` name the other files, read the release notes against the ten places the engine relies on Postgres behaviour, run the three suites, write the user-facing change list, release. Writing its Windows commands turned up two rename leftovers. **A2:** the installer saved the launcher as `bb.exe` (the rule that renamed the SQL schema to `bb.*` also caught the launcher's file name), so `fox` was not a command after installing; the installer only worked because it runs the file by its full path. It is `fox.exe` again, a leftover `bb.exe` is removed, and the internal temp and rollback names the same rule had produced (`/tmp/bb.new`, `.bb.new`, `prev/bb.exe`) now follow the command or name nothing. **A4, M5:** the WSL distro is `fox` (`brand.VMInstance`), but the Windows guide, the handoff notes, the Guide page and a comment called it `foxbyte`, so `wsl --unregister foxbyte` in the uninstall section would have removed nothing, and a troubleshooting line said `wsl -d appdb`. The README's "not a vector database, despite the name" line, which only made sense for the first name, lost its	`feat/postgres-18`

		last four words. Verified: `go test ./...`, `go vet` for macOS and Windows, the TypeScript check, and the installer's line endings (CRLF throughout, the generated block in step).	
21 Sep 2026	M6, M4, M5, C8, A4, I1; known issues	**A fresh install runs PostgreSQL 18, and an install made on 16 keeps running 16.** The engine ran PostgreSQL 16 (Sept 2023); 18 is current, and it brings things this product's story rests on — asynchronous I/O for the hot path, page checksums on by default for a database that sells tamper-evidence, `uuidv7()`, and the `pg_upgrade --swap` machinery that makes the next major cheaper than this one. Nothing was in production, so this was the cheapest moment the move will ever have. The version was a free literal in four files that nothing kept in step; it now lives in `internal/branch/images.go` (`PGMajor`, `SupportedPGMajors`, `PostgresImageFor`, which also gives `brand.ImageRepo` its first caller — the full image name had been hardcoded, so a rebrand sweep edited it as text), and `TestPostgresMajorIsInStepEverywhere` holds the Dockerfile default, the release workflow's matrix and the Windows preload to it. **The image follows the data, not the binary:** the engine reads `PG_VERSION` from main's data directory and runs that major's image, so `fox update` onto this release leaves an install on 16 running 16 — `fox update` promises not to touch data, and moving a cluster between majors is a migration. That needs the old image to stay pullable, so the release workflow now builds one image per supported major from one parameterised Dockerfile (`ARG PG_MAJOR`), and a local build passes the major its tag names. Before every `docker run`, a branch whose `PG_VERSION` does not match the image's major is refused with both versions and the way out, where Postgres would have failed inside the container with "database files are incompatible with server", in a log nobody reads. Two smaller things fell out of reading the paths: the Windows setup probed a pre-16-Sep legacy image name to decide whether the preload was already loaded, which a newer distro never has, so every re-run loaded the 685 MB tarball again; and the published image now carries `org.opencontainers.image.source`, so its package links itself to the repository instead of starting out unlinked and private — which is what stopped the first install of `v0.1` with `unauthorized`. **Not yet proven:** the engine depends on Postgres behaviour no compiler checks (wal-g's base backups and restores, the `recovery.signal` PITR protocol, HA failback parsing `pg_controldata` by its English labels, the Blackbox's `PG_CONTEXT` string matching and event-trigger tag lists), so M6 stays Partial until `make integration`, `integration-v2` and `integration-update` have run on 18. No command yet moves an install from 16 to 18; that, and a portable `fox backup export` to take first, are the next batch. Verified here: `gofmt`, `go build`, `go vet` for macOS/Linux/Windows, `go test ./...` with six new unit tests, and the drift test checked to fail when the Dockerfile default is set back to 16.	`feat/postgres-18`
21 Sep 2026	M3; B1, D1, H4	**Two guard tests only failed on Windows, and both were about how a file is written rather than what is in it.** The first green run of the merged branch failed on the `windows-latest` job alone. **Paths:** `TestDurableNamesAreNotSpeltOutTwice` exempts the package that defines the durable names by matching `internal/branch/` in the path, and the directory walk yields `..\..\internal\branch\branch.go` there — so the exemption matched nothing and the test reported all nine	`fix/windows-ci`

		definitions in the package that owns them as duplicates. The walk now hands every rule a forward-slash path, with the separator replaced explicitly rather than by `filepath.ToSlash`, so `TestDurableNameExemptionsReadWindowsPaths` can feed it the Windows spelling and fail on every OS if it regresses — proved by reverting the helper and watching it fail on macOS. **Line endings:** `TestGeneratedFilesAreInStep` compares each generated file byte for byte with what `brand.json` produces, and Windows checks text files out with CRLF unless `.gitattributes` says otherwise; `web/src/brand.ts` was the one generated file with no rule, so a correct file looked like drift. The TypeScript sources are pinned to LF with the shell scripts, Go, SQL and the workflows; `install.ps1` stays CRLF and the generator keeps it that way. Reproduced locally first by converting `brand.ts` to CRLF, which failed with the same message. Neither is a defect in the engine: both tests were right to be strict and wrong about the platform.	
21 Sep 2026	E3, L1–L11 (new L11), L4, L6, L5, L9; known issues	**The web console was redesigned around a FoxByte theme, and nothing mints a key behind the user's back any more. E3:** the first `fox start` used to create an account called `local@foxbyte`, mint a key for it and cache it in `~/.fox/config` in plaintext — a credential nobody asked for, on an account nobody owned. Start now creates neither: it prints the connection string with `<API_KEY>` left to fill in and says how to make one, and the first account a person creates (web sign-up or `fox user create`) takes the admin grant on main through the new `auth.Store.OnFirstUser` hook. **L11:** the stylesheet was split in two — tokens (colour, a 1.2 type scale, a 4px space scale, radii, elevation, motion) and components that refer only to tokens, so the product can be reskinned from one block. The palette is the brand: an ember gradient (`#ff7a2f` → `#ffc24a`) on graphite in the dark theme, the same ember darkened to `#d2570c` for the light one; filled controls take `--accent-ink` rather than white, which was the reason the old orange buttons were hard to read. The mark is a fox head built from the same two angles as the branch graph (`components/brand.tsx`), drawn from CSS variables so it follows the theme, and shipped as `web/public/favicon.svg` for the tab. **L4:** the console is three cards instead of a loose stack — schema rail, a titled query card with its own toolbar, and a result with a status bar above the grid (command, rows, columns); the editor follows the theme instead of being a dark box on a white page. **L5, L9:** every filter moved into one bordered bar above its table, and page-level controls (branch, Refresh, Verify, Add rule) into a page header, so a page reads title-then-controls-then-content. **L6:** a new key is shown with the connection string already assembled around it, which is what the E3 change asks a person to do next. Tables share one style: a sticky quiet header, roomy cells, no zebra stripes, action columns that stay narrow and result grids that do not. The README opens with the logo (a light and a dark PNG, rendered from `docs/brand/logo.html`, which GitHub picks between by the reader's colour scheme). **What the page leads with changed** (5 Oct 2026, from an outside review). The hero said FoxByte gives “branches, time-travel, a database per AI agent, and high availability” and the six cards were branching, time-travel, agents, scale-to-zero, HA and the gateway — **the Blackbox was not on the landing page at all**, which is the one thing a competitor does not have. The first three cards are now the record, the agent and the policy gate; HA leaves the	`feat/foxbyte-ui`

		grid because the README itself calls it a single-VM demonstration and a landing page should not lead with its least proven claim. The terminal beside the hero demonstrated `fox ha failover` and now shows a `DROP TABLE` refused by the gate and the Blackbox accounting for it — the only line on the page where the product says no, which is the thing being sold., and its three console screenshots were retaken on the new theme — they still showed the purple UI of two names ago. One thing outside the UI came with it: the brand generator rewrote `install.ps1`'s generated block with LF line endings, so a file `.gitattributes` keeps as CRLF came back mixed — the block now follows whatever the file already uses. Verified: `gofmt`, `go build`, `go vet` for macOS, Linux and Windows, `go test ./...`, the TypeScript check and the production web build, plus every page driven in the browser pane in both themes and at phone width against a canned control plane.	
21 Sep 2026	All; B1, D1, H4, L4; known issues	**The product is FoxByte and the command is `fox` — and the names it stores no longer carry the brand, so the next rename costs one file.** Renamed from OxynDB, which was renamed from VectoraDB, and each of those meant a fresh install because the name was written into the data. The names now split in two. What people read comes from `brand.json`: the product, the command, the environment variables (`FOX_*`), the state directory (`~/.fox`), the repository and module (`github.com/thefoxbyte/foxbyte`), these documents, the UI. `cmd/brandgen` turns that file into what cannot read it at runtime — Go constants, the shell library the suites source, the web UI's constants, and the generated block in each installer, since those are fetched standalone from GitHub — and `TestGeneratedFilesAreInStep` fails the build if any of them drifts. What is written into a database or onto disk is now brand-free and frozen: schema `bb`, roles `db_client` and `db_admin`, superuser `dbadmin`, database `appdb`, SQLSTATEs `BBX01` / `BBX02`, key prefix `key_`, cookies `dbengine_*`, pool `dbpool`, network `dbnet`, containers `pg-*`, object store `objstore` with volume `objstore-data` and bucket `wal-archive`, anchor format `ledger-anchor/1`. `TestNoRetiredProductNames` fails on any retired name in the repository, and `TestStoredNamesAreBrandFree` fails if the product is named in the SQL the engine installs. Renaming also stopped being able to break behaviour: the database used to raise `'OxynDB guardrail: … is blocked by policy'`, the SQL console matched that prose with a regular expression and a unit test asserted it — three places coupled by a brand string, where renaming one silently stopped the console recognising a blocked change; the message carries no product name now. The web wordmark was spelt across tags, which no text sweep catches, and is derived from the brand instead. Users keep what they have: an environment variable set under a retired prefix is still read, a state directory from a retired name is moved rather than abandoned (losing it left the object store's credentials out of step with its data), and a VM or distro from a retired name is still found. **Fixed on the way:** `setup` hung for ever when another project's container was called `minio` (the object store has a name of ours now, and the bucket step gives up with a message); containers are cleaned by label rather than by name prefix, so one left by an older name cannot keep holding ports and mounts; and renaming the object store while `AWS_ENDPOINT` still said `http://minio:9000` broke archiving, backups and restore at once — caught by the suites, now derived from one constant	`chore/foxbyte`

		and pinned by a test. **New:** `fox uninstall` lists what it will remove, asks, and removes it — including binaries, VMs and state directories left by retired names. Verified: `go test ./...`, `gofmt`, `go vet` for macOS, Linux and Windows, the TypeScript check and web build, every CI script, and the three integration suites in a throwaway VM, which gained a check that a foreign `minio` container does not break `start`.	
18 Sep 2026	A1, A2, A8, A9, M4, M5	**Every reference names the repository exactly: `github.com/thefoxbyte/foxbyte`.** The rename wrote the organisation in lower case and the repository with a capital D, but the organisation was created as `FoxByte` and the repository as `foxbyte`. GitHub resolves any casing, so the install scripts, `fox update`'s release lookups and the links all kept working in a browser, which is exactly why it would have gone unnoticed — but the Go module path is case-sensitive, so it now matches the repository too (`module github.com/thefoxbyte/foxbyte`; the capital letter moves it ahead of other modules in import order, and `gofmt` re-sorted the two files where that mattered). Changed: 37 repository references (release downloads, the raw-install URLs, the GitHub API path and the `FOX_REPO` defaults) and 65 module-path references. Deliberately unchanged: container images stay `ghcr.io/thefoxbyte/…`, because image names must be lowercase, and `~/.fox/auth.db` is the account database's file name, not the repository. `TestNoOldProductNames` now also fails on the repository written with the wrong casing, while allowing the lowercase image names (checked by planting each form). Verified: every CI step locally — `gofmt`, build, vet, `go test ./...`, the release-checksum, release-asset, installer and checklist scripts — and the `fox update` suite, which drives the release lookup against a fake GitHub.	`fix/repo-path-foxbyte`
18 Sep 2026	All; known issues	**The product is FoxByte and its command is `fox` — renamed from VectoraDB and `vdb` before it was ever published.** Nobody else ran it yet, so every identifier changed at once rather than leaving the old names behind as internals to explain later: the brand (FoxByte), the command and binaries (`fox`, `fox-verify`, release assets `fox-<os>-<arch>`), environment variables (`FOX_*`), the state directory (`~/.fox`), the Go module and repository (`github.com/thefoxbyte/foxbyte`, `thefoxbyte/foxbyte`), the SQL schema, settings and roles (`bb.*`, `bb.actor`, `db_client`, `db_admin`), the policy SQLSTATEs (`BBX01`, `BBX02`), the API-key prefix (`key_`), containers (`pg-<branch>`), the ZFS pool, Docker network, Postgres superuser and database, WAL bucket (`wal-archive`), the anchor format (`ledger-anchor/1`), the Lima and WSL instance names, and these documents (`FOX_*`). Feature IDs are unchanged. The rename was scripted in a fixed order, most specific name first, over tracked text files only; `go.sum` and the npm lock file were left out of the sweep because their base64 hashes can hold the old three letters by chance, and the lock file's own package name was changed by hand. No length-dependent code was affected — container names are parsed with `TrimPrefix`, not by position, though the new prefix is a character longer. `TestNoOldProductNames` now fails the build if an old name reappears anywhere in the repository, except on a line that records history by saying it was renamed from one (checked by planting each old form and watching it fail). What this breaks by design is anything outside the repository still holding old names: an existing install (its pool, containers, schema, bucket	`chore/rename-foxbyte`

		and keys) is replaced by a fresh one, MCP client configs need `fox mcp` and `FOX_API_KEY`, and releases and images start again under the new names. Verified: `go build`, `go test ./...`, `gofmt`, `go vet` for macOS, Linux and Windows, the web build, all six PDFs re-rendered, and the three integration suites in a test VM created under the new name.	
18 Sep 2026	L4; known issues	**The console shows a uuid as a uuid.** Reported from the UI: the `id` column of every table in the user's own schema was displayed as a list of sixteen numbers. pgx decodes a `uuid` column into `[16]byte`, and that is what JSON makes of a byte array — `[195,64,152,78,…]` where `c340984e-87bf-4006-a013-72aca6fc7d77` belongs. Reading the whole type surface rather than that one column found three more of the same kind: an interval arrived as `{"Microseconds":7200000000,"Days":1,…}`, an `inet` carried its JSON quotes into the cell, and a `bytea` was turned into whatever its bytes looked like as text. Instead of formatting each type by hand, the console now asks pgx's own type map for Postgres's text representation (`Map.Encode` with `TextFormatCode`) — what psql would print — for any value decoded into a byte array or a struct, which covers the geometric types, ranges, `macaddr`, time of day and hstore in the same stroke, and future ones too. Array members are rendered by their own element type, so a `uuid[]` is a list of uuids rather than a list of lists of numbers. Everything that already read well is untouched by construction: scalars, timestamps, numeric, jsonb and arrays of scalars take the paths they always did, and only a real `bytea` column is hex-encoded, since pgx also returns `[]byte` for a type it has no codec for (a custom enum arrives as the bytes of its text form) and hex would make those unreadable. Two flaws in the suite came out of running it, both mine from earlier batches and both fixed rather than retried: the new bytea assertion passed its literal through as text (`decode('0102','hex')` is unambiguous), and §5b's filter counts used a fixed table name, so a second run on the same install counted the first run's entries as well — it now names its table per run, since the ledger is append-only and nothing clears it between runs. Verified: `go test ./...` with three new unit tests covering both the fixed types and the unchanged ones, `go vet` for macOS, Linux and Windows, `gofmt`, the TypeScript check, new integration-v2 §6 assertions over the REST API (267 / 267, with `make integration` 91 / 91), and in the browser against the real install — the reporter's own `app_user` table now lists `0955eb65-85ea-435d-8f48-13c51921ab4f` and its neighbours in the ID column.	`fix/console-uuid-cells`
18 Sep 2026	H3, K6, L5, K2; known issues	**An agent can no longer be recorded as a person, and the MCP server acts as an account instead of trusting whoever starts it. H3:** the Blackbox's claim is that it says who changed what, and `actor` was already non-forgeable — the Gateway logs each client in as its own role, and `session_user` survives `SET ROLE`. `actor_kind` was not: it was read from `bb.actor_kind`, an ordinary session setting, so `SET bb.actor_kind = 'human'` was enough for an agent's changes to be recorded as a person's. It is derived from the login role now — an `agent-<id>` role is an agent, a per-user role is a person (one named for an email address is a person even if the address begins with "agent-"), and only the engine's own shared roles, which have no identity of their own, still take the injected value, set by engine code rather than by a client. A mechanism that looked promising was checked and dropped: `pg_settings` does not list custom placeholder GUCs at all in	`feat/attribution-mcp-auth`

		PostgreSQL 16, so a setting's connection-time value cannot be read back to tell the Gateway's own injection from a later `SET` — deriving from the role needs no such trick. `tool` and `session` stay client-declared by design (an `application_name` is whatever a client sends) and the Blackbox page now labels them **declared**, so the page distinguishes what the database attributes from what the client says about itself. **K6:** `fox mcp` took no credential — any client that could start it got all sixteen tools, and its changes were recorded against the shared role rather than an account. It now requires an API key (`FOX_API_KEY` , or `--key` for trying it by hand) and acts as that key's account, with no unauthenticated path: started without one it prints how to make a key and exits. `run_sql` records the account as the actor (`ClientQueryTextAs`), and a key scoped to one branch — the kind the Agent Branch API issues — is held to it: every call is pinned to that branch, an absent branch argument included, naming another branch is refused, and `create_branch` , `delete_branch` , `list_branches` , `blackbox_diff` and `branch_before_change` are refused outright. `docs/mcp.md` , the README, the Guide and the Docs page carry the key in their client configs. Two test flaws surfaced on the way and were fixed rather than retried: the suite's MCP helpers passed no key, and §3b read a count from main five seconds after a restart, comparing an empty string as an integer — it now waits for main over TCP (a socket probe answers from the temporary initdb server, as `waitReady` documents) and for a numeric count. Verified: `go test ./...` with three new unit tests, `go vet` for macOS, Linux and Windows, `gofmt` , the TypeScript check, the API drift check, `make integration-v2` 271 / 271 and `make integration` 91 / 91 in the throwaway VM (11 new checks, including both forgery attempts) — a run in between failed 34 checks on the test VM's own Docker daemon (`setns … No such file or directory` from `docker exec` against containers mid-restart, after a day of back-to-back suites); restarting the VM cleared it, which is worth knowing before reading such a run as a regression, and on the real install: on a throwaway branch upgraded to the new function, a per-user role that set `bb.actor_kind='agent'` and `bb.actor='agent-imposter'` was still recorded as `h3tester@example.com` /human, and an `agent-` role that claimed `'human'` and a chief executive's address was still recorded as `agent-h3fake` /agent — both read back from the Blackbox page, with the declared labels on tool and session. The branch, its test roles and the spare server were removed afterwards.	
18 Sep 2026	D2, D3, D4, L3, K2; known issues	**A restore can reach any point the archive covers, backups follow the primary, and a promoted standby archives WAL.** **D3:** `fox restore --to` always fetched the LATEST base backup, whatever point was asked for, so a target earlier than the newest backup could not be reached at all — recovery would start after it. The backup is chosen in Go now: the target time is parsed, the listing is read from a throwaway container, and the newest backup of this database (same system identifier) that had finished by then is fetched; a target no base backup precedes is refused, naming the oldest one rather than quietly restoring something later. The recovery script moved into the binary (`restore_pitr.sh` , run with `bash -c` exactly as `BranchBeforeEntry` runs its own), so installs whose image predates this get the fix from `fox update` alone, without a rebuilt image; `--to latest` is unchanged. **D2:** `fox backup create` backed up `pg-main` even after a	`feat/durability-gaps`

		failover, when main is stopped and the promoted standby holds every write — it follows the primary pointer now and says when the primary is not `pg-main`; `fox backup list` and `fox status` no longer depend on `pg-main` running, since backups live in object storage rather than in any one container. **D4:** the standby ran with no object-storage environment and no archive settings, so once promoted it archived nothing — everything written while main ran on it was outside both backups and point-in-time restore. Its container now carries the primary's wal-g environment and `archive_mode=on`, which archives only outside recovery, so nothing is double-pushed while it streams and the archive continues on the new timeline the moment it is promoted; a standby created before this change is named as unarchiving when `fox up` brings it back, because `archive_mode` can only be set at startup. **L3, K2:** time travel had no surface in the web at all, so the Dashboard gained a read-only **Base backups** panel — the earliest point reachable, each backup's time and size, and a copy-ready `fox restore --to` — over a new `GET /api/backups`; restoring stays a CLI command, as it needs a host port and leaves a container behind. Verified: `go test ./...` with four new unit tests, `go vet` for macOS, Linux and Windows, `gofmt`, the TypeScript check, the API drift check, `make integration` 91 / 91 (15 new checks, including a restore that reaches a write made after a failover) and `make integration-v2` 261 / 261 in the throwaway VM, and the panel driven in the browser against the real install — where the command it offered for the oldest of 61 backups restored the database to 24 Aug 2026, choosing the right base backup and coming up without the `fox` schema, which did not exist then.	
17 Sep 2026	L5, B1, G5, I8, C1, C2, C3; known issues	**The Blackbox page filters and verifies, branches can be created from a branch, a continuous import can be watched and cut over from the browser, and `fox vm` exists. L5:** the page had four of the API's eight filters and never showed the statement. It now filters by table, risk (drop, drop-column, type-change, security-definer, policy) and a since/until date range as well, fetches `with=session`, expands a row to its session, branch and full statement, and has a **Verify** button that reports the chain intact or names the first broken entry. Two defects came out of driving it in the browser rather than by API: typing in the table box lost the last keystroke, because the loader skipped a request while one was in flight (a request sequence number now makes a filter change always reload and drops stale answers), and the expanded statement was dark text on a dark panel, because the global `pre` rules only colour `pre code`. **C1, C2:** `fox branch create <n> --from <branch>` and `POST /api/branches {"name","from"}` (404 when the parent is unknown) copy any branch, with a parent picker on the Dashboard. The new integration checks immediately failed on a real bug: `zfs destroy` removed only the `main@for-<branch>` snapshot, so a second create from the same non-`main` parent hit the leftover and returned 409. Delete now also destroys the clone's own origin snapshot — matched by prefix and `@for-<branch>` suffix so it can never remove another branch's — and clone re-takes a stale snapshot. **I8:** continuous imports are readable and finishable without the CLI — `GET /api/replication`, `GET /api/branches/{name}/replication`, `POST …/replication/cutover` (409 unless the branch really is replicating), reading `pg_subscription`, `pg_subscription_rel` and `pg_stat_subscription` — behind a **Continuous imports** panel on the Import page with	`feat/blackbox-page-gaps`

		tables copied, the last message from the source and a **Cut over** button. Watching that panel exposed the worst bug of the batch: the import died about two minutes in, every time. The Gateway's reaper suspends a branch idle with no client connections, and a subscription's apply worker is a background worker, so a replicating branch looked idle — replication stopped silently and the branch vanished from the panel. The reaper now leaves a replicating branch running. **G5:** OAuth defaulted to `http://localhost:8080` and `http://localhost:5173`, neither of which a default install serves, so callbacks and the post-login redirect missed unless both URLs were set; the public URL now defaults to `https://localhost:8080` and the web origin to the public URL, and the session cookie follows the origin (`Secure` on https; `SameSite=None` only for a console on another origin, `Lax` otherwise). **B1:** `fox vm status` reports the engine VM's state, CPUs, memory and disk, and `fox vm shell` opens a shell in it — Lima on macOS, WSL on Windows, and on Linux both say there is no VM. Verified: `go test ./...` (six new unit tests), `go vet` for macOS, Linux and Windows, `gofmt`, TypeScript type-check, the API drift check (53 paths), `make integration` 76 / 76 and `make integration-v2` 261 / 261 in the throwaway VM, `fox vm` by hand on this Mac, and the whole batch driven in the browser against a spare control plane on the real install (see the test baseline).	
17 Sep 2026	A8	**A new release shows on the next `fox start`, not up to 6 hours later.** `fox update --check` reported v0.8.7 while `fox start` said nothing. The start-time check remembered its answer for 6 hours — "up to date" included — and this Mac's cache held "up to date" from 15:12, before v0.8.7 was published, so every start in that window skipped GitHub. Only a found release is remembered now, so the notice still prints without a request once it has appeared; "up to date" and failed checks are not, and caches written earlier that hold "up to date" are ignored. Every other start asks for the releases list, conditional on the last ETag, so an unchanged list is a 304 that GitHub does not count against the rate limit. The unit tests' fake GitHub sent a fixed ETag and so answered "not modified" right after a publish; its ETag now follows the list, as GitHub's does. Verified: `go test ./...` with the new `TestNewReleaseShowsOnNextStart`, `go vet` for macOS, Linux and Windows, and against real GitHub using copies of this Mac's stale caches — v0.8.7 was found in 577 ms and then remembered.	`fix/test-isolation`
17 Sep 2026	L4, H8, L9; known issues	**Dropping a table from the console works, and the console lists your tables apart from FoxByte's.** A user could not drop a table from the SQL console however they tried, even with `SET bb.allow_destructive=on` and a restart. Reproduced on a throwaway branch: as `db_client` the override was refused; as the same account's own role — already in `db_admin` — the guardrail let it through. The console logged every user in as the shared `db_client` role, which is never in `db_admin`, so nobody could override from it; and each run is a new connection, so a `SET` in one run was gone by the next, and the API refuses two statements in one run. Now the console logs in as the signed-in user's own role (acting as `db_client`, so data access and ownership are unchanged, as through the Gateway), and `allow_destructive` on the query request applies the override to that run. In the page an **Allow destructive changes** switch sits by Run and turns itself off after one run; a blocked change shows an **Allow & run again** button to admins	`fix/console-permissions`

		and the grant command to others. Granting and revoking moved into the web: `GET|POST /api/branches/{name}/admins` and `DELETE …/admins/{email}` , with a **Who can override** panel on the Policies page — only an admin of a branch may change the list there, an unknown email is refused, and the last admin can't be removed. The schema list now hides FoxByte's `fox` tables by default, behind a **Show system tables** button (not remembered between visits, and not revealed by the filter); reload shows that it is working, refreshes the open table too, and reports errors instead of showing "No tables yet"; the list refreshes itself after DDL. Found along the way and recorded, not changed: about twelve tables in this install's `public` schema were left by test runs. Verified: `go test ./...` , `go vet` for macOS, Linux and Windows, TypeScript type-check and build, the API drift check, new integration-v2 §6 assertions (not yet run in the VM), and a live run against a spare control plane on the real stack (as above; temporary key, server and branch removed afterwards).	
16 Sep 2026	K5, K6; known issues	**An agent can now reach the database it is given.** The Agent Branch API handed back a DSN built from the container's address, which only resolves inside the VM — so on macOS and Windows an agent running on the host was given a database it could not open, while the README said "the agent connects to that dsn". Publishing a host port was the wrong fix and is deliberately refused by the design (`startContainer` : a direct connection would bypass the Gateway, its key, TLS and the Blackbox attribution), so the agent now connects the way every other client does — through the Gateway — with two additions that keep it no more privileged than before. Its key is scoped to one branch (new `api_keys.scope` , with a migration, since the schema runs with CREATE TABLE IF NOT EXISTS and every existing install already has that table): `Authn` refuses a scoped key outright, so it reaches neither the control plane nor the Agent API, and the Gateway opens only the branch it names. And the Gateway logs it in as the branch's own `agent-<id>` role, whose password is derived from the install secret with HMAC rather than stored or passed between processes, so `session_user` is still the agent and the recorded actor is still non-forgeable — `EnsureUserRole` is deliberately not called on that path, as it would reset the role's password and invalidate the DSN already issued. The key is revoked when the branch is deleted, but not on a reset, where the branch and the scope survive. Separately, the cap, the listing and the TTL reaper used `docker ps` , so a suspended agent branch — the usual state, since the Gateway suspends idle branches — escaped all three; they now use `docker ps -a` , and listings report `suspended` rather than claiming `ready` . Verified: `go test ./...` , `go vet` for macOS, Linux and Windows, six new unit tests (derivation is deterministic per branch and is not the install secret; the DSN parses, carries `sslmode=require` and omits the key in listings; the scope check rejects another branch, an empty target and a prefix match; a scoped key fails `userFromRequest` while an account key still passes; a store created in the pre-scope shape gains the column on open), plus new integration assertions that the DSN routes through the Gateway, that the agent's key cannot open `main` and is refused 401 by both HTTP APIs, and that it stops working once the branch is deleted. `agentDSN` and `randomPassword` had no callers left once the DSN changed, so both were removed with their tests, keeping the "never the superuser"	`118363d` (PR #39)

		assertion in `TestAgentGatewayDSN`. The VM suites still need a run.	
16 Sep 2026	K2, L7, K6; known issues	**The API description now matches the code, and the MCP server is documented.** The spec is what client generators read, and it disagreed with the handlers: `/api/import` was documented as `{from, as}` while the code reads `{source, target, continuous}` and answers with an SSE stream; branch creation was documented 200 where the handler returns 201; the ledger's `since` / `until` filters, the multipart fields of `/api/import/file`, `/api/openapi.yaml` and all seven `/auth/*` routes were absent; and the `/blackbox` aliases existed only in prose, so a generated client got `/ledger` alone. All of that is corrected, the aliases are declared as path-item `$ref`s (one line each, so they cannot drift from the originals), and `internal/controlplane/openapi_test.go` keeps it honest: `net/http.ServeMux` exposes no way to enumerate its routes, so the test reads the `HandleFunc` literals out of `controlplane`, `auth` and `agentapi` and fails when a served route is undocumented or the spec names a path nobody serves. Two things the review list got wrong, found by reading the code rather than trusting the note: `POST /api/pipelines` genuinely returns 200, so "creates return 201" was applied only where the handler really does (branches, API keys, register); and the `FOX_WEB_ORIGIN` default on the Docs page (`http://localhost:5173`) was already correct, so it was left alone. **K6:** new `docs/mcp.md` documents all 16 tools with their arguments, the paste-ready client config, a stdio check, and the honest limits (no auth of its own, `run_sql` as the non-superuser `db_client`, `branch_before_change` taking minutes); linked from the README and the Docs page. K6 stays Partial — authentication is still open. **L7:** the Guide drops `make web-dev`, adds `sslmode=require` to all five connection strings and moves the agent API to `https://localhost:8088`; the Docs page grows from 7 endpoints to the full set plus an "Agents over MCP" section. Verified: `go test ./...`, `go vet` for macOS, Linux and Windows, `npm --prefix web run build`, the spec re-parsed as OpenAPI 3.1 (48 paths, every alias pointer resolving), and `scripts/test_feature_checklist.sh`.	`docs/api-accuracy`
16 Sep 2026	A4, A7, M5	**Windows install verified end to end, and the summary is green again.** A fresh install on a real Windows machine from v0.8.6 (after `wsl --unregister foxbyte`) exercised everything the earlier run could not: the distro was created from the prebuilt image, "Loading the preinstalled container images" ran with no apt install of Docker or btrfs tools — so the `ghcr.io` tag fix works — the engine download reported "fox-linux-amd64 from v0.8.6 verified against SHA256SUMS" (A7's first real-world proof), and setup ended with one summary carrying the working connection string and API key. Removing the installer's duplicate block had left that summary in plain text, so the engine now prints "FoxByte is running." in green itself: a Windows-only helper turns on the console's escape-sequence handling once and falls back to plain text when output is piped or the console refuses, so `install.log` never collects escape codes. A4 and M5 move to verified.	`fix/setup-summary-colour`
16 Sep 2026	A2, A4, M5	**Windows install checked on a real machine; duplicate summary removed.** Running the branch's installer on a Windows machine confirmed the new path: step 3 downloaded and verified the 685 MB `foxbyte-distro.tar.gz` (the old installer never fetched it), PATH was set and setup completed.	`fix/download-integrity`

		It also showed the install ending with two near-identical "FoxByte is running" blocks — the engine's own summary and the installer's — so the installer now prints only its log path and leaves the summary (with the connection string and API key) to `fox setup`. What that run did **not** prove: it used the released v0.8.5 `bb.exe`, which predates this batch, and reused an existing distro, so the preload re-tag, the API-key summary and the btrfs storage wording (A4) were never executed. They need a run from a release containing this branch after `wsl --unregister foxbyte`, so A4 stays Partial.	
16 Sep 2026	A1, A2, A4, A7, M5; known issues	**Downloads are verified, and Windows uses its prebuilt distro.** Nothing FoxByte published was checked by its own installers, yet these files run as root inside the user's distro. Now `deploy/install.sh`, `deploy/install.ps1` and `fox setup`'s engine refresh all verify every download against the release's `SHA256SUMS`, and anything that cannot be verified stops the install (`FOX_NO_VERIFY=1` skips deliberately; a local `FOX_DIST` build has nothing to check against). The engine refresh reuses the same verified download as `fox update` through the new `update.Client.Release`, and keeps the installer-staged engine rather than installing unchecked bytes. The Windows installer now prefers `foxbyte-distro.tar.gz` (~685 MB, Docker + btrfs tools + engine + images already inside), so setup skips apt, a docker build and three registry pulls; releases without that asset (v0.7.1, v0.8.1) or an unverifiable copy fall back to the Ubuntu rootfs. The distro's Postgres image is tagged `ghcr.io/thefoxbyte/postgres-walg:16` — the name the engine actually runs, so the preload stops being wasted — and older distro images are re-tagged after load. Windows setup also prints the connection string with the API key (read from the guest's config), and its storage error now describes btrfs and `dbpool-storage.service` instead of the old ZFS pool. Verified: new `scripts/test_install_sh.sh` 6 / 6 (tampered, missing listing, unlisted asset, skip flag, local build) and new Pester cases, `go vet` for macOS, Linux and Windows, unit tests on every package; in the VM `make integration` 52 / 52 and `make integration-update` 51 / 51. The Windows install itself still needs a manual run.	`fix/download-integrity`
16 Sep 2026	L8, L9, L10; A9	**Batch 2 verification: the web console reviewed signed in, and `fox update` proven on macOS.** The three pages that had only ever been type-checked were exercised in a real browser against a running stack, each answer compared with the CLI. **L8 Integrity:** the report matched `fox blackbox integrity` field for field; **Create checkpoint** anchored a new entry (and was correctly a no-op when nothing new existed); **Export** answered 200 `application/x-ndjson`; **Branch from before a change** restored a branch in 4 s that lacked exactly the table its entry created, with main untouched. **L9 Policies:** warn → block → warn, disable → enable and add → remove of a custom rule all matched `fox policy list` afterwards and reached only the selected branch; the statement check agreed with `fox policy check`; on `main`, where the account has no `db_admin`, the change was refused with a readable banner and the rule stayed warn. **L10 sidebar:** every link opened its page with the right highlight and breadcrumb (`/ledger` → Changes), collapse persisted across a reload and was applied before first paint, the theme toggle persisted, and no page logged a console error. Two small things recorded rather than fixed: the Integrity entry list needs a refresh to show a change recorded after the page	`docs/batch2-verification`

		opened, and the new-rule form pre-fills Command with `ALTER TABLE` as a real value that reads like a placeholder. The under-860px drawer is still unchecked — the automation could not change the page viewport. **A9:** updated this Mac for real twice (installed v0.8.3 with the installer → `fox update` to v0.8.4 → to v0.8.5), each time checking versions, restarted servers, containers not recreated, branch data, Blackbox, gateway, UI and the kept previous binary; Windows stays untested by choice. Test branches, the temporary key and the `db_admin` grant were all removed afterwards. Documentation only — no product change in this commit.	
16 Sep 2026	A8, A9; known issues	**The new-release notice now actually appears.** Testing `fox update` against real releases on a Mac (v0.8.3 installed with the installer, then the published v0.8.4) showed the notice missing, while `fox update --check` reported the release correctly a moment later. (Precisely: it printed on the first start after a release was published, then stayed silent — the first fetch of a new asset is fast, repeats are throttled.) Cause: the check made two requests, and the second — downloading the release's `SHA256SUMS` — is throttled hard by GitHub for repeated downloads of the same asset. Measured from this Mac: 0.5 s the first time, then 7 s from Go and 75 s with curl and with HTTP/1.1, so the 1.5 s budget was never met (with a 30 s budget the notice did print). The start check now asks only for the releases list — newer, published, with every file this platform needs attached — which takes about 130 ms, and remembers its answer in `~/.fox/update-notice.json` for 6 hours (`FOX_UPDATE_CHECK_INTERVAL`; `0` checks every start), so repeated starts usually make no request at all. Nothing about installing changed: `fox update` still downloads `SHA256SUMS` and verifies every file before anything is replaced. Verified: unit tests (new `TestAvailableMakesNoAssetRequests`, `TestNoticeIsRememberedBetweenStarts`), `go vet` for macOS, Linux and Windows; on this Mac a build stamped 0.8.3 printed the notice within the default budget (2.1 s cold, 1.2 s from the remembered answer); in the VM `make integration-update` 51 / 51 (2 new §1b checks) and `make integration` 52 / 52. A9's macOS path is now verified end to end (versions, restarted servers, containers, branch data, Blackbox, gateway, UI, kept previous binaries), and again on the released fix: v0.8.5 (which contains it) was installed with `fox update` from v0.8.4 — servers restarted on the new binary, `pg-main` not recreated, UI 200, previous binary kept — and a build of the merged code printed the notice in 2.1 s cold and 1.2 s from the remembered answer. Windows testing is deferred by the owner (16 Sep 2026).	`fix/update-notice-throttle`
15 Sep 2026	C3, D4, H7, I1, I5, I6, B1; known issues	**Safety fixes, batch 1. C3:** `fox branch suspend` refuses `main`, the HA standby, and whichever container serves main (the CLI allowed suspending main, which the gateway then couldn't wake). **D4:** reading the code showed the HA hazard was worse than recorded — after `fox ha failover`, `ha enable` and `ha disable` deleted the promoted standby, i.e. the live primary and every write since the failover, `ha failover` again stopped it, and `fox up` / `start` restarted the stale old main as a second writable primary. Now those three are refused after a failover, `fox up` brings the promoted standby back (recreating its container from its storage after `fox stop`) and leaves the old main stopped, and the new `fox ha failback` moves main back to its own container keeping every write: main is rebuilt as a replica of the standby, catches up, the standby is stopped cleanly, main replays to its	`fix/safety-batch1`

		final checkpoint and is promoted; any failure before that leaves the standby primary. **H7:** `fox blackbox revert --to` keeps working but its help and output now say it is a point-in-time restore of main (same as `fox restore`) and point to `branch-before`. **I5 / I1:** `.sql` and Postgres imports run the whole dump, then fail listing every statement that failed (the instance is kept) — they used to report success. **I6:** a JSON array or NDJSON file with an invalid document fails, naming the element or line, and loads nothing from it — an array used to stop silently at the bad element. Integration §3 fix (test-only): its reset of the `pit` table ran `DROP TABLE` without the guardrail override, so it had silently failed for a long time and passed only because the old `ha disable` threw the failover rows away; failback keeps them, which exposed it. Verified: unit tests on every package, `go vet` for macOS, Linux and Windows; in the VM `make integration` 52 / 52 (24 new checks: suspend guards, HA guards, `fox up` after a failover, failback, import errors), `make integration-v2` 214 / 214 and `make integration-update` 49 / 49 (no regressions).	
15 Sep 2026	M1, M2; new document	**New companion document: FOX_Checklist.** `docs/FOX_Checklist.pdf` (source `docs/FOX_Checklist.html`) lists every feature in this document by the same id with its sub-features, whether each is built, partly built or not built, and what tested it — done & tested (automated), done (manual test only), partial, known issue, not started — plus a "Where we are" summary per area, suggested next steps, a "Not started" roadmap, and a note confirming FoxByte has no custom storage engine (PostgreSQL's own engine is used by design; the storage layer is the ZFS/btrfs copy-on-write drivers and wal-g durability). `make feature-doc` now renders both PDFs, and first runs `scripts/test_feature_checklist.sh`, which CI also runs: it fails if a feature id is in one document and not the other, or a row's badge disagrees with its status (checked against a deliberately broken copy). Documentation and tooling only; no product change.	`docs/feature-checklist`
15 Sep 2026	new A8, A9; B1, M1, M2, M4	**New-release notice on `fox start` and `fox update`.** When a newer release is published, `fox start` ends with one line saying so and suggesting `fox update` — no prompt, checked in the background with a 1.5 s limit, silent when offline, off with `FOX_NO_UPDATE_CHECK=1` and on development builds. It lists releases through the GitHub API and offers only a promoted (not draft or prerelease), newer release with every file the platform needs listed in `SHA256SUMS`. `fox update` installs it in one command: downloads and checks every file first (nothing changes unless all verify), checks the new engine runs in the VM / WSL distro, stops the three servers (databases keep running), installs the engine (servers restarted on the old one if that fails), runs `fox up`, `fox ledger upgrade --all` and `fox start` on the new engine, then replaces `fox` on the computer; previous binaries are kept in `~/.fox/updates/prev`. `--check`, `--yes`, `--version`. Data, containers, images, storage and settings are not touched. The only change to existing behaviour is the extra notice line after `fox start`. The release job now fails unless every file `fox update` needs is attached and listed in `SHA256SUMS` (`scripts/check-release-assets.sh`, tested in CI). The first VM run caught a real bug: stdin from `/dev/null` looked like a terminal, so `fox update </dev/null` silently did nothing — it now asks for `--yes`; three test checks were also corrected (history compared by count instead of by the entries themselves, and a settings	`feat/fox-update`

		file that doesn't exist in the VM). Verified: unit tests on every package, `go vet` for macOS, Linux and Windows; in the VM `make integration-update` 49 / 49 (Linux path against a local fake GitHub) and the existing suite 28 / 28 (no regressions). The macOS and Windows update paths are unit-tested but not yet run against a real release.	
15 Sep 2026	new L10; L1–L9	**Web console: sidebar layout for the signed-in app.** The top navbar held every page in one row and was already full. Signed-in pages now use a sidebar grouped into Overview (Dashboard), Data (Console, Import, Pipelines), Blackbox (Changes, Integrity, Policies), Account (API keys) and Help (Guide, Docs), with GitHub, a light/dark switch, the signed-in account, Log out and Collapse at the bottom; it collapses to an icon rail (remembered per browser) and becomes a drawer on narrow screens. A slim top bar shows the section and page. Home and Log in keep the public navbar, which now offers "Open dashboard" to a signed-in user; Guide and Docs appear in whichever layout fits. Every address is unchanged; no API or behaviour change. TypeScript type-check and production build pass; a preview build embedded in the engine served every page (/, /dashboard, /login, /blackbox) with the new layout code. Not yet reviewed in a browser while signed in.	`feat/web-sidebar-layout`
15 Sep 2026	M2, M4, M5; known issues	**Release checksums can no longer be dropped by the other release job.** v0.8.1's `SHA256SUMS` lost the Windows distro image's entry: `wsl-distro.yml` ran first (the `release` workflow had been disabled), then a manual `release` run replaced the whole file with only the binaries' checksums — so the Windows installer would install the distro image without verifying it. Both workflows now publish through one script, `scripts/release-checksums.sh`, which merges its own entries into the published file by exact name, keeps the other job's, and re-downloads after uploading to confirm its entries survived (merging again if the other job uploaded at the same moment). `wsl-distro.yml` also creates the release before publishing checksums. `scripts/test_release_checksums.sh` (run by CI) tests it against a fake `gh`: no file yet, either job finishing first (same result), a re-run replacing its own entry, exact-name matching, another job uploading at the same moment, a failed upload — 9 / 9 on macOS and on Ubuntu (gawk). v0.8.1 itself still needs `wsl-distro.yml` re-run for its tag by a repository admin to restore the distro image's checksum.	`fix/release-checksums`
15 Sep 2026	D1, A4 (Windows distro), CI; known issues	**MinIO images moved to quay.io (release fix).** The `wsl-distro` release job failed: `minio/minio` and `minio/mc` no longer exist on Docker Hub, so the image preload's `docker pull` was refused — and a fresh install's `fox up` would have failed the same way (existing installs kept working from their cached images). The engine and the distro build now use `quay.io/minio/minio:RELEASE.2025-09-07T16-13-09Z` and `quay.io/minio/mc:RELEASE.2025-08-13T08-35-41Z` — the exact versions already running and tested, so nobody gets a silent MinIO upgrade. An install with the old images cached keeps using them (no pull needed); `FOX_MINIO_IMAGE` / `FOX_MC_IMAGE` override. New `internal/branch/images.go` with unit tests, including one that keeps `deploy/wsl-distro/build.sh` preloading the same names. Verified in the VM: both pinned images pull and save as the distro build does, the stack restarted on the quay.io server image against the existing data, and integration-v2 214 / 214 and the existing suite 28 / 28 pass (backups, PITR, branch-before included).	`fix/minio-images-quay`

15 Sep 2026	new H17, H18; H15, H16, B1, K1, K2, K6; known issues	**Blackbox 2.0 Phase 7 — impact analysis and diff.** `fox impact "<DDL>"` reports what a change would affect before it runs: the objects that depend on the table, view, index or column it changes (new read-only SQL function `bb.blast_radius`, following views built on views), the other running branches that have it (with the branch each was cloned from, read from ZFS/btrfs), the policy verdict, and a low/medium/high score with its reasons. MCP `execute_change` results now include it. `fox blackbox diff <a> <b>` (and `fox branch diff`, which the README already advertised) finds where two branches' Blackbox histories split and lists the changes only on each side and the objects both changed — diff only, nothing is merged. New REST `POST /api/branches/{n}/impact`, `GET /api/ledger/diff` and `/api/blackbox/diff`; MCP `impact` and `blackbox_diff`. The first VM run caught a real bug — `bb.blast_radius` pinned its search path, so unqualified table names weren't found — fixed before this commit. Verified in the VM: integration-v2 214 / 214 (31 new §8 checks) and the existing suite 28 / 28 (no regressions).	`feat/blackbox-phase7-impact-diff`
14 Sep 2026	new H16; H9, B1, K1, K2, K5, K6	**Blackbox 2.0 Phase 6 — agent provenance.** Blackbox now records which agent session, task and parent session caused a change. New append-only `bb.agent_sessions`. The Agent API accepts optional `task_id`, `parent_session_id` and `session_id` when creating an agent branch — every change the agent then makes there carries them (requests without them are unchanged, response included). New MCP tool `execute_change`: previews policy rules, runs the SQL without superuser rights with the MCP session, task, parent session and a sha256 call hash attached, and returns JSON (applied / blocked with the `BBX01` rule / error / preview) with the Blackbox entries it wrote. New `fox blackbox sessions` and REST `…/ledger/sessions` (also under `/blackbox`). `run_sql` is unchanged. Verified in the VM: integration-v2 183 / 183 (23 new §7 provenance checks) and the existing suite 28 / 28 (no regressions).	`feat/blackbox-phase6-provenance`
14 Sep 2026	M (CI); H15 tests	**Windows CI fix (tests and repo config only).** Two new policy unit tests assumed LF line endings, but a Windows checkout converts `.sql` and `.md` files to CRLF, so `TestSchemaPolicy` and `TestPolicyContractExample` failed on the `windows-ci` runner. The tests now normalise line endings, and `.gitattributes` pins `*.sql` to LF (like `*.sh` and `*.go`), so the SQL embedded in `fox` is the same bytes on every platform. No product change. Reproduced locally in a CRLF checkout (both tests failed as on the runner) and verified the fix there: every Go package passes with CRLF `.sql` and `.md` files, and a fresh checkout now gets LF `.sql` files.	`feat/blackbox-phase5-policy`
14 Sep 2026	H9 (security fix); known issues	**Capture can't be switched off by a client.** The Blackbox 2.0 kill switch `bb.v2 = 'off'` was a session setting any client could set, so a client could stop the capture rows (transaction id, WAL position, provenance, override use) for its own changes. It is now honoured only when set for the whole database (`ALTER DATABASE`, owner or superuser) or in a superuser's own session; a client's own `SET` — even a `db_admin` member's — is ignored. The check moved inside the capture trigger's fail-safe block. The documented kill switch and the engine's superuser use of it are unchanged. Verified in the VM: integration-v2 160 / 160 (3 new §2 checks) and the existing suite 28 / 28.	`feat/blackbox-phase5-policy`

14 Sep 2026	new H15, L9; H4, H9, B1, K1, K2, K6	**Blackbox 2.0 Phase 5 — policy gate.** Every schema change is checked against policy rules before it runs (event trigger `bb_policy_start`, after the unchanged guardrail). A warn rule lets it through with a NOTICE (SQLSTATE `BBX02`); a block rule refuses it with `BBX01`, records a `BLOCKED` Blackbox entry and an evaluation that survive the rollback, and can be overridden per rule by `db_admin` members (`SET bb.policy_allow`). DETAIL is the JSON defined in `docs/policy-errors.md`. Four built-in rules ship, all warn: `alter-column-type`, `drop-column`, `drop-index`, `grant-to-public`. New: `fox policy list|check|block|warn|enable|disable|add|remove|evaluations`, REST `/api/branches/{n}/policies…` (changes need `db_admin`), MCP `policy_check`, web Policies page. Verified in the VM: integration-v2 157 / 157 (50 new §6 policy checks) and the existing suite 28 / 28 (no regressions).	`feat/blackbox-phase5-policy`
14 Sep 2026	new H14; whole document; H2, H6, H10–H13, B1, K1, K2, K3, K4, K6, L5, L8	**Schema Ledger renamed Blackbox.** Everything people read now says Blackbox: this document, the README, the web console (nav "Blackbox", new address `/blackbox`), CLI help, MCP and OpenAPI descriptions, SDK descriptions and READMEs, code comments. New names were added next to the old ones, and nothing was removed: `fox blackbox …`, `/api/branches/{n}/blackbox…` routes, MCP `verify_blackbox` / `blackbox_integrity` / `blackbox_entries`, SDK `blackbox()` / `verify_blackbox()` / `verifyBlackbox()`. Unchanged on purpose: SQL objects (`bb.schema_ledger` …), the anchor format id, the Go package, command output lines and JSON field names. Also: the policy-gate error contract `docs/policy-errors.md` (Phase 5, written and approved before any policy code: `BBX01` block / `BBX02` warn, JSON DETAIL, `db_admin` per-rule override, all default rules warn, blocked attempts recorded). Verified in the VM: integration-v2 107 / 107 (10 new §5 alias checks) and the existing suite 28 / 28 (no regressions).	`feat/blackbox-phase5-policy`
14 Sep 2026	new H13; D3, B1, K1, K2, K6, L8	**Blackbox 2.0 Phase 4 — branch from just before a ledger entry.** `fox ledger branch-before <id>` creates a new branch holding main exactly as it was the moment before that change committed: it restores the newest base backup that finished before the change and replays archived WAL up to, but not including, the change's transaction (the xid captured in Phase 2; older entries fall back to their timestamp). main is not modified; the published Postgres image is unchanged (the restore script is embedded in the engine); `fox restore` and `fox ledger revert` are untouched. New: `fox ledger entries` (entries with their ids — the existing ledger views are unchanged and show none), REST `GET …/ledger/entries` and `POST …/ledger/{id}/branch`, MCP `ledger_entries` and `branch_before_change`, a panel on the web Integrity page listing recent entries. Test-only fix in `integration_ledger_v2.sh` §2: the "base ledger still records that DDL" check counted rows by table name, so it failed when the suite ran twice without the existing suite's ledger wipe in between; it now counts only the current run's rows. Verified in the VM: integration-v2 97 / 97 (twice back-to-back, including 34 new §4 checks) and the existing suite 28 / 28 (no regressions).	`feat/ledger2-phase4-branch-before`
14 Sep 2026	new H10, H11, H12, L8; H2, H9, B1, K1, K2, K6, M1, M4	**Blackbox 2.0 Phase 3 — tamper-evidence outside the database.** Checkpoints record a Merkle root over new ledger entries in `bb.ledger_checkpoints` and as read-only	`feat/ledger2-phase3-checkpoints`

		anchor files outside the database (format published in `docs/ledger-anchor-format.md`), on a schedule and on demand. `fox ledger integrity` checks the ledger against the anchors and catches history rewritten even with a recomputed hash chain; the new open-source `fox-verify` does the same independently, live or from an export. New: `fox ledger checkpoint|integrity|export`, REST `…/ledger/integrity|checkpoint|export`, MCP `ledger_integrity`, web Integrity page, `fox-verify` release binaries. Verified in the VM: integration-v2 63 / 63 and the existing suite 28 / 28 (no regressions).	
14 Sep 2026	new H9; H4, H8; test script	**Blackbox 2.0 Phase 2 — capture foundation.** New side table `bb.ledger_ext` records the transaction id, WAL position, agent task / tool-call hash and override use for every ledger row, through a fail-safe trigger with a kill switch (`bb.v2 = off`); installed at start and by `fox ledger upgrade`. The base ledger and its hash chain are untouched. Phase 1 verified in the VM (existing suite 26/28, v2 29/29) and Phase 2 verified (v2 42/42, no regressions); two false failures in `integration_test.sh` fixed (test-only).	`56bfb81` (PR #24)
14 Sep 2026	D3, H3, H4, K5, K6, B1; new H8	**Blackbox 2.0 Phase 1 — three security fixes.** PITR restore uses the per-install MinIO credentials. The destructive-DDL override is honoured only for superusers and `db_admin` members (new `fox admin grant|revoke|list` and `fox ledger upgrade`; the first local user is granted automatically). Agent branches log in as their own non-superuser `agent-<id>` role and MCP `run_sql` runs as `db_client` (compatibility switches `FOX_AGENT_SUPERUSER`, `FOX_MCP_SUPERUSER`). New `scripts/integration_ledger_v2.sh` (`make integration-v2`) pins unchanged behaviour and tests the fixes.	`201676c` (PR #23)
14 Sep 2026	Whole document	Document created. Inventory of every feature implemented as of `8b1856f`: install & setup, CLI, branching & storage, durability, security, gateway, auth, Blackbox, import, pipelines, APIs/SDKs/MCP, web console, CI & releases. Known issues and the test baseline recorded.	branch `docs/feature-implemented`

Test baseline

Recorded before any Blackbox 2.0 work starts, so later changes can be checked against it.

Suite	Result	Details
Go unit tests (`go test ./...`, host)	All pass	14 Sep 2026, go1.26.6 darwin/arm64, commit `8b1856f`. Packages with tests: cmd/fox, auth, branch, controlplane, daemon, host, proxy (later also ledger, mcp, update). 15 Sep 2026 with `feat/fox-update`: all pass. 16 Sep 2026 with the start-notice fix: all pass. 21 Sep 2026 with the FoxByte rename: all 12 packages with tests pass, including the new `internal/brand` and `internal/brand/gen` (generated-file drift, the installers' variables, the state directory moving from a retired name, environment variables read under a retired prefix) and the repository-wide guards `TestNoRetiredProductNames`, `TestStoredNamesAreBrandFree` and `TestDurableNamesAreNotSpeltOutTwice`. 17 Sep 2026 with the Blackbox page and small gaps: all 10 packages pass, including the new `TestVMSubcommand`, `TestParseReplication`, `TestIsOriginSnapshotFor`,

		`TestURLDefaults`, `TestSessionCookieSameSite` and `TestCanSuspend`; `go vet` clean for macOS, Linux and Windows. 18 Sep 2026 with the durability gaps: all pass, including `TestPickBackupForTime`, `TestParseRestoreTime`, `TestRestorePITRScript` and `TestStandbyRunArgsArchive`. Same day with attribution & MCP authentication: all pass, including `TestActorKindDerivedFromRole`, `TestAuthenticateWithoutAKey` and `TestApplyScope`. No test files: agentapi, ledger, mcp, secrets, tlsutil, version, web.
Integration (`make integration`, Lima VM)	52 / 52	14 Sep 2026. **Before Phase 1** (`a5a52b6`): 25 passed, 3 failed — bad-key count, PITR (the `minioadmin` bug), append-only check. **After Phase 1** (`33f49ef`): 26 passed, 2 failed — PITR fixed, no regressions. The remaining two were test flaws (psql prints the bad-key error twice; the append-only check discarded stderr); the protections were confirmed by hand and the checks fixed. **With Phase 2** (`feat/ledger2-phase2-capture`): 27 passed, 1 failed — the bad-key check again, because its first fix (`grep -q`) broke under the script's `set -o pipefail`; corrected to count matches and re-checked in the VM. **With Phase 3** (`feat/ledger2-phase3-checkpoints`): 28 / 28 — every check passes. **With Phase 4** (`feat/ledger2-phase4-branch-before`): 28 / 28 on the final code (also 28 / 28 on the first Phase 4 build) — no regressions. **With the Blackbox rename** (`feat/blackbox-phase5-policy`): 28 / 28. **With the policy gate**: 28 / 28. **With the capture kill-switch fix**: 28 / 28. **With Phase 6**: 28 / 28. **With Phase 7**: 28 / 28. **With `fox update`** (`feat/fox-update`): 28 / 28. **With safety fixes batch 1** (`fix/safety-batch1`): 52 / 52 — 24 new checks (suspend and HA guards, `fox up` after a failover, failback, import errors); first run 51 / 52 and second 49 / 52, both on test flaws (a container-absent check, and §3's unguarded table reset), fixed. **With the start-notice fix**: 52 / 52. **In the throwaway test VM** (`fix/test-isolation`): 53 / 53. **With the Blackbox page and small gaps** (`feat/blackbox-page-gaps`): 76 / 76 — 23 new checks (§1b hosting and login URLs, §2b branch from a branch, §5b continuous import and the reaper). The first run was 72 / 1: creating a branch from a non-`main` parent a second time failed on a leftover ZFS snapshot — a real bug the new checks found, fixed and re-run. **With the FoxByte rename** (`chore/foxbyte`): 104 / 104, including a new §0 (a foreign container named "minio" does not break the stack) and §12 (`fox uninstall` removes everything, keeps data with --keep-data, and finds nothing left on a second run). **With the durability gaps** (`feat/durability-gaps`): 91 / 91 — 15 new checks (§3b restore target selection, and six in §6 for backups and archiving after a failover). **With attribution & MCP authentication**: 91 / 91.
`fox update` (`make integration-update`, Lima VM)	51 / 51	15 Sep 2026, `feat/fox-update`. First run 41 / 45: one real bug (stdin from `/dev/null` treated as a terminal) and three flawed checks, all fixed; final run 49 / 49. **With safety fixes batch 1**: 49 / 49. **With the FoxByte rename**: 51 / 51. **With the start-notice fix** (`fix/update-notice-throttle`): 51 / 51, including two new §1b checks — the remembered answer prints with GitHub unreachable, and a forced check then prints nothing.
Blackbox 2.0 (`make integration-v2`, Lima VM)	214 / 214	14 Sep 2026, `33f49ef`: §0 behaviour unchanged and §1a–§1c Phase 1 fixes all pass. With Phase 2: 42 / 42. **With Phase 3** (`feat/ledger2-phase3-checkpoints`): 63 / 63, including 20 new §3 checkpoint, verifier and tamper checks and the §3b

		scheduler check. **With Phase 4** (`feat/ledger2-phase4-branch-before`): 97 / 97, including 34 new §4 branch-before and entry-listing checks; run twice back-to-back after fixing a §2 check that wasn't repeatable (first final run 96 / 97 on that check only). **With the Blackbox rename** (`feat/blackbox-phase5-policy`): 107 / 107, including 10 new §5 checks that every Blackbox name matches the original. **With the policy gate**: 157 / 157, including 50 new §6 policy checks. **With the capture kill-switch fix**: 160 / 160. **With Phase 6** (`feat/blackbox-phase6-provenance`): 183 / 183, including 23 new §7 provenance checks. **With Phase 7** (`feat/blackbox-phase7-impact-diff`): 214 / 214, including 31 new §8 impact and diff checks. **With safety fixes batch 1**: 214 / 214. **In the throwaway test VM** (`fix/test-isolation`): 227 / 227. **With the console permission fixes**: 255 / 255. **With the Blackbox page and small gaps**: 261 / 261, including 6 new §5b checks for the page's filters and session column. **With the FoxByte rename**: 277 / 277. **With the durability gaps**: 261 / 261 — no regressions. **With attribution & MCP authentication** (`feat/attribution-mcp-auth`): 271 / 271, including 11 new §1c checks for the MCP key, its branch scope and both forgery attempts. Three earlier runs were not clean and none of it was the product: the suite's own MCP helpers passed no key, count-based §5b assertions counted the new §1c tables on main (moved to their own branch), §3b compared an empty count as an integer five seconds after a restart (it now waits for main over TCP and for a numeric answer), and one run lost 34 checks to the test VM's wedged Docker daemon.
Web console (browser, signed in)	**L3–L5, L8–L10, I8, D2–D3, H3 pass**	16 Sep 2026, Chrome on macOS against the running stack (v0.8.5). Every Integrity and Policies result was compared with the CLI; the sidebar was walked link by link. **17 Sep 2026**, against a development control plane on a spare port of the real install: the Blackbox page's Verify agreed with `fox ledger verify` (270 chained entries), the table filter returned exactly the 6 rows SQL counted, risk and date filters matched their SQL counts, and an expanded row showed its session, branch and statement; the Dashboard created a branch from another branch and the child's console read the parent's row; and an import replicated a live source into a branch, showed "1 of 1 tables, streaming" in the Continuous imports panel, carried a row inserted on the source, and cut over from the panel — after which later source changes did not arrive and the copied rows stayed. Two defects were found this way and fixed (a filter request race that dropped the last keystroke, and an unreadable statement in the expanded row), as was the reaper suspending a replicating branch. Test branches, containers and the spare server were removed afterwards. **18 Sep 2026**, same arrangement: the Dashboard's new **Base backups** panel listed this install's 61 real backups with the times and sizes `wal-g` reports, flagged the newest, named 25 Aug 2026 as the earliest point reachable, and gave a copy-ready command matching each backup's own finish time; the unbounded first version filled the page, so it now shows six with "Show 55 older backups". The command it offered for the oldest backup was then run against the real install: it chose `base_000000010000000000000021` — the newest backup preceding the target, not the newest of the 61 — and the restored copy came up with no `fox` schema at all, which is correct for 24 Aug, three weeks before the Blackbox existed. The disposable container was removed afterwards. Also on 18 Sep 2026, for H3: on a throwaway branch upgraded to the new `_ctx`, a per-user role that declared itself an agent (`SET`

		`bb.actor_kind='agent'`, `bb.actor='agent-imposter'`) and an `agent-` role that declared itself a person (`'human'`, a chief executive's address) were both read back on the Blackbox page as what they really were — `h3tester@example.com`/human and `agent-h3fake`/agent — with **declared** labels on the tool column and on the session in the expanded row. The branch and its two test roles were removed afterwards. Not covered: the under-860px drawer, and there is still no automated UI test.
Windows (Pester, windows-ci.yml)	In CI	Runs on every push/PR to main.

Feature index

Grouped by area. Badges: Working Partial Known issue

A · INSTALL & SETUP

A1	One-line installer (macOS / Linux)	Working
A2	Windows installer (install.ps1)	Partial
A3	`fox setup` on macOS (Lima)	Working
A4	`fox setup` on Windows (WSL2 + btrfs)	Partial
A5	Linux native	Working
A6	Command forwarding & file streaming	Working
A7	Always-latest engine binary	Working
A8	New-release notice on `fox start`	Working
A9	`fox update`	Partial

B · CLI & SERVICES

B1	The `fox` command set	Working
B2	Background services (daemon)	Working
B3	First run (sample data, `check`, `connect`)	Working

C · BRANCHING & STORAGE

C1	Storage drivers (ZFS / btrfs)	Working
C2	Branch lifecycle	Working
C3	Protection of `main`	Working
C4	Private networking (no host ports)	Working
C5	Scale-to-zero & wake on connect	Working
C6	Storage quotas	Partial
C7	Capacity & status	Working
C8	Provisioning & image pull	Working

D · DURABILITY

D1	WAL archiving to object storage	Working
D2	Base backups	Working
D3	Point-in-time restore	Working
D4	HA standby & failover	Partial
D5	Portable export & restore	Working
D6	PostgreSQL major upgrade	Working

E · SECURITY FOUNDATIONS

E1	Per-install generated secrets	Working
E2	TLS everywhere	Working
E3	Zero-signup local API key	Working

F · SQL GATEWAY

F1	Routing by database name	Working
F2	API-key authentication	Working
F3	Backend login (SCRAM)	Working
F4	Per-user Postgres login roles	Working
F5	Attribution session settings	Working
F6	Ctrl-C (CancelRequest) forwarding	Working
F7	Errors & insecure build tag	Working

G · ACCOUNTS & AUTH

G1	Auth store (SQLite)	Working
G2	Email & password users	Working
G3	API keys	Working
G4	Web sessions & signup control	Working
G5	OAuth (GitHub / Google)	Partial
G6	Security log	Working

H · BLACKBOX

H1	DDL recording	Working
H2	Hash chain, append-only, verify	Partial
H3	Attribution	Working
H4	Destructive-DDL guardrail	Working
H5	Risk flags	Working
H6	Ledger access (CLI/REST/MCP/UI/SDK)	Working
H7	`fox ledger revert --to`	Working
H8	Override permission & ledger upgrade	Working
H9	Blackbox 2.0 capture (xid, LSN, provenance)	Working
H10	Checkpoints & anchors	Working
H11	fox-verify (independent verifier)	Working
H12	Blackbox integrity & export	Working
H13	Branch from just before a Blackbox entry	Working
H14	Blackbox names & aliases	Working
H15	Policy gate (warn / block)	Working
H16	Agent provenance & execute_change	Working
H17	Impact analysis	Working
H18	Blackbox diff, and promoting a branch's changes	Working

I · IMPORT & MIGRATION

I1	From Postgres	Working
I2	From MySQL ≤5.7 / MariaDB	Working
I3	From MySQL 8.x	Working
I4	From MongoDB	Working
I5	From a .sql dump	Working
I6	From JSON / NDJSON	Working
I7	From CSV	Working
I8	Continuous replication & cutover	Working
I9	Guard handling during import	Working
I10	Live progress (CLI & SSE)	Working
I11	Import REST endpoints	Working
I12	Web Import page	Working
I13	Exact identifier fidelity	Working

J · ETL PIPELINES (EXPERIMENTAL)

J1	Pipeline engine	Working
J2	Data-quality tests	Working
J3	Stored pipelines & run history	Working
J4	Pipeline editor (web)	Working

K · APIS, SDKS & AGENTS

K1	Control-plane REST API	Working
K2	OpenAPI spec	Working
K3	Python SDK	Working
K4	TypeScript SDK	Working
K5	Agent Branch API	Working
K6	MCP server	Working

L · WEB CONSOLE

L1	Embedded single-origin UI	Working
L2	Login & signup	Working
L3	Dashboard	Working
L4	SQL console & browser	Working
L5	Blackbox page	Working
L6	API keys page	Working
L7	Landing, Guide & Docs pages	Working
L8	Blackbox integrity page	Working
L9	Policies page	Working
L10	App sidebar layout	Partial
L11	Design system & FoxByte theme	Working

M · BUILD, CI & RELEASES

M1	Makefile targets	Working
M2	Linux CI	Working
M3	Windows CI	Working
M4	Releases & GHCR image	Working
M5	WSL distro / ZFS builds	Partial
M6	Postgres image	Working
M7	Product name in one file	Working
M8	Two editions	Working
M9	Licensing	Partial

N · REALTIME

N1	Realtime change feed	Working

AREA A

Install & setup

A1 One-line installer (macOS / Linux)

Working

WHAT IT DOES	Installs the `fox` binary with one command.
HOW IT WORKS	`deploy/install.sh` detects OS (darwin/linux) and arch (arm64/amd64), downloads `fox-<os>-<arch>` from the GitHub release, verifies it against the release's `SHA256SUMS` (`verify_file`, using `sha256sum` or `shasum`), installs to `$PREFIX/bin` (sudo only if needed). On macOS it also stages `fox-linux-<arch>` into `$PREFIX/share/foxbyte` for the VM.
HOW TO USE	`curl -fsSL https://raw.githubusercontent.com/thefoxbyte/foxbyte/main/deploy/install.sh | sh` · env `FOX_VERSION`, `FOX_REPO`, `FOX_DIST` (local folder), `FOX_PREFIX` (default `/usr/local`).
HOW TO TEST	Unit: `scripts/test_install_sh.sh` (run by CI) against a fake release — a verified release installs; a tampered binary, a release without `SHA256SUMS` and an asset missing from the listing each install nothing; `FOX_NO_VERIFY=1` installs anyway; a local `FOX_DIST` build still installs. Manual: `FOX_DIST=./dist FOX_PREFIX=$HOME/.local sh deploy/install.sh` then `fox version`.
STATUS & LIMITS	Working. Every download is checked against the release's `SHA256SUMS` before it is installed, and anything that cannot be verified stops the install (`FOX_NO_VERIFY=1` skips deliberately; `FOX_DIST` is the user's own build, so there is nothing to check it against). The listing travels over the same TLS connection as the files, so this proves integrity — a complete, unaltered download — not authorship; signatures are still not built. **Signed (22 Sep 2026, audit v2 G22):** it also checks `SHA256SUMS.sig`, an Ed25519 signature by the FoxByte release key, against the public key in the script itself (which comes from the repository, not the release) with `openssl pkeyutl -verify -rawin`. A release without a signature, or signed by another key, is refused (`FOX_ALLOW_UNSIGNED=1` installs an old, unsigned one on checksums alone). Where openssl cannot do Ed25519 — macOS ships LibreSSL — it says the signature was not checked and carries on, unless `FOX_REQUIRE_SIGNATURE=1`; fox checks it on every update. Tested with OpenSSL 3 and with macOS's LibreSSL (scripts/test_install_sh.sh). **It can no longer hang silently** (3 Oct 2026, reported from a real machine as "stuck, doing nothing"). Two faults compounded. `fetch()` was bare `curl -fSL` with no timeout and no retry, and curl's default connect timeout is **300 seconds per address** while the release asset host resolves to three — so a network fault that lets connections stall rather than refusing them left the installer waiting for up to a quarter of an hour. Meanwhile the checksums and the signature were both fetched with curl's stderr discarded and *nothing printed beforehand*, so the script's first visible output came only after both had arrived: on a bad connection, minutes of silence indistinguishable from a hang. Now `--connect-timeout 15 --retry 2 --retry-delay 2` bounds the wait (measured against a blackholed address: gives up in 49s, where the old flags were still waiting at 70s and would have gone to 300s per address), `--speed-limit 1024 --speed-time 60` catches a transfer that opens and then stalls, and the script says which release it is checking before it fetches anything. There is deliberately no `--max-time`: the largest asset is several hundred megabytes and a slow but working connection has to be allowed to finish. The `wget` branch gained the same bounds. **And the same on Windows** (5 Oct 2026): `install.ps1`'s three `Invoke-WebRequest` calls had no timeout either, and `Get-File` did not retry. Which parameter to use is not the same on both PowerShells, and choosing wrong would cut the 460 MB distro image short on a slow link: Windows PowerShell's `-TimeoutSec` bounds the request and not the reading of the response body, so a long download is unaffected; PowerShell 7.0-7.3 changed it to mean the whole operation, body included, and nothing there bounds the connect alone; 7.4 restored the old meaning as `-ConnectionTimeoutSec`. So the bound is 30s on Windows PowerShell, `-ConnectionTimeoutSec` where that exists, and *nothing* on the versions in between, where waiting beats truncating a download that is working — detected by asking for the parameter rather than by version number. `Get-File` now makes three attempts. The file stays CRLF and BOM-free ASCII, which `windows-ci` checks and which a careless edit breaks.
ADDED IN	`c9afda7` (PR #1); URLs repointed to the foxbyte org in `0b125f9`.

A2 Windows installer (install.ps1)

Partial

WHAT IT DOES	From a bare Windows machine to a running database with `irm … | iex`.
HOW IT WORKS	`deploy/install.ps1` `Invoke-Install`: (1) installs WSL (`wsl --install --no-distribution`, elevates if needed; registers a `RunOnce` resume across a reboot); (2) downloads `fox-windows-amd64.exe`, `fox-linux-amd64`, `foxbyte-docker-context.tar.gz`; (3) downloads the Ubuntu noble WSL rootfs verified against Ubuntu's SHA256SUMS; (4) adds the folder to PATH; (5) runs `fox setup`. Forces TLS 1.2, BOM-free ASCII.
HOW TO USE	PowerShell: `irm https://raw.githubusercontent.com/thefoxbyte/foxbyte/main/deploy/install.ps1 | iex` · env `FOX_VERSION`, `FOX_REPO`, `FOX_PREFIX`, `FOX_NO_SETUP`, `FOX_NO_ELEVATE`, `FOX_ROOTFS_URL`.
HOW TO TEST	Run on a real Windows machine on 16 Sep 2026 (from the branch, with the released v0.8.5 assets): the installer downloaded and verified the 685 MB distro image, added fox to PATH and completed setup. Pester: `deploy/install.Tests.ps1` (including the checksum block: listing parsing, the binary-mode `*name` form, an unlisted asset refused, a mismatch deleting the file, `FOX_NO_VERIFY=1`), `deploy/windows-install.Tests.ps1` (E2E group needs `FOX_TEST_E2E`); `windows-ci.yml` checks BOM/ASCII and PS 5.1 parsing.
STATUS & LIMITS	Works. It now downloads the prebuilt `foxbyte-distro.tar.gz` (~685 MB) so setup skips apt, the image build and the registry pulls, falling back to the Ubuntu rootfs when a release does not publish it (v0.7.1, v0.8.1) or the copy cannot be verified; every FoxByte asset is checked against `SHA256SUMS`. **Fixed 21 Sep 2026:** since the rename the installer saved the launcher as `bb.exe` — the rule that renamed the SQL schema to `bb.*` also caught the launcher's file name — so after an install `fox` was not a command; setup still ran because the installer calls the file by its full path. It is `fox.exe` again (from the generated `$Cli`), a stray `bb.exe` from those installs is removed, and the Pester test that had been asserting `bb.exe` now asserts `fox.exe`. **21 Sep 2026:** the distro image is published zstd-compressed and the installer downloads `foxbyte-distro.tar.zst`, falling back to the gzip image older releases published and then to the Ubuntu rootfs; an image the release does not list is skipped rather than downloaded into a 404. Unproven on a real Windows machine until the next install (see M5). PowerShell and .NET have no Ed25519, so it checks checksums only; `fox update` checks the release signature, and `gh attestation verify` proves a download's origin.
ADDED IN	`4fc8afb` (PR #1); one-command flow `0c71da9`, verified downloads `d0c1f26` (PR #7); resume fix `288c431` (PR #8).

A3 `fox setup` on macOS (Lima)

Working

WHAT IT DOES	Creates/starts a Linux VM, installs Docker + ZFS, installs the engine and starts the stack.
HOW IT WORKS	`internal/host/host_darwin.go` `setupDarwin`: requires `limactl`; refreshes the engine binary (A7); creates a `template://ubuntu` VM if missing; then, on **every** run, checks the VM is provisioned (`provisionedCheck`: the ZFS tools, and Docker installed *and running*) and runs `provisionGuest` when it is not — so an install cut short is finished by the next `fox setup`. The provisioning script (`internal/host/provision.go`) recovers the two states a killed apt leaves: dpkg "interrupted" (`dpkg --configure -a` first) and packages "half-installed, reinstall required", which apt calls already installed though their files are missing (reinstalled). It passes its environment with `env`, since current Ubuntu's sudo refuses `-E`. Always reinstalls the engine to `/usr/local/bin/fox` in the VM; then forwards `start`.
HOW TO USE	`brew install lima && fox setup` · env `FOX_LIMA_INSTANCE` (default `fox`, falls back to a VM from a retired name, then `default`), `FOX_GUEST_BINARY`, `FOX_NO_REFRESH`.
HOW TO TEST	Unit: `TestProvisionScriptRepairsAnInterruptedRun`, `TestProvisionedCheckWantsARunningDocker`. Run on 21 Sep 2026 from a Mac against the throwaway VM, with Docker removed and its reinstall killed mid-unpack — dpkg "interrupted", `docker.io` "install reinstreq half-installed", no `docker.service`: `fox setup` said the VM's setup was not finished, reinstalled Docker, and brought PostgreSQL 18 up; dpkg then reported no problems, and a second `fox setup` skipped provisioning (1.7 s). That first run also found the `sudo -E` refusal and that a plain `apt-get install` leaves a half-installed package alone. Manual from scratch: `limactl delete -f fox; fox setup; fox status`.
STATUS & LIMITS	Working. Fixed 21 Sep 2026: provisioning ran only when the VM was created, so an install cut short left a VM every later setup called ready; and with no build context in the VM, a failed image pull ended the install (what stopped the first install of v0.1, on a private package) — the engine now carries the context (C8). A VM whose *creation* was interrupted is left to Lima, which resumes it on `limactl start`.
ADDED IN	`c9afda7` (PR #1); always-reinstall `77ef8ba` (PR #18).

A4 `fox setup` on Windows (WSL2 + btrfs)

Partial

WHAT IT DOES	Imports a dedicated `fox` WSL distro, provisions it and starts FoxByte with quiet step-by-step output.
HOW IT WORKS	`host_windows.go` `setupWindows` : checks `wsl --status` ; refreshes the engine; `importDistro` (`wsl --import` , writes `/etc/wsl.conf` with systemd); `waitForSystemd` ; `provisionGuestWSL` on every run (Docker, btrfs, storage unit, preloaded images, image context, engine binary); `verifyBtrfs` ; quiet `start` with one retry; `finishSetup` . All guest output goes to `install.log` next to `bb.exe` . A `dbpool-storage.service` mounts the btrfs image before Docker at boot; `forwardStdin` waits for systemd on a cold distro.
HOW TO USE	`fox setup` (run by install.ps1) · env `FOX_WSL_DISTRO` , `FOX_GUEST_BINARY` .
HOW TO TEST	Unit: `host_wsl_test.go` (TestResolveWSLDistro, TestWSLArgs, TestDecodeWSLList, TestWinPathToMnt, TestParseKernelRelease). Verified on a real Windows machine (16 Sep 2026, v0.8.6, after `wsl --unregister foxbyte`): the distro was created from the prebuilt image, "Loading the preinstalled container images" ran and no apt install of Docker or btrfs tools appeared, the engine reported "fox-linux-amd64 from v0.8.6 verified against SHA256SUMS", and setup ended with a single summary carrying the working connection string (`postgresql://dbadmin:key_…@localhost:6432/main?sslmode=require`). Manual: `wsl --unregister foxbyte; fox setup; fox status` .
STATUS & LIMITS	Works. Fixed 16 Sep 2026: the preloaded Postgres image is now tagged with the name the engine runs (`ghcr.io/thefoxbyte/postgres-walg:18` since 21 Sep 2026, `:16` before), and a distro image built before 16 Sep is re-tagged after load, so its preload is not wasted. The final summary prints a connection string without the API key or `sslmode` . The storage-unit error text still mentions ZFS.
ADDED IN	`4fc8afb` , `8305e5b` (PR #1); quiet flow `0c71da9` + prebuilt image `d0c1f26` (PR #7); btrfs `fc4a1b8` (PR #9); initdb race fix `cb6655c` (PR #21).

A5 Linux native

Working

WHAT IT DOES	On Linux the engine runs in-process; no VM.
HOW IT WORKS	`host_other.go` : `fox setup` prints "Linux host — no VM needed. Run `fox start` ". ZFS and Docker must be installed; `ensureDocker` / `ensureReady` give instructive errors.
HOW TO USE	`sudo fox start` . The services listen on `127.0.0.1` ; to serve other machines set `FOX_LISTEN=0.0.0.0` (TLS is then required, and a certificate problem stops the service instead of falling back to HTTP).
HOW TO TEST	Integration suite runs on Linux (inside the Lima VM): `make integration` .
STATUS & LIMITS	Working. Until 22 Sep 2026 the three services listened on every interface, so a Linux server install was reachable from its network out of the box, with open sign-up (audit v2 G07). Default is now loopback (`listenAddr` , `FOX_LISTEN`).
ADDED IN	`4fc8afb` (PR #1).

A6 Command forwarding & host file streaming

Working

WHAT IT DOES	On macOS/Windows every engine command transparently runs inside the VM; `fox import --from ./file` works for any file on the host.
HOW IT WORKS	`host.Maybe` (`host.go`) forwards unless on Linux, in the guest (`FOX_IN_GUEST`) or a local command. macOS: `limactl shell … env FOX_IN_GUEST=1 fox …` (auto-starts the VM). Windows: `wsl.exe -d appdb -- env … fox …` . `importLocalFile` rewrites a local file import to `--from - --kind <ext> --srcname <name>` and pipes the file over stdin.
HOW TO USE	Automatic. Env `FOX_GUEST_BIN` .
HOW TO TEST	Unit: `TestImportLocalFile` , `TestWSLArgs` . Manual: `limactl stop foxbyte && fox status` (VM auto-starts); import a CSV from the Desktop.
STATUS & LIMITS	Working. Only `import` streams files; `pipeline run spec.json` passes the path unchanged (must be readable in the VM).
ADDED IN	`c9afda7` , `4fc8afb` , `8305e5b` (PR #1); streaming `44cc05c` .

A7 Always-latest engine binary

Working

WHAT IT DOES	Re-running `fox setup` installs the newest Linux engine rather than a stale bundled copy.
HOW IT WORKS	`refreshEngineBinary` (`host.go`) downloads `fox-linux-<arch>` from the latest (or pinned) release into `~/.fox/` with a 5-minute timeout, atomic rename and an ELF check; best-effort fallback to the installer's copy. `bundledLinuxBinary` prefers the cache.
HOW TO USE	Automatic. `FOX_NO_REFRESH=1` skips it; `FOX_REPO` / `FOX_VERSION` override the source.
HOW TO TEST	Unit: `host_test.go` — TestIsELF, TestBundledLinuxBinaryPrefersCache, TestRefreshEngineBinaryDisabled, TestDownloadFile; `internal/update` TestReleaseForSetup (newest complete release, a pinned older tag, an incomplete release refused).
STATUS & LIMITS	Working, and seen on Windows on 16 Sep 2026: setup printed "Engine fox-linux-amd64 from v0.8.6 verified against SHA256SUMS" before installing it. The download is verified against the release's `SHA256SUMS` (size and SHA-256 as it streams) and an unverifiable or mismatched build is discarded in favour of the engine the installer staged — the engine runs as root inside the VM, so unchecked bytes are never installed. `FOX_NO_REFRESH=1` skips the refresh; `FOX_NO_VERIFY=1` falls back to the old unverified download.
ADDED IN	`77ef8ba` (PR #18).

A8 New-release notice on `fox start`

Working

WHAT IT DOES	When a newer release is published, `fox start` ends with one line: *FoxByte vX is available (you have Y). Run `fox update` to get the new capabilities.* No prompt, and start is never held up.
HOW IT WORKS	New package `internal/update`. The check starts in the background when `start` begins and gets 1.5 s; the notice prints after start's own output only if the check finished in time. It lists releases through the GitHub API (not `/releases/latest` , which can point at a release still being published) and offers only one that is newer, not a draft or prerelease, and has every file this platform's update needs attached. That is a single request: it does **not** download `SHA256SUMS` , because GitHub throttles repeated downloads of the same release asset (measured 16 Sep 2026 from this Mac: 0.5 s the first time, then 7 s–75 s), so the first check after a new release was fast and every later one missed the budget. `fox update` still downloads `SHA256SUMS` and verifies every file before installing anything. The releases list is cached with its ETag in `~/.fox/update-check.json` , and a newer release, once found, is remembered in `~/.fox/update-notice.json` for 6 hours, so the notice keeps printing without a request. "Up to date" is never remembered, so every other start asks — a conditional request that GitHub answers with a 304 when nothing changed, which does not count against its rate limit. Runs on the computer: in `host.Maybe` around the forwarded `start` on macOS/Windows, and in `start` itself on Linux; the engine inside the VM never checks. Offline, rate limited or any error: nothing printed.
HOW TO USE	Automatic. `FOX_NO_UPDATE_CHECK=1` turns it off; `FOX_UPDATE_CHECK_TIMEOUT` (e.g. `3s`); `FOX_UPDATE_CHECK_INTERVAL` sets how long a found release is remembered (default `6h` ; `0` checks on every start); `FOX_REPO` like the installers. Development builds (`0.1.0-dev`) never check.
HOW TO TEST	Unit: `internal/update/update_test.go` (versions, release selection, checksums, cache, a slow server doesn't delay or print, offline, turned off; `TestAvailableMakesNoAssetRequests` — the notice makes no asset request while the update path still verifies; `TestNoticeIsRememberedBetweenStarts` ; `TestNewReleaseShowsOnNextStart` — a release published after an up-to-date check shows on the next start). Integration: `make integration-update` §1, §1b (the remembered answer prints with GitHub unreachable; a forced check then prints nothing), §2, §3, §8. Also checked for real on a Mac (installed 0.8.3, published v0.8.4): the notice prints within the default budget.
STATUS & LIMITS	Working. Releases are published as prereleases and promoted by hand (M4), so the notice appears once a release is promoted. macOS assumes the VM has the Mac's CPU for the notice (Lima's default); `fox update` checks the real one. Fixed 16 Sep 2026: against real GitHub the notice appeared only on the first start after a release was published. The check's second request (`SHA256SUMS`) is fast the first time that asset is fetched and throttled to seconds or minutes on repeats, so every later start fell past the budget. It now needs one request and is remembered for 6 hours. Fixed 17 Sep 2026: remembering "up to date" hid a new release for up to 6 hours — `fox update --check` reported v0.8.7 while `fox start` stayed silent, because its cache said "up to date" from before the release. Only a found release is remembered now, and caches holding "up to date" are ignored; checked against real GitHub with a copy of that cache (v0.8.7 found in 577 ms).
ADDED IN	`feat/fox-update` .

A9 `fox update`

Partial

WHAT IT DOES

Installs the newest release in one command — the new engine, restarted servers, Blackbox upgrades on running branches, and `fox` on the computer — without touching databases, branches, backups or settings. Before, updating meant re-running the installer, then `fox setup` (which left old servers running), then `fox blackbox upgrade --all`.

HOW IT WORKS

`internal/host/host_update.go`, run on the computer on every platform. (1) Downloads every file this platform needs into `~/.fox/updates/<tag>/`, each checked against `SHA256SUMS` as it streams — nothing changes unless all verify. (2) Copies the new engine into the VM / WSL distro and runs its `version` (wrong CPU or corrupt → stop). On macOS, asks for sudo now if `/usr/local/bin` isn't writable; Homebrew installs are refused. (3) `fox _update-guest stop-services`, run with the new engine, stops the three servers (they must stop: `start` leaves running ones alone); Postgres, MinIO and branch containers keep running. (4) `_update-guest install-engine` writes the engine next to the installed one and renames it over (the old one is kept in `~/.fox/updates/prev/fox` in the VM); if this fails the servers are restarted on the old engine. Windows also refreshes the Postgres image build context in the distro and next to `bb.exe` (previous kept as `.prev`; only future image builds use it). (5) With the new engine: `fox up` (running containers left alone; main's Blackbox re-applied), `fox ledger upgrade --all`, `fox start`. (6) Replaces `fox` on the computer — macOS: write-and-rename (sudo if needed); Windows: the running `bb.exe` is renamed to `bb.exe.old` and the new one copied in — keeping the previous copy in `~/.fox/updates/prev/`, and refreshes the engine `fox setup` would install. Then checks the versions and that the control plane answers. On Linux the host binary is the engine, so steps 2–5 run on the machine itself. Never re-imports the distro, recreates storage, changes images or rewrites data files.

HOW TO USE

`fox update` (asks to confirm) · `--check` only reports · `--yes` / `-y` no prompt (required without a terminal) · `--version vX.Y.Z` a specific newer release. Going back: reinstall the older version with the installer (`FOX_VERSION=…`); the previous binaries are kept.

HOW TO TEST

Unit: `internal/host/host_update_test.go` (step order with a fake VM; check only, up to date, dev build, tampered download, no terminal, stdin closed, answered no and an engine that doesn't run all change nothing; a failed install restarts the old engine and leaves the host binary alone), `internal/update/update_test.go` (downloads, install with the previous copy kept, Windows-style executable swap, image context replace with path-traversal refusal), `cmd/fox/update_test.go` (flags). Integration: `make integration-update` (Lima VM, Linux path, local fake GitHub). Manual, not automated: on a Mac, `fox start` shows the notice and `fox update` replaces `/usr/local/bin/fox` and the VM engine; on Windows, the `bb.exe` swap and image context refresh.

STATUS & LIMITS

Partial: the Linux path is verified end to end in the VM, and the macOS path was run for real on 16 Sep 2026 (installed v0.8.3 with the installer, then updated to the published v0.8.4: versions, restarted servers, containers, branch data, Blackbox, gateway and UI all checked). It was run a second time for the released fix (v0.8.4 → v0.8.5) with the same checks. The Windows path is unit-tested and compiles but has not been run against a real release — deferred by the owner (16 Sep 2026). The first release that contains `fox update` must be installed the usual way. Checksums prove the files match the release, not who published it (no signatures). No automatic rollback: the previous binaries are kept for a manual one. Doesn't update the VM, WSL distro, Postgres or MinIO images. `fox setup`'s own engine download (A7) still uses `/releases/latest` without checksums. **Signed releases (22 Sep 2026):** a release is offered only when `SHA256SUMS.sig` is attached and is an Ed25519 signature over its SHA256SUMS by the release key built into fox (`internal/update/signing.go`); one signed by another key, or not at all, is passed over — with `--version`, refused. Unit: TestResolveRequiresTheReleaseSignature; the update suite's fake releases are signed with a test key its builds are given with `-ldflags -X`. A release that exists but whose signature does not check is now reported rather than passed over (24 Sep 2026): `fox update --check` used to say "up to date", which would have hidden a forged or broken release. Proved against the real v0.4: a build carrying another key refuses it by name, and one carrying the release key offers it. **Every install older than v1.0.1 could not update to v1.0** (3 Oct 2026, reported from a real machine): step 2 above validates the staged engine by running its `version` and parsing what it prints, and the binary doing the parsing is the one *already installed*, not the one being installed. Releases up to and including v1.0 took the last whitespace field of that line. When the editions split added `(standard edition)` the last field became `edition)`, so those installs refused with "the new engine doesn't run here" after downloading a binary that ran perfectly well. The parser was corrected before v1.0 shipped, but that correction travels in the new binary, which is not the one deciding — it could not help anybody. Standard therefore prints `fox <version>` and nothing more, restoring the shape every released parser can read; Enterprise may still name its edition, because no Enterprise build predates the correction and `fox update` keeps a machine in its own edition. Locked by `TestStandardVersionLineIsReadableByOlderInstalls`, which carries a copy of the old parser so it cannot be changed from the new side, and by a field count in `scripts/test_editions.sh`. **The gap that let it through:** `integration_update.sh` always has the current build do the updating, so the parser in every shipped binary was never once exercised — the thing being tested was not the thing that runs. The affected installs are reached with the installer, not `fox update`. **Now checked on both sides** (3 Oct 2026): the `upgrade` suite in the nightly builds each recent tag from its own source and checks it accepts this build (M2), and `release-verify` does the same hop over the published assets before a release is promoted out of prerelease (M4). The first catches it before a tag exists; the second catches anything that only shows up in the artifact.

ADDED IN `feat/fox-update`.

AREA B

CLI & services

B1 The `fox` command set

Working

WHAT IT DOES

One CLI for the whole product.

HOW IT WORKS

`cmd/fox/main.go`: `host.Maybe` forwarding, then a command switch with hand-rolled flag parsing (`addrFlag`, `durFlag`, `restoreArg`).

HOW TO USE

Command	Purpose
`fox version` · `setup` · `start` · `stop` · `up` · `down` · `status` · `logs [svc]` · `psql`	Lifecycle, status, logs, psql on main
`fox branch create <n> [--from <branch>]|list|delete|reset [--from p]|suspend|resume`	Branches (C2)
`fox vm [status|shell]`	The engine VM: state and size, or a shell inside it (macOS/Windows)
`fox uninstall [--keep-data] [--yes]`	Remove FoxByte from this machine: containers, storage, state, the VM and the command
`fox backup create|list` · `fox restore --to <ts|latest>`	Durability (D2, D3)
`fox backup export [--out f] [--branch n]…` · `fox backup restore <file> [--branch n] --as <new>`	Portable export (D5)
`fox pg status` · `fox pg upgrade [--dry-run] [--yes] [--export f | --i-have-a-backup] | --rollback | --finalize`	PostgreSQL versions (M6, D6)
`fox ha enable|status|failover|disable|failback`	HA (D4)
`fox blackbox …` (every `ledger` subcommand; same as `fox ledger …`) · `fox ledger [branch] [--limit N]` · `ledger verify` · `ledger upgrade [branch|--all]` · `ledger checkpoint|integrity|export [branch]` · `ledger sessions [branch]` · `ledger diff <a> <b>` · `ledger entries [branch]` · `ledger branch-before <id> [--as n]` · `ledger revert --to`	Blackbox (H6–H8, H10, H12, H13)
`fox import --from <src> [--as n] [--continuous]` · `fox import-cutover <n>`	Import (I*)
`fox pipeline run spec.json [--as n]`	Pipelines (J1)
`fox impact "<SQL>" [--branch] [--object] [--column] [--json]` · `fox branch diff <a> <b>`	Impact analysis and diff (H17, H18)
`fox policy [list] | check "<SQL>" | block|warn|enable|disable|remove <rule> | add <rule> … | evaluations`	Policy gate (H15)
`fox user create` · `fox apikey create|list|revoke` · `fox admin grant|revoke|list [--branch]`	Accounts (G2, G3, H8)
`fox gateway [--addr :6432] [--idle 2m]` · `fox controlplane [--addr :8080]` · `fox serve [--addr :8088]` · `fox mcp`	Run a service in the foreground

HOW TO TEST

Unit: `cmd/fox/main_test.go` — TestAddrFlag, TestDurFlag, TestRestoreArg. Behaviour via `scripts/integration_test.sh`.

STATUS & LIMITS

`fox vm` is listed as a local command (`host.go`) but has no implementation ("unknown command"). `branch create` has no `--from`.

ADDED IN

`a7580f6` (initial); renamed `foxbyte` → `fox` in `c9afda7`.

B2 Background services (daemon)

Working

WHAT IT DOES	`fox start` runs three detached TLS servers: control plane `:8080` (REST + web UI), gateway `:6432` (idle 2m), agent API `:8088`.
HOW IT WORKS	`internal/daemon`: re-executes the binary per service, logs to `~/.fox/<name>.log`, pidfile `<name>.pid`, `Setsid` detach; `Stop` sends SIGTERM then SIGKILL after ~3s.
HOW TO USE	`fox start` · `fox status` · `fox logs gateway|api|controlplane` · `fox stop`
HOW TO TEST	Unit: `daemon_test.go` TestStopTerminatesProcess. Integration §1 (stop/start).
STATUS & LIMITS	Working. The Postgres containers, the object store and the HA standby carry `--restart=unless-stopped`, so they come back after a crash and after the machine or the engine VM reboots, while a container stopped on purpose (`fox stop`, suspending a branch) stays stopped; the transient restore and branch-before containers deliberately do not. The three background servers themselves are still not supervised — `fox check` reports each one, which is the part a person acts on.
ADDED IN	`a7580f6`.

B3 First run (sample data, `check`, `connect`)

Working

WHAT IT DOES	A new install arrives with something in it. `fox start` seeds three related sample tables into `main`, opens the web console with the one-time setup token already in the link, and `fox check` diagnoses the install line by line. `fox connect` prints the connection string an application uses and then opens `psql`; `fox branch create` ends with that string and how long the branch took, instead of a Docker container id.
HOW IT WORKS	`internal/branch/demo.sql` (embedded) creates `users`, `projects` and `events` — five rows each, two foreign keys — applied once by `ensureDemoData` in `Up()`, before the first base backup, and guarded by a marker in the state directory. `internal/host/browser.go` runs the platform's own opener and declines inside the VM, over SSH, with no display and when `FOX_NO_BROWSER` is set; on macOS and Windows the host asks the engine for the token (`forwardCapture`) because the state directory is in the VM. `internal/errhint` maps the failures a new user meets — Docker not running, no VM, a port in use, an API key used as a password, a statement timeout — to one sentence and the next command, and the console does the same in front of raw Postgres errors (`web/src/errhint.ts`).
HOW TO USE	`fox check` · `fox connect [branch] [--dsn]` · `fox psql [branch]` · `fox demo seed|drop|sql [branch]`. `FOX_NO_DEMO=1` starts empty; `FOX_NO_BROWSER=1` leaves the console closed; `FOX_CONSOLE_URL` points the opener elsewhere.
HOW TO TEST	Unit: `errhint_test.go`, `demo_test.go`, `connect_test.go`, `browser_test.go`, `firstrun_test.go`. Integration §11d (seed, re-seed, drop, sequences, Blackbox entries, `connect --dsn`, `check` passing and then failing with the gateway stopped).
STATUS & LIMITS	Working. `fox check` runs inside the engine, so it proves the servers are listening there, not that the host's port forwarding works. The seed is recorded in the Blackbox as CLI-attributed entries, so a first user's own change is not entry #1. The integration suites run with `FOX_NO_DEMO=1` and exercise the seed explicitly.
ADDED IN	Onboarding batch 1.

AREA C

Branching & storage

C1 Storage drivers (ZFS / btrfs)

Working

WHAT IT DOES	Instant copy-on-write copies of a whole Postgres data directory. ZFS on macOS/Linux, btrfs on Windows.
HOW IT WORKS	`internal/branch/storage.go`: `storage` interface; `activeStorage()` picks by `FOX_STORAGE`. ZFS: pool `foxbyte` on a file (`FOX_ZPOOL_DEVICE`, size `FOX_ZPOOL_SIZE`=30G), clone = `zfs snapshot … @for-<dst>` + `zfs clone`. btrfs: loop image at `/dbpool/branches`, clone = `btrfs subvolume snapshot`, new subvolumes get `chattr +C` (nodatacow).
HOW TO USE	Automatic (Windows launcher sets `FOX_STORAGE=btrfs`).
HOW TO TEST	Integration §2 (branch isolation). Manual: `fox branch create t1 && sudo zfs list -r foxbyte`.
STATUS & LIMITS	Working. Until 17 Sep 2026 ZFS `destroy` removed only the `main@for-<branch>` snapshot, so creating or resetting from a non-`main` parent a second time failed on the leftover; delete now also destroys the clone's own origin snapshot (`isOriginSnapshotFor`, unit-tested), and clone re-takes a stale snapshot. btrfs snapshots the subvolume directly and was never affected.
ADDED IN	ZFS `a7580f6`; driver split + btrfs `fc4a1b8`, nodatacow `81ff8f8` (PR #9).

C2 Branch lifecycle

Working

WHAT IT DOES	Create, list, delete, reset (start over from parent), suspend and resume isolated full copies of the database.
HOW IT WORKS	`branch.go`: `Create` (CHECKPOINT parent, clone, chown, start container, `waitReady` over TCP), `Delete`, `Reset` (delete + create), `List`, `Suspend` (`docker stop`), `Resume`, `Wake` → `EnsureRunning`.
HOW TO USE	`fox branch create qa [--from staging]` · `list` · `suspend qa` · `resume qa` · `reset qa [--from main]` · `delete qa`; REST `POST /api/branches` with `{"name","from"}` (404 when the parent doesn't exist), `DELETE /api/branches/{name}`; the Dashboard's create form has a parent picker.
HOW TO TEST	Unit: TestContainerName, `TestIsOriginSnapshotFor`. Integration §2 (isolation), §2b (`--from` copies the parent's data, leaves main alone, names a missing parent, and the REST 404/201), §4 (suspend/resume). Manual: the same in the browser — create a parent, add a row, create a child from it and read the row in the console.
STATUS & LIMITS	Working. A branch is created from `main` unless `--from` / `from` names another branch. CLI create does no name validation (REST does).
ADDED IN	`a7580f6`; reset `00e7eae` (PR #13); TCP readiness `cb6655c` (PR #21).

C3 Protection of `main`

Working

WHAT IT DOES	Stops the primary from being deleted or reset by mistake.
HOW IT WORKS	`Delete` and `Reset` refuse `main`; `Suspend` refuses `main`, the HA standby, and whichever container is serving main after a failover (`suspendRefusal`, so the CLI and REST both get it); `EnsureRunning` never auto-starts main (avoids split-brain after failover); the reaper never suspends main, restore or standby; ZFS reservation keeps space for main (C6).
HOW TO USE	Automatic.
HOW TO TEST	Unit: `TestSuspendRefusal`, `TestPrimaryPointerGuards`. Integration §4 (suspending main refused, main keeps running), §6 (suspending the promoted standby refused). Manual: `fox branch delete main` → "refusing to delete the primary branch 'main'".
STATUS & LIMITS	Working. Until 15 Sep 2026 `fox branch suspend main` was allowed from the CLI and left main stopped (the gateway doesn't wake it) — now refused.
ADDED IN	`a7580f6`.

C4 Private networking (no host ports)

Working

WHAT IT DOES	Branch databases are reachable only through the gateway, not directly from the host or LAN.
HOW IT WORKS	Containers `pg-<name>` join docker network `foxbyte` with no published ports; the gateway dials the container IP (`BackendAddr` → `containerIP` ; main via `PrimaryContainer()`). `FOX_DEBUG_PORTS` re-publishes ports for debugging. The ports that are published — MinIO 9000/9001, the PITR restore on 5433, and a branch under `FOX_DEBUG_PORTS` — are bound to `127.0.0.1` (`loopbackPort`): Docker publishes on every interface otherwise, and on Linux its rules sit in front of ufw/firewalld. The HA standby publishes nothing; the gateway reaches it by container address like any branch. `fox start` re-creates an object store an older install published on every interface (`objStorePublic` ; its data is on a named volume).
HOW TO USE	Automatic.
HOW TO TEST	Unit: TestParsePublishedPort, TestDSN, TestPublishedPublicly, TestPublishedPortsAreLoopback. Integration §1c: the object store is published on loopback only. Manual: `docker ps` shows no `5432` mapping for `pg-*` .
STATUS & LIMITS	Working. Until 22 Sep 2026 the object store (holding every WAL segment and base backup), the restore container and the standby were published on every interface (audit v2 G08, G09); all are loopback or unpublished now.
ADDED IN	`797bc14` , `b664b56` (PR #11).

C5 Scale-to-zero & wake on connect

Working

WHAT IT DOES	Idle branches stop automatically; the next connection wakes them.
HOW IT WORKS	Gateway `reaper` (`proxy.go`) polls every idle/3 (5–30s) and suspends a branch only when idle ≥ threshold and `ActiveConnections` = 0. On connect, `EnsureRunning` routes, recreates or resumes the container and re-enables the guard (`reconcileGuard`).
HOW TO USE	`fox gateway --idle 2m` (default via `fox start`); `--idle 0` disables.
HOW TO TEST	Integration §7 (idle 8s: branch suspended, standby untouched). Manual: wait >2 min, then connect through the gateway.
STATUS & LIMITS	Working. Idle clocks are in memory (reset on gateway restart). Cold-start latency not measured.
ADDED IN	`a7580f6` .

C6 Storage quotas

Partial

WHAT IT DOES	Reserves space for `main` and optionally caps each branch.
HOW IT WORKS	`reserveMain` : `zfs set reservation` (default 2G) on every Init. `setBranchRefquota` : `zfs set refquota` on new branches when configured.
HOW TO USE	`FOX_MAIN_RESERVATION=5G FOX_BRANCH_REFQUOTA=20G fox start` (`none` disables).
HOW TO TEST	Manual: `sudo zfs get reservation,refquota dbpool/branches/main dbpool/branches/qa` .
STATUS & LIMITS	ZFS only; per-branch cap is opt-in (default uncapped); not implemented on btrfs.
ADDED IN	`21f7259` (PR #12).

C7 Capacity & status

Working

WHAT IT DOES	Shows services, main health, backups, branches, storage used/available.
HOW IT WORKS	`Status` , `Branches` (state, used, referenced, connections), `StorageInfo` (`zfs list` / `btrfs filesystem usage`).
HOW TO USE	`fox status` ; `GET /api/status` ; web Dashboard.
HOW TO TEST	Integration §1 (`mainReady`). Manual: `fox status` .
STATUS & LIMITS	Working. On btrfs "used" is apparent size, not the copy-on-write delta.
ADDED IN	`a7580f6` ; StorageInfo `4b0be5c` .

C8 Provisioning & image pull

Working

WHAT IT DOES	`fox up/start` prepares Docker, the storage pool and the Postgres image automatically.
HOW IT WORKS	`provision.go` : `ensureDocker` → `ensurePool` → `ensureImage` (inspect, else `docker pull` the image for this install's PostgreSQL major — `postgres-walg:18` for a fresh install, the major in main's `PG_VERSION` otherwise (M6) — else `docker build` from `FOX_IMAGE_CONTEXT` , a `docker/postgres` beside the working directory, or — always there — the copy built into `fox` (`docker/postgres/context.go` embeds the Dockerfile and entrypoint; `chooseImageContext`), with `--build-arg PG_MAJOR` set to that major, so a local build of an older install's image is still that major).
HOW TO USE	Automatic.
HOW TO TEST	Unit: `TestWriteContextMatchesTheSources` (the built-in copy is byte-for-byte the files the release builds from), `TestChooseImageContext` . Integration §11b: `fox start` from `/tmp` with an image tag on a `.invalid` registry builds it from the built-in copy and runs main on it.
STATUS & LIMITS	Working. The pull needs the GHCR package to be public; when it fails for any reason, fox builds the image itself, which needs Docker Hub and GitHub (the base image and wal-g) — the error says so. A build takes a few minutes, once.
ADDED IN	`c9afda7` ; GHCR pull `aae9e20` (PR #12).

AREA D

Durability

D1 WAL archiving to object storage

Working

WHAT IT DOES	Continuously ships main's write-ahead log to an S3-compatible store, off the commit path: the local one beside main by default, or a remote bucket (`fox backup target set`).
WHERE IT GOES, SAID UNPROMPTED	`fox check` carries a **backup target** line: a warning while the archive is still the local store beside `main` , naming the consequence rather than the configuration — one disk or host failure takes the database and every backup of it at the same moment, and point-in-time restore has nothing left to replay from. **A warning and not a failure:** needing no bucket is the right default for a fresh install, and a new user should not meet a red line before putting anything in the database. But it is the largest reduction in what an incident can cost, it needs no new code to fix, and nothing else in the product said it unprompted — so it is repeated on every `check` until a remote target is set. The line does *not* probe the bucket: reaching it means starting a container and a network call under a ten-minute deadline, and `check` is run often enough that a hang there would make every other line untrustworthy — `fox backup target show` reaches the bucket and reports its Object Lock. An unreadable target file fails rather than falling back to local, because backups going somewhere the owner did not choose is the outcome the check exists to prevent. Phase 0 of the HA plan: hours rather than weeks, and the only item on that list available without new machinery.
HOW IT WORKS	`Up` starts MinIO (`quay.io/minio/minio:RELEASE.2025-09-07T16-13-09Z` , generated credentials) and creates bucket `wal-archive` ; main runs with `archive_mode=on` , `archive_command='wal-g wal-push %p'` , `archive_timeout=60` , lz4. **Target (22 Sep 2026, audit v2 G18):** `internal/branch/target.go` — no file means the local store; `~/.fox/backup-target.json` (0600) names a bucket, prefix, endpoint, region and keys. Every wal-g call takes its prefix as `WALG_S3_PREFIX` (the target's root, plus a cluster prefix after an upgrade) and its keys from `~/.fox/s3.env` via `--env-file` ; the local store's root credentials likewise come from `minio.env` / `mc-local.env` , so no secret key is a command-line argument (`TestNoPasswordOnCommandLines`).
HOW TO USE	Automatic with `fox start` . `fox backup target` shows where it goes; `fox backup target set s3://bucket[/prefix] --endpoint <https-url> --access-key <key> [--region r] [--path-style] [--allow-http]` (the secret key is asked for, or `FOX_BACKUP_S3_SECRET_KEY`) checks the bucket and its Object Lock, saves, restarts main and takes the first base backup there; `fox backup target local` goes back. Local MinIO console at `http://localhost:9001` .
HOW TO TEST	Integration §3. Manual: log in to MinIO with values from `~/.fox/secrets.json` and see WAL files.
STATUS & LIMITS	Working. Branches don't archive: they are disposable by design; `fox backup export --branch` keeps one that matters. Plain-HTTP endpoints are refused without `--allow-http` . Tested against MinIO with Object Lock; AWS S3 is expected to behave the same but was not tested. HA must be off to change the target. Backups on the old target stay there. The object store's ports are bound to 127.0.0.1 (C4). **Images:** MinIO stopped publishing to Docker Hub, so the server and client images come from quay.io, pinned to the tested releases (`internal/branch/images.go`); an install that already has `minio/minio:latest` / `minio/mc:latest` cached keeps using them, and `FOX_MINIO_IMAGE` / `FOX_MC_IMAGE` point at a mirror.
ADDED IN	`a7580f6` ; generated credentials `797bc14` .

D2 Base backups

Working

WHAT IT DOES	Takes and lists full base backups of main.
HOW IT WORKS	`Backup` : `docker exec <primary> wal-g backup-push` , following the primary pointer (`PrimaryContainer`) so it backs up whichever container is serving main and says so when that is not `pg-main` . `BackupList` and the API's `Backups` read the listing from a throwaway container instead (`walgEnv`), so backups can be listed with main stopped or a failover in force — they live in object storage, not in any one container. **Schedule (22 Sep 2026, audit v2 G19):** `Up` takes a first base backup when this cluster has none (`ensureFirstBackup`); the control plane (`StartBackupScheduler`) takes one when the newest is older than `FOX_BACKUP_INTERVAL` (24h, `off` disables), then keeps `FOX_BACKUP_RETAIN` (7) full backups and their WAL (`wal-g delete retain FULL`); objects under Object Lock retention stay regardless. `fox status` and `/api/status` (`backup` : target, newest, age, stale) report the newest backup and warn when it is missing or older than twice the interval.
HOW TO USE	`fox backup create` · `fox backup list` · `GET /api/backups` · the **Base backups** panel on the Dashboard (newest 6, with the rest one click away, and a copy-ready `fox restore --to` for each). `fox backup prune [--keep N]` .
HOW TO TEST	Integration §3, §3b (the API lists them newest-first with the newest flagged), §6 (after a failover: the backup is taken from the promoted standby and says so, one more backup is stored, and `backup list` works with main stopped).
STATUS & LIMITS	Taking one from the primary and listing them both survive a failover. The API surface is read-only: a restore needs a host port and leaves a container behind, so it stays a CLI command. Until 22 Sep 2026 backups were manual only, and a new install had none, so point-in-time restore had nothing to start from until someone ran `fox backup create` . No automated test restore yet. **Proven, not assumed:** `fox backup verify` (and the control plane weekly, `FOX_RESTORE_CHECK_INTERVAL`) writes a probe row into `bb.restore_probe` — the Blackbox's schema, which its event triggers skip, so this records nothing — forces the WAL switch, waits for the archiver, restores the newest base backup into `pg-restore-check` with no published port (so a person's own `fox restore` is untouched), reads the probe row back and removes the container. The result is written to `~/.fox/restore-check.json` and reported by `fox check` . A backup nothing has ever restored is a hope, not a backup, and until this nothing had ever restored one end to end on a real install.
ADDED IN	`a7580f6` ; primary-aware backups, the listing API and the Dashboard panel `feat/durability-gaps` .

D3 Point-in-time restore

Working

WHAT IT DOES	Brings up a separate copy of main at a past timestamp (or latest) on port 5433.
HOW IT WORKS	`Restore` chooses the base backup first (`restoreBackupName` : the target time is read with `parseRestoreTime` , the listing comes from a throwaway container, and `pickBackupForTime` takes the newest backup of this database — same system identifier — that had finished by then), then runs the recovery in the disposable `pg-restore` container: `wal-g backup-fetch <that backup>` , `recovery_target_time` , promote; `waitRecovered` polls up to 80s. The script is `restore_pitr.sh` , embedded in the binary and run with `bash -c` as `BranchBeforeEntry` does, so an install whose image predates the fix gets it from `fox update` alone. `--to latest` still fetches LATEST and replays everything.
HOW TO USE	`fox restore --to '2026-09-14 10:00:00+00'` or `--to latest` . The Dashboard's **Base backups** panel shows the earliest point reachable and a copy-ready command per backup.
HOW TO TEST	Unit: `TestPickBackupForTime` , `TestParseRestoreTime` , `TestRestorePITRScript` . Integration §3 ("PITR restores 3 rows"), §3b (a target before the newest backup reaches 3 rows and not 9, the chosen backup is named and is not the newest, a target no backup precedes is refused naming the oldest and leaves nothing behind, and `--to latest` still reaches the end of the archive), §6 (a restore reaches a write made after a failover); integration-v2 §1a (no `minioadmin` in the restore container, no hardcoded password printed).
STATUS & LIMITS	Uses the per-install MinIO credentials (fixed in Blackbox 2.0 Phase 1 — a hardcoded `minioadmin` pair used to make restore fail on fresh installs), and the printed hint no longer shows a password. A target earlier than every base backup is refused with the oldest one's time, since recovery can only start from a backup the target follows. A timestamp this binary cannot read is still passed to Postgres with LATEST as the base, as before. One restore at a time: the container is always named `pg-restore` and port 5433 is fixed. Reads from the backup target in force (a remote bucket included; integration §11c).
ADDED IN	`a7580f6` ; broke with `797bc14` (PR #11); credentials fixed in `feat/ledger2-phase1-security` ; backup selection `feat/durability-gaps` .

D4 HA standby & failover

Partial

WHAT IT DOES

A single-VM demo of an asynchronous hot standby with manual failover.

HOW IT WORKS

`ha.go`: `HAEnable` (replication hba rule, `pg_basebackup -R`, start `pg-standby`); `HAFailover` (`pg_promote()`, stop old primary, write `standby` to `~/.fox/primary` so the gateway routes main to it); `HADisable`; `HAStatus`. After a failover `haGuard` refuses `enable`, `disable` and `failover` (each would delete or stop the promoted standby, the only primary), and `Init` (`fox up` / `start`) brings the promoted standby up — recreating its container from its storage if `fox stop` removed it — instead of restarting the stale old main as a second primary. The standby's container carries the same object-storage environment and `archive_mode=on` as a primary (`standbyRunArgs`); `on` archives only outside recovery, so nothing is pushed while it streams — the primary is archiving those same segments — and the moment it is promoted it continues the archive on its own timeline. `HAFailback` returns to main without losing writes: main's data directory is rebuilt with `pg_basebackup` from the standby (the dataset is kept, since branches clone its snapshots), main starts as a streaming replica and catches up, the standby is stopped cleanly, main replays up to the standby's final checkpoint (read with `pg_controldata`), then main is promoted, the pointer goes back to `main` and the old standby is removed. Any failure before the promotion restarts the standby and leaves it primary.

FAILOVER DRILLS, AND WHAT MAY BE QUOTED

`fox ha drill --yes` ends the primary without warning, recovers through the real failover path, and writes a dated report to `~/.fox/drills/`. Phase 6 of the HA plan, where the drill is the stated deliverable rather than the standby: “we fail over in about a minute” is a claim, and a report from a real failover is evidence. **Three decisions make it a drill rather than a demonstration.** It **kills** the primary rather than stopping it — `docker stop` sends SIGTERM, Postgres does a fast shutdown, and a fast shutdown flushes its remaining WAL to the standby before exiting, so a stopped primary loses nothing and the number would be a flattering fiction; a machine that dies does not say goodbye. It kills it **with writes in flight**, half way through the run, from one session that reads back each commit the server acknowledged — the only honest definition of data loss is a write the client was *told* had happened which then had not, and a drill against an idle database measures an idle database. And it recovers through `HAFailover`, the same function `fox ha failover` calls, so what is measured is the thing that ships. Measured on the single-VM topology: **0 of 60 acknowledged writes lost, 400ms from the kill to a write on the new primary**. **The report spends more words on what those do not mean than on what they are**, because the first real run lost nothing with *asynchronous* replication — on one host with a standby microseconds behind that is the usual outcome, and it is exactly the figure somebody would put in a tender. So a measurement is never printed without the configuration that produced it, and the report says which of the two may be quoted: with zero-data-loss mode on, RPO zero is a property of the configuration and the measurement is “a confirmation, not the evidence”; asynchronous with nothing lost is “Nothing, on the strength of this”, because a busier primary, a slower link or a standby doing anything else would all lose writes and nothing bounds how many; asynchronous with writes lost is what *that* workload cost at *that* rate on *that* machine, and what may be promised instead is the archive — `archive_timeout` seconds, *once the archive is somewhere the machine’s failure cannot reach*, which is why the report shouts while backups are still on the same disk as the database. `archive_timeout` is now one constant rather than a literal in the primary’s arguments and another in the report, with a test holding them equal, because a report that disagreed with the setting would be worse than no report. **And a section saying what it did not test**, since the measured figures are the part that gets quoted: losing a machine (the standby shares this host’s disk and power), noticing (detection and the decision were manual, and the RTO clock starts at the kill), and a second site (there is only one) — the first and third being phase 1, the second phase 3. The drill’s own `DROP TABLE` was refused by the Blackbox guardrail, which is the guardrail working, so it creates its schema and empties it with a qualified `DELETE`; the one place that overrides anything is the cleanup, which says so. A drill refused before it starts prints the refusal and nothing else — the first version printed a report of zeroes, which buried it.

ZERO DATA LOSS, OPT IN

`fox ha sync on` makes the primary hold a commit until the standby has it on disk (`synchronous_standby_names = 'ANY 1 (fox_standby)'`); `fox ha sync off` releases it. RPO zero inside the data centre, which is what a regulated buyer usually means by “active-active”. Phase 5 of the HA plan, and **the one feature here that deliberately breaks the rule the rest of the engine keeps** — never slow the commit — so it is opt in, off on every install, and says what it costs in the words the operator reads. **With one standby a quorum buys nothing.** The plan described `ANY 1 (…)` as a quorum “so one slow standby cannot stall writes”; that holds only with more than one standby to choose from, and with the single standby this engine builds today it means every commit waits for that one standby — **if it stops, writes stop**. The quorum form is still what is written, because phase 1 extends it by adding names rather than by rewriting this, but the CLI says the truth rather than the aspiration. **The escape hatch is why this is safe to offer:** `ALTER SYSTEM RESET synchronous_standby_names` and a reload work *while every commit is waiting*, since neither needs a commit of its own — verified against a primary with two backends stalled in `SyncRep`, both released and the next commit succeeded. **The standby needs a name Postgres can match, and `pg_basebackup -R` does not give it one:** it writes no `application_name`, so the walreceiver reports as `walreceiver` and the setting would match nothing — in place, unsatisfiable, every commit waiting for a server that can never answer. The name comes from the standby's `cluster_name` on its own command line, which is also its phase-4 slot name: one word names this standby in `pg_replication_slots`, in `pg_stat_replication` and in `synchronous_standby_names`. It is postmaster-scoped, so `ha sync on` restarts a standby that predates it rather than asking for a rebuild — what `realtime setup` already does for the same reason. **Named rather than `*`**, because `*` matches any walsender and `pg_basebackup -X stream` is one: a backup in progress would satisfy the quorum and a commit would be called durable when only a backup had it (checked — with the standby stopped and a backup running, a commit still waits). **Enabling proves itself and rolls back:** after applying, it asserts `sync_state = 'quorum'`, and if the standby is not counted the database is already refusing to finish a commit, so the setting is undone and the command fails saying nothing changed. **The trap it guards:** `ALTER SYSTEM` writes `postgresql.auto.conf` and `pg_basebackup` copies `PGDATA`, so a standby built while the mode was on carries the setting — checked, it is in the copy — and promoting it gives a primary waiting for a standby of its own that does not exist. Every write would stop at the moment of a failover, which is the moment it must not; `ha failover` clears it on the promoted node, and `failback` and `disable` do the same. The *intent* is a marker file kept separately from the setting, so `ha status` and `check` can tell "nobody asked for this" from "it was asked for and cannot be honoured right now", and `ha enable` puts it back once a standby streams. **`fox check` gained a durability line**, and the failing case is the point: on with nothing counted is a **failure**, not a warning, because from outside a stalled primary looks like a hung database — so the one command that releases it is on the screen, with the fact that it works while commits are stalled. `synchronous_commit` is left at its default `on`, which already means wait-for-flush once names are set.

FENCING A PRIMARY ON ANOTHER MACHINE

`ha failover` on a machine that follows a remote primary probes that primary first and **refuses while it answers**, because promoting would make a second one. The fencing half of phase 3 of the HA plan, done first because **phase 1 shipped a split brain**. On one machine `HAFailover` stops the old primary before promoting, and that is the whole of fencing when both are containers here; across machines it stopped the wrong thing — `PrimaryContainer()` is *this* host's `pg-main`, an idle database nobody uses on a joined node, while the real primary went on serving writes elsewhere. Measured with the primary deliberately left healthy: both machines accepted writes and the promoted one reported `pg_is_in_recovery()` false. Two primaries, diverging, with nothing to say so. **Verified both ways after the fix:** refused with the primary alive and the standby left in recovery, promoted when the primary was genuinely stopped. `--force` exists for a primary an operator has already isolated by other means and warns in the words that matter — "both machines are about to accept writes and the two copies will diverge with no way to merge them". A promotion now also says what must happen to the old primary: it **must not** be started as a primary again, since it would accept writes the survivor never sees, and `node join` against the survivor is the way back because it replaces that copy. **The probe is a TCP connection and nothing more, deliberately:** the question is "would promoting here make a second primary", and anything answering on that port is reason enough to believe it would — a richer probe would need credentials and a TLS handshake to say "no", so it would report a dead primary for a certificate problem. The conservative error is refusing a promotion that might have been safe; the expensive one is allowing a promotion that was not. **And what it does not do, said plainly rather than implied:** it cannot tell a dead primary from one this machine merely cannot reach, so a network partition where clients can still reach the primary produces that same split brain and no local probing can distinguish the cases. That needs a third opinion, which is the rest of phase 3.

A STANDBY ON ANOTHER MACHINE

`fox node join <token>`, on the second machine: copy the primary over TLS, run the copy as a standby. The last item of phase 1 of the HA plan, and **what it reuses is the point** — the promotion, the failover refusals, slot synchronisation, zero-data-loss mode and the failover drill all work on a standby whose primary is across a network exactly as on one whose primary is the container beside it. Two things differ: where the base backup comes from, and that the walreceiver has a certificate to verify. Confirmed in `pg_stat_wal_receiver` on the second machine — `sslmode=verify-full`, `sslrootcert=/fox-tls/ca.crt`, `status=streaming` — and on the first, `pg_stat_replication` showing `fox_standby | 192.168.105.3 | streaming`. **The promise, measured rather than claimed:** rows written on the primary (the last of them seconds before the end), the primary's database killed with SIGKILL, `fox ha failover` on the surviving machine — which took a write and held every acknowledged row, with `pg_is_in_recovery()` false. That is "survives losing a machine", on two machines. **The joined install's own `main` is idle afterwards**, which is honest rather than elegant — a node hosting a standby of another install is a standby node — and is what keeps the change additive, since no existing path changes meaning. **Three sharp edges the live testing found.** A **promoted standby still claimed to follow its old primary**: it held the only copy of the data while its state said it was a copy of something else, and the guards built on that state then refused the *right* operations, `ha enable` among them — a promotion clears the join now and says "it is the primary now, holding every write that reached it". **`ha enable` would have rebuilt a remote standby from the local main**: same container, same storage, so a machine following another primary would have quietly stopped being a copy of anything that mattered; it refuses and names `node leave`. And **the CA path has to be one constant**: `pg_basebackup -R` records `sslrootcert` as the path it saw during the copy and the walreceiver reads that path from inside the standby container later, so a host path there would leave the standby verifying nothing. **What two VMs do not prove:** they share a kernel, a disk and a power supply, so this establishes the networking, the credentials, the TLS path and the promotion — not survival of a power cut. **Cross-host failback is not built**: after a failover the survivor is the primary, and giving the role back means that machine joining this one and failing over again — supported in principle, unverified, so not claimed.

PUBLISHING A PRIMARY FOR A STANDBY ELSEWHERE

`fox node expose --address A --peer B --yes` puts Postgres on a network — the one thing the engine otherwise refuses to do, since `startContainer` publishes no host port precisely so a direct connection cannot bypass the gateway, its API key, TLS and ledger attribution. It reverses that narrowly: only when asked, only on the address given (bound to it, not to every interface), only over TLS with a certificate the standby verifies, only for `db_replica`, and only from the one peer address named, as a `/32`. `fox node unexpose` takes it back off; `fox node status` and `fox check` report it on every run, because an operator should never *discover* that their database is on a network. Phase 1 of the HA plan. **`internal/tlsutil` is deliberately not reused:** it says of itself that the point “is not a trusted chain — it is encryption on the wire”, which is right for a browser accepting a self-signed certificate once and wrong for replication, where a standby that encrypts without verifying has an encrypted channel to whoever answered and sends a database-copying credential down it. So there is a small CA per install, its key never leaving the machine, issuing a server certificate naming the dialled address; the standby gets the CA in its join token and connects with `sslmode=verify-full`. **Measured between two machines, before and after writing it:** `db_replica` over TLS with the CA answers `IDENTIFY_SYSTEM`; without TLS it is refused; the **superuser** is refused even over TLS from the permitted address; `verify-full` without the CA is refused client-side; and a real `pg_basebackup` copied 47 MB over TLS. **Three things that environment caught which reading could not.** A **silent TLS bypass that nearly existed**: a published install carries both `host … samenet` (for a standby on this machine) and `hostssl … peer/32`, and `host` matches plaintext — had docker’s port publishing made a remote connection appear to come from the docker gateway, it would have matched the samenet rule and TLS would have been optional while looking mandatory. Docker preserves the source address, so it is refused; that was luck rather than design, and it is a test now. **The private key must not live in `PGDATA`**: the first instinct was to put it there, since it is already bind-mounted and the TLS settings are all `sighup` so TLS would have needed no restart at all — but `pg_basebackup` copies `PGDATA`, so every standby that joined would have received the primary’s private key. The certificates live in a read-only mount instead. And **only the published port needs the restart**, which is why the command says connections drop for a few seconds rather than pretending otherwise. **The join token settles what the plan got wrong.** It asked to “distribute the install secret … since every role password is derived from the secret” — which is exactly what it must not do, because a node holding that secret can derive the *superuser’s*, handing a standby the credential that owns the database. The token carries the CA certificate, the address and port, and `db_replica`’s password; a test asserts the install secret does not appear in it. The cost, stated rather than discovered: rotating that password means re-issuing the token — the right side of the trade, since a rotation is rare and a superuser credential on a second machine is forever.

WHO MAY COPY THE DATABASE

A replication role of its own, `db_replica`: `LOGIN REPLICATION NOSUPERUSER NOBYPASSRLS NOINHERIT`, no table grants, a member of nothing, with a password derived from the install secret like every other role’s. The first item of phase 1 of the HA plan — “a dedicated replication role rather than widening `allowReplication`” — done first because it is the one part of that phase worth having on a single machine. **What it replaced:** `ha enable` appended `host replication all all scram-sha-256` to the primary’s `pg_hba.conf` — any role, from any address — and the standby then connected as the **bootstrap superuser**, the only role that had `REPLICATION`. The credential that copied the database was the credential that owned it, and the rule admitting it admitted everything. On one machine with no published port the reach of that is small, which is why it lasted; phase 1 ends with Postgres reachable from another host, and that rule is what would have been waiting there. **Measured against a live install rather than reasoned about:** the superuser attempting physical replication over the network now gets `FATAL: no pg_hba.conf entry for replication connection … user "dbadmin"` while its ordinary connections are untouched; `db_replica` reading a user table gets `ERROR: permission denied`, and `IDENTIFY_SYSTEM` answers. That first one had to be checked rather than assumed, because the image’s default `pg_hba.conf` ends with `host all all all scram-sha-256` — it is only because `all` in the database column does *not* match a physical replication request that removing the wide rule actually closes the door, and the suite asserts that against a live server instead of inferring it from the documentation. **The address is `samenet`**, the subnets the server is directly attached to, which inside a container is the docker network the engine made: narrower than `all`, and as narrow as it gets until a standby on another machine has an address worth naming. **The wide rule is deleted, not outranked** — `pg_hba` is first-match-wins, so leaving it below the narrow one would look like a narrowing and be none. The role’s password reaches the base backup through a 0600 env file rather than a `docker run -e`, because a command line is visible in `docker inspect` and in ps; an existing test that scans for passwords passed as arguments caught the first version of this and sent the writer to `pgenv.go`, where the one piece of code that knows how to write a credential file lives. **And `fox check` gained a `replication` line**, for the upgrade path:

	that rule lives in the data directory, a routine `fox update` has no reason to rewrite it, the only thing that does is another `ha enable`, and so an install whose HA was enabled by an older version keeps the wide rule with nothing to say so.
REALTIME ACROSS A PROMOTION	With the change feed on and a PostgreSQL major that has failover-aware slots, starting the standby first creates its physical slot `fox_standby` on the primary and then names it on the standby's command line, and the base backup writes a database name into `primary_conninfo` — the two things `sync_replication_slots` needs in order to copy a subscriber's position to the standby. The backup itself streams through no slot, deliberately: see N1, "Surviving a failover". `ha disable` drops the slot, because one nobody will consume holds write-ahead log that cannot be reclaimed, and it does so only while the primary is up to be asked — `fox stop` then `ha disable` should not warn about a slot it cannot even look for. With the feed off, or on PostgreSQL 16, a base backup and a standby's command line are byte-for-byte what they have always been.
HOW TO USE	`fox ha enable | status | failover | disable | failback` — after a failback, take `fox backup create` and `fox ha enable` for a new standby.
HOW TO TEST	Unit: `internal/branch/ha_test.go` (TestHAGuard, TestPrimaryPointerGuards, TestLSNAtLeast, TestParseControlCheckpoint, `TestStandbyRunArgsArchive`). Integration §6: streaming replica; write after failover; the promoted standby's `archive_mode` is on and its archiver is shipping segments; a backup taken from it; `backup list` with main stopped; `fox status` naming the container serving main; a point-in-time restore reaching a write made after the failover; `ha enable|disable|failover` and suspending the standby refused; `fox up` after the standby's container is removed recreates it and leaves the old main stopped; `ha failback` exits 0, main is primary and not in recovery, the post-failover write is on main, the standby is gone, the gateway writes to main, WAL archiving resumed. §7.
STATUS & LIMITS	Demo: no automatic detection, fencing or multi-host. Failback briefly stops writes (between stopping the standby and promoting main). Fixed 15 Sep 2026: `ha enable` or `ha disable` after a failover deleted the live primary and the writes made since, and `fox up` restarted the old main as a second primary. Fixed 18 Sep 2026: a promoted standby archived no WAL, so nothing written while main ran on it could be backed up or restored — it archives now, and a standby created before this change is named as unarchiving when `fox up` brings it back (`archive_mode` is a postmaster setting, so only a rebuild adds it).
ADDED IN	`a7580f6`; btrfs support `fc4a1b8`.

D5 Portable export & restore

Working

WHAT IT DOES	A copy of an install's databases that survives a PostgreSQL major change and can live anywhere: one verified `.tar` on the user's own disk, restorable into a new branch of the same or a newer major.
HOW IT WORKS	`internal/branch/export.go`. For each branch (by default main and every branch that is not an agent's; `--branch` narrows), a throwaway container of the engine image runs `pg_dump -Fc` and `pg_dumpall --roles-only` against it over the docker network — roles per branch, because every branch is its own cluster and a per-user login role lives only where that user connected. The dump keeps owners and privileges and includes the Blackbox; pg_dump writes triggers and event triggers after the data, so a restore records nothing new. The Blackbox head (count and newest hash) is read on both sides of the dump and the dump retaken if they differ, so the manifest describes exactly what the dump holds. A suspended branch is woken for its dump and put back to sleep. The archive holds `manifest.json`, a `SHA256SUMS` over every member, and the files; it is written to `.partial` and renamed. On macOS and Windows the engine writes it inside the VM and the host copies it out (`limactl copy`; the `\\wsl.localhost` share), verifies it against its own sums, and deletes a copy that does not verify. `ReadExport` refuses a damaged member, a missing one, and a member named outside the archive. `fox backup restore` verifies again, refuses an export from a newer major than the install runs, restores roles then data into a new branch, and checks the Blackbox came across exactly before bringing its definitions and the role passwords up to this install.
HOW TO USE	`fox backup export [--out <file>] [--branch <name>]…` (default `~/.fox/exports/`) · `fox backup restore <file> [--branch <name>] --as <new branch>`
HOW TO TEST	Unit: `TestExportRoundTrip`, `TestDumpAtStableHead` (a schema change mid-dump retakes the dump; one that never settles fails), `TestReadExportRefusesDamage`, `TestReadExportRefusesPathsOutsideTheArchive`, `TestExportBranchesSkipsTheDisposable`, `TestParseLedgerHead`, `TestValidName`, `TestParseExportAndRestoreArgs`. `make integration-pg-upgrade` §1: an export of a real 16 install holds each branch's dump and roles and not the standby, leaves a suspended branch suspended, a byte flipped in the middle is refused and restores nothing, and a restore into a new branch has the data, every Blackbox entry unchanged, a chain that verifies, and roles the account can log in with through the Gateway.
STATUS & LIMITS	Working in the test VM (72 / 72 for the suite, 21 Sep 2026). The host half (copying out of the VM) is exercised on Linux only by the suite; on macOS and Windows it is compiled and unit-tested, and first runs for real on a user's machine. Plain archive, no encryption: the roles file holds the roles' SCRAM verifiers, and the archive is written 0600. Database-level settings (`ALTER DATABASE … SET`) are not in a pg_dump; the engine sets its own again on start.
ADDED IN	`feat/pg-upgrade`.

D6 PostgreSQL major upgrade

Working

WHAT IT DOES	Moves an install from the PostgreSQL major its data was created with to the one this `fox` ships, keeping every database, role, API key and Blackbox entry — after showing every change, asking, and taking an export. The old databases are kept until the user deletes them, so it can be undone.
HOW IT WORKS	`internal/branch/pgupgrade.go`, orchestrated by `host.RunUpgrade` (plan → confirm → export → apply, one code path for macOS, Windows, Linux and inside the VM). **Plan**: each branch's major from `PG_VERSION`, which are carried (not agents'), HA state, the storage, and the PostgreSQL changes crossed — `pgNotes`, written by hand from each release's Migration section, each with a probe run on every running branch, so changes found in the user's own data are listed first (an expression index, an MD5 password, an inheritance tree…). **Refusals**: HA on, main served by the standby after a failover, main not running, a Blackbox that does not verify, branches on mixed majors, an earlier upgrade's databases still held. **Apply**: dump every carried branch with the *new* major's `pg_dump` (a suspended one is woken and its Blackbox verified first); remove the Postgres containers; move every branch dataset into a holding area outside the branch namespace (`storage.stash`: a sibling dataset under ZFS, a dot-directory under btrfs, so no listing or routing sees it); reload each carried branch into a new dataset on the new image, main first; check each Blackbox came across exactly; give the new main its own WAL archive prefix (a file in its dataset, outside PGDATA — a new cluster's segment names would otherwise collide with the old one's); take a base backup. A reload, not `pg_upgrade`: that needs both majors' binaries in one image, cannot hard-link across datasets, and keeps indexes built under the old collation library; a reload rebuilds every index. Any failure moves everything back before the command returns. **Rollback** deletes the upgraded branches (clones of the new main first) and moves the old ones back; the old main has no prefix file, so it archives to the root and its backups are visible again. **Finalize** deletes the holding area.
HOW TO USE	`fox pg status` · `fox pg upgrade --dry-run` · `fox pg upgrade [--yes] [--export <file> | --i-have-a-backup]` · `fox pg upgrade --rollback` · `fox pg upgrade --finalize`. Guide: `docs/postgres-versions.md`.
HOW TO TEST	Unit: `TestUpgradeRefusals` (every refusal, and failback rather than disable after a failover), `TestUpgradeRenderWarnsStrictly` (what is lost and kept, found changes first), `TestRunUpgradeOrder` (never apply without plan, confirmation and export unless told; a failed export stops everything), `TestEveryUpgradeHopHasNotes` (bumping `PGMajor` without notes fails), `TestNotesAreComplete`, `TestClonesFirst`, `TestWalgPrefixFor`, `TestParsePgUpgradeArgs`. `make integration-pg-upgrade` (72 / 72): a real 16 install stays on 16 by itself after one pinned start; the plan finds its expression index, refuses with HA on and says what to do, and declining changes nothing; the upgrade leaves main and a suspended branch on 18 with their data, the index, every Blackbox entry unchanged and none added, verifying, the account's API key working through the Gateway, one base backup under a new prefix and none of 16's, and the old databases held where the branch list cannot see them; 18 data on the 16 image is refused before Postgres sees it; rollback deletes a branch made after the upgrade, returns 16 with its data, Blackbox and backups; a second upgrade and a finalize leave nothing held.
STATUS & LIMITS	Working in the test VM, 16 → 18 (72 / 72, 21 Sep 2026). Every database is offline while it runs, and a reload takes time in proportion to the data. Carried branches become full copies, no longer sharing blocks with main, and the held copy uses its space until `--finalize`. Point-in-time restore starts again at the upgrade. Agent branches are not carried. HA must be off. A rollback deletes branches made after the upgrade (it says which before asking). On macOS and Windows the pre-upgrade export's copy to the host is unit-tested only until a user runs it. **The suite runs on ZFS** (macOS and Linux); on Windows the branches are btrfs subvolumes and the holding area is a hidden directory beside them, moved with `mv` — that path is compiled but has not yet run.
ADDED IN	`feat/pg-upgrade`.

AREA E

Security foundations

E1 Per-install generated secrets

Working

WHAT IT DOES	Every install gets its own Postgres and MinIO passwords — nothing hardcoded.
HOW IT WORKS	`internal/secrets` : `Load()` reads or creates `~/.fox/secrets.json` (0600, atomic write) with crypto/rand values (`pg_password` , `minio_user` , `minio_password`); env overrides are applied but never written. Used by the engine, gateway backend login and role password sync. Since 22 Sep 2026 the Postgres secret is the superuser's password only: every other login role — `db_client` , each account's role, each agent's — has its own, derived from it by HMAC (`ClientRolePassword` , `UserRolePassword` , `AgentRolePassword`), so learning one role's password gives neither the secret nor any other role. The engine hands the secret to `docker` in `~/.fox/pg.env` (0600, `--env-file`) instead of `-e PGPASSWORD=…` , so it never appears in the process list.
HOW TO USE	Automatic. Env `FOX_PG_PASSWORD` , `FOX_MINIO_USER` , `FOX_MINIO_PASSWORD` .
HOW TO TEST	Indirect (TestDSN isolates HOME). Unit: TestRolePasswordsAreDistinct, TestNoPasswordOnCommandLines (fails if a password is passed as an argument again). Integration §1d: the install secret no longer logs in as `db_client` ; `pg.env` is 0600. Manual: `cat ~/.fox/secrets.json` in the VM.
STATUS & LIMITS	Working, except PITR ignores it (D3). Process-local caching only. Until 22 Sep 2026 the secret was every role's password, and passed on `docker` command lines (audit v2 G10).
ADDED IN	`797bc14` (PR #11).

E2 TLS everywhere

Working

WHAT IT DOES	Gateway, control plane and agent API all serve TLS with zero setup; clients can use `sslmode=require` .
HOW IT WORKS	`internal/tlsutil` : `EnsureCert` uses `FOX_TLS_CERT` / `_KEY` or generates a self-signed ECDSA P-256 cert (10 years, SAN localhost/127.0.0.1/::1) under `~/.fox/tls` with a cross-process file lock; min TLS 1.2. Gateway answers SSLRequest with `'S'` and upgrades; GSS gets `'N'` . Without a certificate, a service refuses to start unless it listens on loopback. The gateway requires TLS from any client not on this machine: a plaintext startup from another address is answered `28000 this gateway requires TLS` before the password is asked for (`needsTLS`).
HOW TO USE	`psql "postgresql://dbadmin:<key>@localhost:6432/main?sslmode=require"` ; `curl -k https://localhost:8080/api/status` .
HOW TO TEST	Unit: TestReadStartupSkipsSSL, TestReadStartupUpgradesTLS, TestNeedsTLS, TestIsLoopback. Manual: `openssl s_client -connect localhost:8080` .
STATUS & LIMITS	Working. Until 22 Sep 2026 TLS failed open: a certificate that could not load left all three services on plain HTTP / cleartext passwords, and the gateway accepted `sslmode=disable` from anywhere (audit v2 G06). Clients on the same machine may still skip TLS. The default certificate is self-signed, so `sslmode=require` encrypts but does not verify the server; `FOX_TLS_CERT` / `_KEY` for a real one.
ADDED IN	`0bd52f3` ; file lock `b664b56` (PR #11).

E3 Keys are made by a person, not by start

Working

WHAT IT DOES	`fox start` creates no account and no API key. It points at the web UI, and the person who will use the key signs in and makes it.
HOW IT WORKS	Nothing is minted at start. The first account created on an install — through the web sign-up with the setup token, or `fox user create` — gets the admin grant on main (the account with the lowest id, so two sign-ups at once cannot both get it), through `auth.Store.OnFirstUser` (the auth package cannot reach the engine, so the engine sets the hook) and `branch.AdminForFirstAccount`. Keys are made by `fox apikey create` or the API keys page, and shown once.
HOW TO USE	`fox start` prints a setup token; create your account at https://localhost:8080 with it (`fox setup-token` shows it again), or run `fox user create <email>`. Then make a key on the API keys page.
HOW TO TEST	Integration §1 (a fresh install has no account and no key until one is created; the banner and `fox setup-token` show the same token, stored 0600; sign-up without it or with a wrong one is refused; it is deleted with the first account; the first account can override the guardrail). Unit: TestFirstAccountNeedsTheSetupToken, TestOnlyOneFirstAccount.
STATUS & LIMITS	Working. Until 21 Sep 2026 the first start created an account of its own (`local@foxbyte`) with a key cached in `~/.fox/config` in plaintext, and printed it: convenient, but a credential nobody asked for, belonging to an account nobody owned. Both are gone, and so is the cached key. Until 22 Sep 2026 whoever reached the sign-up page first became the admin (audit v2 G03); the first account over HTTP now needs the setup token, which only someone who can read the state directory has.
ADDED IN	`dcc32ab` (PR #11); replaced by sign-in on `feat/foxbyte-ui`.

AREA F

SQL gateway

F1 Routing by database name

Working

WHAT IT DOES	One Postgres endpoint (`:6432`); the database name you connect to is the branch.
HOW IT WORKS	`proxy.Serve` → `handle` : reads the StartupMessage (protocol 3.0, ≤1 MiB), `EnsureRunning(target)` , rewrites `database` → `foxbyte` and `user` →login role, then relays bytes both ways. No dbname → `main` .
HOW TO USE	`psql "postgresql://dbadmin:<key>@localhost:6432/qa?sslmode=require"`
HOW TO TEST	Unit: TestBuildParseStartupRoundtrip. Integration §1, §6.
STATUS & LIMITS	Working. No pooling or rate limiting.
ADDED IN	`a7580f6` .

F2 API-key authentication

Working

WHAT IT DOES	Every client must use a `key_` API key as its Postgres password.
HOW IT WORKS	`requestClientPassword` asks for a cleartext password (inside TLS), `VerifyKey` checks it; bad key → `28P01 invalid API key` . AuthenticationOk is sent only after the backend login succeeds. Startup (TLS, startup message, password) must finish within 30 s, and at most `FOX_GATEWAY_MAX_CONNS` (1000) connections are served at once; the next gets `53300 too many connections` (audit v2 G11). An account key opens main, its account's own branches, and any branch for an admin; any other name is answered `3D000 no branch` before it is woken.
HOW TO USE	`PGPASSWORD=key_… psql "postgresql://dbadmin@localhost:6432/main?sslmode=require"`
HOW TO TEST	Integration §1 (bad key rejected, key accepted). Unit: TestMaxConns, TestNeedsTLS.
STATUS & LIMITS	Working.
ADDED IN	`e42e792` ; auth store `235ff41` .

F3 Backend login (SCRAM)

Working

WHAT IT DOES	The gateway logs in to the branch itself, so clients never see the database password.
HOW IT WORKS	`backendAuth` (`auth.go`) supports cleartext, md5 and SCRAM-SHA-256 using the per-install password; failures → `08006` .
HOW TO USE	Automatic.
HOW TO TEST	Exercised by integration §1, §6, §8 (no unit test for SCRAM).
STATUS & LIMITS	Working.
ADDED IN	`e42e792` .

F4 Per-user Postgres login roles

Working

WHAT IT DOES	Each user logs in as a role named after their email, so the ledger's actor (`session_user`) can't be forged; permissions stay those of non-superuser `db_client` (RLS and GRANTs apply).
HOW IT WORKS	`loginUser` → `branch.EnsureUserRole` : `CREATE ROLE <email> LOGIN NOSUPERUSER … IN ROLE db_client` , `ALTER ROLE … SET role = db_client` ; cached per process; falls back to `db_client` on error.
HOW TO USE	Automatic. Check with `select session_user, current_user;` → `<email>` , `db_client` .
HOW TO TEST	Manual query above (no automated test).
STATUS & LIMITS	Working. Each role has its own password, derived from the install secret (E1); until 22 Sep 2026 all shared the secret itself (audit v2 G10). A branch cloned before then carries the old `db_client` password until something logs in as it: `EnsureAppRole` sets it once per process first. A branch deleted and made again under the same name is a fresh clone, so what a process remembers about its roles is dropped with it (`forgetBranchRoles` , 23 Sep 2026): the caches used to say "already provisioned" and the login failed as a wrong password. Found by the SDK contract tests.
ADDED IN	`96c8c31` (role split `aad84e4`).

F5 Attribution session settings

Working

WHAT IT DOES	Tags every session with who, what kind, which branch and a session id for the Blackbox.
HOW IT WORKS	`ledgerOptions` appends `-c bb.actor=<email> bb.actor_kind=human|agent bb.branch=<b> bb.session=<random>` ; `agent` when the branch name starts with `agent-` .
HOW TO USE	Automatic. `SHOW bb.actor;`
HOW TO TEST	Integration §8 (DDL attributed to the human key).
STATUS & LIMITS	Working; `actor_kind` , `branch` , `session` are ordinary settings a client could change (the actor itself comes from F4).
ADDED IN	`e42e792` .

F6 Ctrl-C (CancelRequest) forwarding

Working

WHAT IT DOES	Pressing Ctrl-C in psql cancels a running query through the gateway.
HOW IT WORKS	`relayStartupCaptureKey` records BackendKeyData (pid:secret → backend address); `forwardCancel` replays a cancel packet to the right backend.
HOW TO USE	`select pg_sleep(60);` then Ctrl-C → "canceling statement due to user request".
HOW TO TEST	Unit: TestReadStartupHandlesCancel. Manual as above.
STATUS & LIMITS	Working; the key map lives in one gateway process.
ADDED IN	`4758737` / `4cccbd6` (PR #17).

F7 Errors & insecure build tag

Working

WHAT IT DOES	Failures reach clients as real Postgres errors; the no-auth bypass can't exist in release builds.
HOW IT WORKS	`sendError` writes FATAL ErrorResponses (`28P01` , `3D000` branch missing, `08006`). `FOX_GATEWAY_NOAUTH` only works in binaries built with `-tags insecure` .
HOW TO USE	Connect to a missing branch → `FATAL: branch "x" does not exist` .
HOW TO TEST	Unit: TestSendError.
STATUS & LIMITS	Working.
ADDED IN	`a7580f6` ; insecure tag `1884445` (PR #12).

AREA G

Accounts & auth

G1 Auth store (SQLite)

Working

WHAT IT DOES	Stores users, API keys, sessions, OAuth identities, pipelines and runs for a single-tenant install.
HOW IT WORKS	`internal/auth`: `OpenFromEnv` (`FOX_DB`, default `~/.fox/auth.db` in the VM), pure-Go SQLite with WAL + busy_timeout so all three services share it. The schema is created and migrated inside `BEGIN IMMEDIATE`, retried on `SQLITE_BUSY`, because `fox start` launches all three services together and on a new install they create the file at the same moment.
HOW TO USE	Automatic.
HOW TO TEST	Unit: TestVerifyKeyConcurrent, TestConcurrentFirstOpen (four stores opening a new file at once, 40 rounds; without the fix 6 of 160 opens failed).
STATUS & LIMITS	Working. Every signed-in user can see every branch (no RBAC). Fixed 17 Sep 2026: on a new install the first `fox start` could leave the control plane and the Agent API down — they lost the race to create the store and exited with `database is locked (SQLITE_BUSY)`.
ADDED IN	`235ff41` (PR #1); WAL `c2dd024`.

G2 Email & password users

Working

WHAT IT DOES	Create an account and log in with email + password.
HOW IT WORKS	`CreateUser` (a bare address, lower-cased, at most 63 bytes — the account's Postgres role is named after it and Postgres cuts role names at 63; bcrypt), `Login` (generic "invalid credentials"); 8-char minimum. `ChangePassword` checks the current one and signs out every other session; `ResetPassword` (admin, CLI) signs out all. `DeleteAccount` removes sessions, keys, OAuth sign-ins, pipelines and branch ownership; the handler also revokes the account's admin grant on main.
HOW TO USE	`fox user create|list|passwd|delete`; `POST /auth/register`, `POST /auth/login`; `POST /api/account/password`; admins: `GET /api/users`, `DELETE /api/users/{id}`. The API keys page has a password form.
HOW TO TEST	Unit: TestPasswordLogin, TestEmailsAreBareAndFitARoleName, TestChangePasswordEndsOtherSessions, TestDeleteAccount. Integration §1, §1d.
STATUS & LIMITS	Working. Until 22 Sep 2026 there was no password change, reset, list or delete, and any string was an email (audit v2 G16). No self-service reset by email: a forgotten password is reset with `fox user passwd <email>`, which needs access to the machine rather than a sign-in — the same thing owning an install means, and the model Grafana and Postgres use. It signs that account out everywhere and is recorded in the security log. **Until 30 Sep 2026 that was true and unfindable:** the command appeared only in `fox help`, which is no use to someone who cannot get in, and was named nowhere on the login page, in the README or in the Guide. It is now on the login page itself, where a locked-out person is actually standing, with the email they typed already filled in; a console test holds it there. Email-based reset stays out of a local install on purpose — there is no mail server in one, and it would add SMTP configuration, a token table, expiry and rate limiting for a path the machine's owner does not need. It belongs with the hosted tier, where users have no shell.
ADDED IN	`235ff41`.

G3 API keys

Working

WHAT IT DOES	Non-interactive credentials for REST (Bearer / `X-API-Key`) and the gateway.
HOW IT WORKS	Secret `key_` + 24 random bytes; only the sha256 is stored; shown once; per-user list/revoke; `last_used` updated at most once a minute.
HOW TO USE	`fox apikey create <email> [name]` · `list` · `revoke`; `GET/POST /api/keys`, `DELETE /api/keys/{id}`; web Keys page.
HOW TO TEST	Unit: TestAPIKeys. Integration §1.
STATUS & LIMITS	Working. Revoking an unknown id still returns success. A key opens what its account may reach (K1): main, its own branches, all of them for an admin.
ADDED IN	`235ff41`.

G4 Web sessions & signup control

Working

WHAT IT DOES	Browser login sessions; sign-up is closed unless opened; failed sign-ins are throttled.
HOW IT WORKS	30-day session token in HttpOnly cookie `dbengine_session`; `Authn` accepts cookie, Bearer or `X-API-Key`. Sign-up is closed unless `FOX_SIGNUP=open` (the first account always needs the setup token, E3); `GET /auth/providers` tells the UI whether the form can be used and whether this is the first account. 10 failed sign-ins for one address or one email in 15 minutes hold further attempts (429, `Retry-After`) — in memory, per process. Auth bodies are capped at 64 KiB.
HOW TO USE	`/auth/login`, `/auth/logout`, `/auth/me`; `FOX_SIGNUP=open fox start` to let people register themselves.
HOW TO TEST	Unit: TestSessions, TestSignupClosedUnlessOpen, TestThrottle, TestLoginIsThrottled, TestAuthBodyIsCapped. Integration §1 (sign-up closed after the first account). Manual: `curl -k https://localhost:8080/auth/providers`.
STATUS & LIMITS	Working. Until 22 Sep 2026 sign-up was open by default and sign-in unthrottled (audit v2 G03, G11). The throttle trusts the connection's address, not `X-Forwarded-For`.
ADDED IN	`235ff41`.

G5 OAuth (GitHub / Google)

Partial

WHAT IT DOES	"Continue with GitHub / Google" sign-in.
HOW IT WORKS	`oauth.go`: state cookie (one use, `Secure` on HTTPS, `SameSite=Lax`, path `/auth/oauth/`), code exchange, identity fetch. A known provider identity signs in its account; otherwise the provider must have **verified** the email (GitHub's primary address from `/user/emails`, Google's `email_verified`): it then joins the account with that email, or becomes a new account only when sign-up is open and the install already has its first account. Redirects to `FOX_WEB_ORIGIN/dashboard`.
HOW TO USE	Set `FOX_GITHUB_CLIENT_ID/_SECRET` or `FOX_GOOGLE_CLIENT_ID/_SECRET`. Nothing else is needed on a default install; set `FOX_PUBLIC_URL` when the engine is reached at another address, and `FOX_WEB_ORIGIN` only when the console is hosted separately.
HOW TO TEST	Integration §1 (the GitHub authorize URL carries `redirect_uri=https%3A%2F%2Flocalhost%3A8080%2Fauth%2Foauth%2Fgithub%2Fcallback` with nothing set, and a password login gets a `Secure`, `SameSite=Lax` cookie). Unit: `TestURLDefaults`, `TestSessionCookieSameSite`, `TestOAuthRules`, `TestStateCookieIsSecure`. Manual, and still untested: a real OAuth app.
STATUS & LIMITS	The defaults now match what the engine serves (`urlDefaults`: public URL `https://localhost:8080`, web origin = public URL), so only the client ID and secret are required. Untested against a real GitHub or Google app. Until 22 Sep 2026 a provider sign-in created accounts even with sign-up closed, and joined any account whose email matched without asking whether the provider had verified it — an account takeover for anyone who could put that address on a GitHub or Google account (audit v2, found alongside G15).
ADDED IN	`235ff41`.

G6 Security log

Working

WHAT IT DOES	A tamper-evident record of what happens at the doors: sign-ins and failed ones, throttled attempts, sign-ups and refused setup tokens, refused OAuth sign-ins, API keys made and revoked, password changes and resets, deleted accounts, admin grants and revokes, branch owners changed, and every request refused for lack of access — at the API, the Agent API and the Gateway (invalid keys included).
HOW IT WORKS	`internal/auth/audit.go` : table `security_events` in the account store; each event's `row_hash` is SHA-256 over `prev_hash` and its fields (`EventHash`), appended under SQLite's write lock (`BEGIN IMMEDIATE`) so two processes cannot chain onto one head; triggers refuse UPDATE and DELETE. Recording never fails the action. The control plane's checkpointer anchors the head in a signed file (`security-anchor/1` , `~/.fox/anchors/_security-log/` , H10's key); `branch.VerifySecurityLog` checks the chain, the anchors and their signatures.
HOW TO USE	`fox audit [--limit N]` , `fox audit verify [--pubkey <file>]` , `fox audit checkpoint` ; admins: `GET /api/audit?limit=N` .
HOW TO TEST	Unit: TestSecurityLogChains (20 concurrent appends chain; the store refuses edits; an edited or removed event is found), TestVerifySecurityLog (a rewritten, re-hashed log fails its anchor; a forged anchor fails its signature). Integration §1d: the events of that section are there, admins read them over the API and others may not, the log anchors signed and verifies, and an event edited behind the store's back is caught.
STATUS & LIMITS	Working (audit v2 G28, 22 Sep 2026). Events after the last anchor are protected only by the chain until the next one. Anyone with the store's file and the signing key could rewrite both; keep a copy of the anchors elsewhere. Not exported with `fox backup export` .
ADDED IN	`feat/blackbox-data-changes` .

AREA H

Blackbox (the record of schema changes, formerly the Schema Ledger)

The ledger lives inside each branch database (`bb.schema_ledger`), installed into `main` by `InstallLedger` during `Init` and inherited by every branch through the copy-on-write clone. Source: `internal/ledger/ledger.sql` .

H1 DDL recording

Working

WHAT IT DOES	Records every schema change — who, with which tool, on which branch, the statement, its status and risk.
HOW IT WORKS	Event triggers: `ddl_command_end` → `log_ddl_end` (one row per object, incl. GRANT/REVOKE and function DDL); `sql_drop` → `log_ddl_drop` . Skips objects in schema `bb` , COMMENT, CREATE EXTENSION, event triggers, CREATE SCHEMA, sequences and `_pkey` indexes. Columns: id, at, actor, actor_kind, tool, session, branch, command_tag, object_type, object_identity, statement, status, risk, prev_hash, row_hash.
HOW TO USE	Automatic for all clients (gateway, console, psql, ORMs, imports).
HOW TO TEST	Integration §8 ("ledger captures DDL, attributed to the human key"). Integration-v2 §9: a stop and start adds nothing to main's or a branch's Blackbox, and a user's change afterwards is still recorded as that user.
STATUS & LIMITS	Working. **Data changes (22 Sep 2026, audit v2 G04):** `internal/ledger/datachanges.sql` gives every user table two triggers — `bb_guard_truncate` (BEFORE TRUNCATE: guarded and recorded, H4) and `bb_record_dml` (AFTER UPDATE OR DELETE, per statement: recorded when the actor is an agent, with the table and the statement; a person's are not, to keep ordinary writes fast; no row counts). An event trigger (`bb_data_attach`) adds them to every new table, including `CREATE TABLE AS` ; dropping, disabling or renaming one is refused without the override. Tested in integration-v2 §10. Not captured: people's DML, INSERT, role/database/tablespace changes (Postgres doesn't fire event triggers for those); some dropped object types (triggers, policies, extensions). Fixed 21 Sep 2026: since GRANT/REVOKE capture was added (PR #13), every `fox start` recorded the Blackbox's own install script — nine `GRANT` / `REVOKE` / `ALTER DEFAULT PRIVILEGES` entries with no actor, per running branch, per start — because `ledger.sql` re-created its event triggers before its privilege block. The triggers are now created last, as the script's header always said they were. Entries recorded before the fix stay: the table is append-only and its chain must keep verifying. **An agent's unqualified data change is now refused, not only recorded** (audit v2 G04): `bb.guard_dml` , a BEFORE statement trigger beside the other two, blocks an `UPDATE` or `DELETE` whose statement carries no `WHERE` at all when the actor is an agent, records the attempt as BLOCKED through a second connection so it survives the rollback, and points at the override. Deliberately generous about what counts as a condition — any `WHERE` anywhere in the statement, including inside a subquery — because a false refusal stops someone working while a false allow only leaves an entry in the record. A person's unqualified delete is their own decision and is not touched; note that every connection to an *agent's* branch is attributed to that agent, so the guard applies there even to an operator's psql, with `bb.allow_destructive=on` available to a superuser or a db_admin member. Driven by the same policy table as TRUNCATE (`bb.policy` , op `UNQUALIFIED DML`).
ADDED IN	`e42e792` ; GRANT/REVOKE/function DDL `db9dc0c` (PR #13); install no longer recorded on `feat/pg-upgrade` .

H2 Hash chain, append-only & verify

Partial

WHAT IT DOES	Makes the ledger tamper-evident: rows can't be edited/deleted normally, and any break in the chain is detected.
HOW IT WORKS	`chain_row` BEFORE INSERT sets `row_hash = sha256(prev_hash|id|at(UTC)|…|risk)` under `pg_advisory_xact_lock`. `bb_append_only` / `bb_no_truncate` triggers + REVOKE block UPDATE/DELETE/TRUNCATE. `LedgerVerify` recomputes every hash and link.
HOW TO USE	`fox ledger verify [branch]`; `GET /api/branches/{name}/ledger/verify`; MCP `verify_ledger`; SDK `verify_ledger()`. Also `fox blackbox verify`, `…/blackbox/verify`, MCP `verify_blackbox`, SDK `verify_blackbox` / `verifyBlackbox` (H14).
HOW TO TEST	Integration §8 (plain DELETE blocked; 0 broken rows). Manual: `fox ledger verify` → "ledger intact — N rows…".
STATUS & LIMITS	A superuser can bypass the triggers (`session_replication_role=replica`) and rewrite the whole chain so this check still passes. Since Phase 3, `fox ledger integrity` and `fox-verify` (H10–H12) catch that by checking against anchors kept outside the database. Changes to `bb.policy` are now recorded, with who made them, in the append-only `bb.policy_history` (17 Sep 2026); that history is not hash-chained or anchored, so a superuser can still rewrite it unseen. Since 22 Sep 2026 anchors are signed (H10), so a rewritten anchor is caught too by whoever holds the public key.
ADDED IN	`db9dc0c`, `d14d9a1` (PR #13).

H3 Attribution

Working

WHAT IT DOES	Records who made each change and whether it was a human or an agent.
HOW IT WORKS	`bb._ctx` derives both attribution fields from the login role, which a client cannot change (`SET ROLE` leaves `session_user` alone): actor = `session_user` for a per-user role (F4) or an agent role, falling back to the injected `bb.actor` only for the engine's own shared roles (`foxbyte`, `db_client`); kind = `agent` for an `agent-<id>` role, `human` for a per-user role — one named for an email address is a person even if the address starts with "agent-" — and for the shared roles the injected `bb.actor_kind`. tool = `application_name` and session = `bb.session` (or the backend pid) stay as the client declares them, and are labelled **declared** on the Blackbox page. Console sets actor to the signed-in email; agent branches get `bb.actor='agent-<id>'` database defaults; MCP passes its key's account.
HOW TO USE	Automatic.
HOW TO TEST	Unit: `TestActorKindDerivedFromRole` (the kind is derived, and the session setting is only the shared-role fallback). Integration §8 (actor_kind = human); integration-v2 §1c: a human session that sets `bb.actor_kind='agent'` is still recorded as human, and an agent session that sets `'human'` is still recorded as an agent.
STATUS & LIMITS	Non-forgeable where an identity exists: gateway users log in as their own role, each agent branch as its own `agent-<id>` role, and the kind now follows the role rather than a session setting — until 18 Sep 2026 a client could `SET bb.actor_kind` and be recorded as the other kind, which is exactly what this record exists to prevent. The engine's own shared roles (console, MCP, imports) have no identity of their own, so their sessions still carry an injected actor and kind — set by engine code, not by the client. Tool and session are declared by the client by design (an `application_name` is whatever a client sends) and are shown as declared.
ADDED IN	`e42e792`; non-forgeable `96c8c31` (PR #16); kind derived from the role `feat/attribution-mcp-auth`.

H4 Destructive-DDL guardrail

Working

WHAT IT DOES	Refuses `DROP TABLE` and `DROP SCHEMA` before they run, and still records the attempt.
HOW IT WORKS	`ddl_command_start` → `guard_ddl_start` looks up `bb.policy` (seeded DROP TABLE/SCHEMA → block), honours `bb.allow_destructive` only when `bb._may_override()` is true (H8), and raises `insufficient_privilege` with a HINT; a `dblink` insert records status `BLOCKED` in its own transaction so it survives the rollback. Imports/pipelines disable the trigger temporarily (I9).
HOW TO USE	Automatic: `DROP TABLE t;` → "FoxByte guardrail: DROP TABLE is blocked by policy".
HOW TO TEST	Integration §8 (DROP blocked; BLOCKED row recorded). integration-v2 §6b: `flag` runs the command and records it `FLAGGED`; an unknown action is refused; a blocked DROP TABLE after a write in the same transaction returns the unchanged error at once instead of hanging.
STATUS & LIMITS	Since Phase 1 the override is honoured only for superusers and members of `db_admin` (H8); anyone else stays blocked. Each use of the override is recorded in `bb.ledger_ext.override_used` (H9). `block`, `flag` (runs, recorded `FLAGGED`) and `allow` (runs, as if unlisted) work, and any other action is refused (`policy_action_valid`, `NOT VALID` so older rows are left alone); every change is recorded in `bb.policy_history`. No CLI/API/UI to manage `bb.policy` yet. Fixed 17 Sep 2026: `flag` did nothing, a typo such as `blok` silently let the command through, and a blocked command after a Blackbox write in the same transaction (`BEGIN; CREATE TABLE …; DROP TABLE …;`) hung the client forever — the dblink connection recording the attempt waited for the append lock its own session held. That attempt now isn't recorded (a WARNING says so) and the block stands. Broader, rule-based checks with warn/block and a machine-readable error are the policy gate (H15), which runs after this guardrail and leaves it unchanged. **TRUNCATE** (22 Sep 2026): blocked by default like DROP TABLE (`bb.policy` op `TRUNCATE`, the same override and error, a BLOCKED entry naming the table); allowed, it is recorded with risk `truncate`. Imports and pipelines, which switch the guardrail off for their load, may truncate, and it is recorded.
ADDED IN	`e42e792`.

H5 Risk flags

Working

WHAT IT DOES	Marks risky changes so they stand out.
HOW IT WORKS	Regexes on the statement that ran set status `FLAGGED` with risk `type-change`, `drop-column` or `security-definer`; drops get risk `drop`, blocked attempts and commands the guardrail marks `flag` get `policy`. The statement comes from `bb._statement_texts` (`ledger.sql`): the query as sent is split into statements with comments removed and string literals emptied, and a position per backend — in a table clients cannot write, moved by a function they cannot call — says which one is running. DDL run from inside a function or `DO` block is judged by the statement Postgres reports running (`PG_CONTEXT`). When the statement can't be established, the whole query is used, as before.
HOW TO USE	Automatic; filter with `?risk=` / `?status=FLAGGED`.
HOW TO TEST	integration-v2 §6b (17 Sep 2026, 24 checks): the tokenizer on comments, nested comments, standard, `E'…'` and dollar-quoted literals, quoted identifiers and multibyte text; a literal mentioning drop column is not flagged; in add-then-drop only the drop is. Unit: `internal/ledger/statement_test.go` pins the safeguards (checked by deliberately removing two of them).
STATUS & LIMITS	Working. Still text matching on the statement, not dependency analysis (impact analysis, H17, does that). Query strings over 256 KB, and runs whose statements can't be lined up with the commands that fire, fall back to the whole query.
ADDED IN	`e42e792`; security-definer `db9dc0c`.

H6 Blackbox access (CLI / REST / MCP / UI / SDK)

Working

WHAT IT DOES	Read the ledger from anywhere.
HOW IT WORKS	CLI `Ledger` (psql table, default 50 rows). REST `GET /api/branches/{name}/ledger` with filters actor, table, risk, status, kind, since, until, limit (≤2000), offset. MCP `changes`. Web Ledger page (L5). SDK `ledger()`.
HOW TO USE	`fox ledger qa --limit 20`; `curl -k -H "Authorization: Bearer $KEY" "https://localhost:8080/api/branches/main/ledger?status=BLOCKED"` Blackbox names: `fox blackbox [branch]`, `GET /api/branches/{name}/blackbox`, web `/blackbox`, SDK `blackbox()` (H14).
HOW TO TEST	No automated test of the endpoints; manual curl above.
STATUS & LIMITS	Working. Any signed-in user can read any branch's ledger. No export format.
ADDED IN	`e42e792`; MCP `d62c676` (PR #14).

H7 `fox ledger revert --to`

Working

WHAT IT DOES	A point-in-time restore of `main` into a disposable container on port 5433 — the same as `fox restore`. Nothing is reverted: main and every branch stay as they are. The help and the command's output say so, and point to `fox blackbox branch-before <id>` (H13) for a branch holding main as it was just before a change.
HOW IT WORKS	Calls `branch.Restore(ts)` (D3). No branch argument.
HOW TO USE	`fox blackbox revert --to '<timestamp>'|latest` (also `fox ledger revert`)
HOW TO TEST	Unit: `TestUsageDescribesRevertAndFailback` (the help describes a restore of main, not a revert of a branch). The restore itself: integration §3 (D3).
STATUS & LIMITS	Working. The name is kept so scripts don't break; its description no longer claims it reverts a branch (it did until 15 Sep 2026). Inherits D3's limit: always restores from the latest base backup.
ADDED IN	`e42e792`.

H8 Override permission (db_admin) & ledger upgrade

Working

WHAT IT DOES	Decides who may override the destructive-DDL guardrail, and applies the current ledger definition to branches that already exist.
HOW IT WORKS	`ledger.sql` creates role `db_admin` and `bb._may_override()` — true for superusers or members of `db_admin`, checked on `session_user` (the login identity a client can't change). `fox admin grant` ensures the user's per-user role and runs `GRANT db_admin` (`internal/branch/security.go`, `cmd/fox/admin.go`). Roles live in each branch's Postgres, so without `--branch` a change applies to main and every running branch; new branches inherit main's. The first local user created by `fox start` is granted automatically. `fox ledger upgrade` re-runs the idempotent `ledger.sql`; main is also upgraded on every `fox start`.
HOW TO USE	`fox admin grant you@x.com` · `fox admin list` · `fox admin revoke you@x.com --branch qa` · `fox ledger upgrade --all` · or the **Who can override** panel on the Policies page (`GET|POST /api/branches/{name}/admins`, `DELETE …/admins/{email}`).
HOW TO TEST	integration-v2 §1b: non-admin override refused with a HINT, admin override works, revoke restores the block, superuser override (imports/pipelines) unaffected. §6: the REST list says whether the caller is an admin, a non-admin can't grant (403), unknown accounts and non-admins are 404. Live on 17 Sep 2026: an unknown email was 404 and removing the only admin was refused (409).
STATUS & LIMITS	Verified in the VM (integration-v2 §1b, 14 Sep 2026). **Existing installs:** the owner runs `fox admin grant <email>` once (only brand-new installs grant the first user automatically), and existing branches need `fox ledger upgrade --all`. Suspended branches are skipped until running. From the web, only an admin of a branch may grant or revoke there (otherwise anyone could grant themselves), and the last admin can't be removed from the web.
ADDED IN	Blackbox 2.0 Phase 1 (`feat/ledger2-phase1-security`).

H9 Blackbox 2.0 capture (transaction, WAL position, provenance)

Working

WHAT IT DOES	Records extra context for every ledger row: the transaction id and WAL position of the change, the agent task and tool-call hash when the session sets them, and whether the destructive-DDL override was in effect. It is the foundation for branching from just before a change (Phase 4) and agent provenance (Phase 6).
HOW IT WORKS	New file `internal/ledger/ledger_v2.sql`, which never alters an object `ledger.sql` owns. Side table `bb.ledger_ext`, keyed by ledger row id, is filled by an `AFTER INSERT` trigger `bb_ext_capture` that reads `txid_current()`, `pg_current_wal_insert_lsn()` and the settings `bb.task`, `bb.parent_session`, `bb.call_hash` and `bb.allow_destructive`. Each row carries `ext_hash` (`bb._ext_hash`). Append-only like the ledger; clients may read, not write. **Fail-safe:** any capture error becomes a WARNING and the user's change still goes through. **Kill switch:** `ALTER DATABASE foxbyte SET bb.v2 = 'off'` (needs the database owner or a superuser), or `SET bb.v2 = 'off'` in a superuser's own session. A client — including a `db_admin` member — setting it in its own session is ignored (`bb._capture_disabled`), so nobody can hide the capture details of their own changes. Installed on main at every `fox start` (best effort — never stops start-up) and by `fox ledger upgrade`.
HOW TO USE	Automatic. Tag a session: `PGOPTIONS='-c bb.task=task-42' psql …`. Inspect: `SELECT * FROM bb.ledger_ext ORDER BY ledger_id DESC LIMIT 5;` Installed version: `SELECT bb.ledger_v2_version();` The Agent API and MCP `execute_change` fill the task, parent session and call hash automatically (H16).
HOW TO TEST	integration-v2 §2: one capture row per ledger row; xid and LSN present; task and call hash captured; hash verifies; override use recorded; a failing capture doesn't block DDL; kill switch (database-wide, and a superuser session); a client's own `SET bb.v2 = 'off'` — also as `db_admin` — does not skip capture; append-only; clients can't write; base chain still verifies.
STATUS & LIMITS	Verified in the VM (integration-v2 §2, 14 Sep 2026). Ledger rows written before 2.0 was installed have no capture row. Blocked attempts are inserted through a separate dblink session, so their capture carries that session's transaction and no task. Existing branches need `fox ledger upgrade --all`. Capture hashes are covered by checkpoint anchors (H10).
ADDED IN	Blackbox 2.0 Phase 2 (`feat/ledger2-phase2-capture`).

`H10` Checkpoints & anchors

Working

WHAT IT DOES

Takes tamper-evidence outside the database. At intervals it records a Merkle root over the newest ledger entries — in the database and as a read-only anchor file somewhere else — so rewritten, deleted or wiped history can be proven later.

HOW IT WORKS

`bb.ledger_checkpoints` (`ledger_v2.sql`) is append-only, and a trigger refuses any checkpoint that doesn't continue the last one (`from_id` = last `to_id` + 1, `prev_root` = last root). The engine (`internal/branch/checkpoint.go`) reads the new entries, refuses if their hash chain is already broken, and builds the anchor with `internal/ledger/integrity.go` : each leaf is sha256(0x00 ‖ recomputed row hash ‖ "|" ‖ capture hash), each node sha256(0x01 ‖ left ‖ right). It records the checkpoint, then writes `~/.fox/anchors/<branch>/<to_id>.json` with mode 0444, never overwriting. A scheduler in the control plane checkpoints running branches every `FOX_CHECKPOINT_INTERVAL` (10m) or once `FOX_CHECKPOINT_ENTRIES` (500) are waiting; `FOX_CHECKPOINTS=off` disables it. `FOX_ANCHOR_DIR` moves anchors (e.g. to a write-once mount); `FOX_ANCHOR_IMMUTABLE=1` adds `chattr +i` . The format is published in `docs/ledger-anchor-format.md` . **Signed (22 Sep 2026, audit v2 G17):** each anchor carries `key_id` and an Ed25519 `signature` over a fixed payload (`ledger.SigningPayload` , specified in docs/ledger-anchor-format.md), by `~/.fox/anchor-signing.key` or `FOX_ANCHOR_KEY` — a file no database can read. `fox blackbox anchor-key` prints the public key. **Off the machine (22 Sep 2026):** with a remote backup target, every anchor — each branch's and the security log's — is copied to `<target>/anchors/<dir>/` after it is written (`MirrorAnchors` ; a `.mirrored` marker beside each, so a failed copy is retried with the next checkpoint). In an Object Lock bucket it cannot then be changed or deleted before its retention ends, even by whoever controls this machine.

HOW TO USE

`fox ledger checkpoint [branch]` · `POST /api/branches/{name}/ledger/checkpoint` · web Integrity page → Create checkpoint

HOW TO TEST

Unit: `internal/ledger/integrity_test.go` . integration-v2 §3: anchor written read-only in the published format; no-op when nothing is new; the database refuses an out-of-sequence checkpoint; checkpoints are append-only. §3b: the scheduler anchors new entries on its own.

STATUS & LIMITS

Verified in the VM (integration-v2 §3 and §3b, 14 Sep 2026). Anchors are only as trustworthy as where they're stored — the default directory is on the same host, so point `FOX_ANCHOR_DIR` at storage the database's operators can't rewrite. Entries written since the last checkpoint are protected by the hash chain only. Cloud write-once anchor writers (S3 Object Lock etc.) are not built (commercial scope). A cloned branch inherits its parent's checkpoint rows but not its anchor files. A signature proves an anchor came from the key holder; on one machine the administrator is one, so anchors held elsewhere remain the proof against them. Off-host storage (S3 Object Lock) comes with the remote backup target.

ADDED IN

Blackbox 2.0 Phase 3 (`feat/ledger2-phase3-checkpoints`).

`H11` fox-verify — independent verifier

Working

WHAT IT DOES

A separate open-source program anyone can run to check a ledger without trusting FoxByte, its engine or the database.

HOW IT WORKS

`cmd/fox-verify` reads every ledger entry over a plain Postgres connection (read access through the gateway with an API key is enough) or from a `fox ledger export` file, recomputes every row hash and Merkle root with the standard-library code in `internal/ledger/integrity.go` , and compares them with the anchor files. Exit 0 = intact, 1 = tampered, 2 = usage or connection error; `--json` prints the full report. Built and released as `fox-verify-<os>-<arch>` next to `fox` . `--pubkey <file>` checks every anchor's signature: unsigned anchors are accepted only before the first signed one, and a mismatched signature or another key's fails (exit 1). Without it, the report says signatures were not checked.

HOW TO USE

`fox-verify --dsn 'postgresql://dbadmin:<api-key>@localhost:6432/main?sslmode=require' --anchors ~/.fox/anchors/main` · offline: `fox ledger export main > main.jsonl` then `fox-verify --export main.jsonl --anchors ./anchors/main`

HOW TO TEST

Unit tests (shared verification code). integration-v2 §3: agrees with integrity on a live branch and on an export; exits 1 on a forged edit. integration-v2 §3: `--pubkey` accepts the install's anchors and refuses one whose signature was altered. Unit: TestAnchorSignatures, TestVerifySignaturesSequence.

STATUS & LIMITS

Verified in the VM (integration-v2 §3 and §3b, 14 Sep 2026). Needs a copy of the branch's anchor files. Reads the whole ledger into memory.

ADDED IN

Blackbox 2.0 Phase 3 (`feat/ledger2-phase3-checkpoints`).

H12 Blackbox integrity & export

Working

WHAT IT DOES	Answers "has this branch's schema history been tampered with?" and exports the full ledger for audits.
HOW IT WORKS	`Integrity` (`internal/branch/checkpoint.go`) loads every entry and the branch's anchor files and runs the same checks as `fox-verify` : the hash chain, the anchor sequence, and each anchor's entry count, Merkle root and last entry. It notes when the database's own checkpoint list disagrees with the anchors (the anchors win). Unlike `fox ledger verify` , which only checks the chain inside the database, it catches a rewritten chain. Export writes every entry with its capture columns as JSON lines.
HOW TO USE	`fox ledger integrity [branch]` (exit 1 when tampered) · `fox ledger export [branch] > ledger.jsonl` · REST `GET …/ledger/integrity` , `GET …/ledger/export` · MCP `ledger_integrity` · web Integrity page
HOW TO TEST	integration-v2 §3: an untampered branch is INTACT via CLI, REST, MCP and `fox-verify` ; a forged edit with a rewritten chain fools the in-database check but is TAMPERED here; so are a deleted newest anchored entry and a wiped ledger; deleting the database's checkpoint rows leaves it INTACT because the anchors are the source of truth.
STATUS & LIMITS	Verified in the VM (integration-v2 §3 and §3b, 14 Sep 2026). Reads the whole ledger on each check (fine at typical sizes; very large ledgers would need streaming). `fox blackbox integrity` also checks signatures against the install's own public key when it exists.
ADDED IN	Blackbox 2.0 Phase 3 (`feat/ledger2-phase3-checkpoints`).

`H13` Branch from just before a Blackbox entry

Working

WHAT IT DOES	Creates a new branch holding main exactly as it was the moment before a chosen schema change — to inspect what things looked like, or to recover from a bad change — without touching main.
HOW IT WORKS	`BranchBeforeEntry` (`internal/branch/branch_before.go`) reads the entry and its captured transaction id and WAL position (H9) from the current primary, and refuses BLOCKED entries (the change never happened), sources other than main (only the primary archives WAL) and names already in use. It switches main to a new WAL segment and waits for the archiver, so the change's commit is in object storage. It lists base backups (`wal-g backup-list --json --detail` in a throwaway container) and picks the newest backup *of the same database* (system identifier) that finished before the change — by WAL position, or by time for entries without capture. It creates empty storage for the new branch and runs the stock image with an embedded script (`internal/branch/restore_before.sh` , passed with `bash -c` , so the published image needs no new file): `backup-fetch` of that backup, `recovery_target_xid` (or `recovery_target_time`) with `recovery_target_inclusive = off` , then promote. Once promoted it checks the entry is absent, clears the recovery settings (`ALTER SYSTEM RESET`), restarts the data directory as an ordinary branch container and re-enables the guardrail. Any failure removes the partial branch. One runs at a time.
HOW TO USE	Find the entry id with `fox ledger entries [--limit N]` (REST `GET /api/branches/main/ledger/entries` , MCP `ledger_entries` ; the existing ledger views don't show ids and are unchanged), then `fox ledger branch-before 612 [--as fix-orders]` (default name `main-before-612`) · REST `POST /api/branches/main/ledger/612/branch` with optional `{"name": "…"}` (201; 400 invalid or BLOCKED, 404 no such entry, 409 name taken, 422 no usable base backup) · MCP `branch_before_change` · the web **Blackbox** page, where expanding any entry on `main` offers "Branch from before this change" (the entry ids come from `with=chain`) · the web Integrity page → "Branch from before a change", which always acts on `main` and now says so rather than leaving its Branch selector to be read as applying to it. Connect through the gateway like any branch; remove it with `fox branch delete` . Documented in the README, the Guide's step 6 and the reference — before this it worked and appeared in no user-facing page.
HOW TO TEST	Unit: `internal/branch/branch_before_test.go` and `ledger_entries_test.go` (entry listing, entry parsing, xid/time targets, backup-list parsing, base-backup choice incl. another database's backups, WAL-archive check, names, the embedded script). integration-v2 §4: the entry id is listed by CLI, REST and MCP; by transaction id via the CLI — rows written before the change (even after the backup) are present, the change and later changes are not, the ledger stops before the entry, no recovery settings left, reachable through the gateway with the guardrail active, main untouched and its integrity result unchanged; by timestamp via MCP for an entry without capture; via REST; refusals for an existing name, a BLOCKED entry, an unknown entry, a non-main source and a bad id, with nothing left behind.
STATUS & LIMITS	Verified in the VM (integration-v2 §4, 14 Sep 2026: by transaction id, by timestamp, via CLI, REST and MCP). Takes minutes, not seconds (base backup fetch plus WAL replay), and the REST call waits for it. Needs a base backup that finished before the change (`fox backup create` ; schedule backups to make older changes reachable). It excludes the change's whole transaction, so other statements in the same transaction are excluded too. Entries recorded before Blackbox 2.0 capture use their timestamp, which also excludes anything that committed after the change started. Only main can be the source; after an HA failover the promoted standby doesn't archive WAL, so later changes can't be branched before. The new branch keeps main's checkpoint rows up to that point but not its anchor files, so `fox ledger integrity` on it notes the difference.
ADDED IN	Blackbox 2.0 Phase 4 (`feat/ledger2-phase4-branch-before`).

H14 Blackbox names & aliases

Working

WHAT IT DOES	Blackbox is the product name for the record of schema changes that was called the Schema Ledger. Every surface gains a Blackbox name; every original name keeps working, so no script, integration, stored record or anchor breaks.
HOW IT WORKS	**CLI:** `fox blackbox …` runs the same handler as `fox ledger …` (`cmd/fox/main.go`). **REST:** `blackboxAlias` (`internal/controlplane/blackbox.go`) sits inside the auth gate and serves `/api/branches/{name}/blackbox[/…]` with the `…/ledger[/…]` handler. **MCP:** `verify_blackbox` , `blackbox_integrity` and `blackbox_entries` are listed next to and resolve to `verify_ledger` , `ledger_integrity` , `ledger_entries` (`resolveTool` in `internal/mcp/mcp.go`). **SDKs:** Python `blackbox()` / `verify_blackbox()` , TypeScript `blackbox()` / `verifyBlackbox()` call the original routes, so they also work against older servers. **Web:** nav "Blackbox" at `/blackbox` ; `/ledger` still opens the same page. **Kept as they were:** SQL objects (`bb.schema_ledger` , `bb.ledger_ext` , `bb.ledger_checkpoints` , `bb._ledger_hash`), the Go package `internal/ledger` , the anchor format `ledger-anchor/1` and `docs/ledger-anchor-format.md` file name, output lines such as `ledger integrity: INTACT` , JSON fields such as `last_ledger_id` , the export file name.
HOW TO USE	`fox blackbox` · `fox blackbox integrity main` · `curl -k -H "Authorization: Bearer $KEY" https://localhost:8080/api/branches/main/blackbox/verify` · MCP `blackbox_integrity` · `db.verify_blackbox("main")`
HOW TO TEST	Unit: `internal/controlplane/blackbox_test.go` (TestBlackboxToLedgerPath, TestBlackboxAlias), `internal/mcp/mcp_test.go` (TestBlackboxToolAliases). integration-v2 §5: CLI verify/entries/integrity, REST verify/list/entries and the auth requirement, and MCP listing, integrity and verify all match the original names.
STATUS & LIMITS	Verified in the VM (integration-v2 §5, 14 Sep 2026; existing suite 28 / 28). The OpenAPI spec documents the alias in the ledger tag and info description rather than repeating every path, so generated clients only get the `/ledger` operations. The screenshot file is still `docs/screenshots/ledger.png` and shows the old page title until recaptured.
ADDED IN	Blackbox rename (`feat/blackbox-phase5-policy`).

H15 Policy gate (warn / block)

Working

WHAT IT DOES	Checks every schema change against policy rules before it runs. A **warn** rule lets the change through and tells the client why it is risky; a **block** rule refuses it. Tools and agents get a machine-readable answer they can act on, defined in the published contract `docs/policy-errors.md`.
HOW IT WORKS	`internal/ledger/policy.sql`, installed after the 2.0 objects by `EnsureLedgerV2` (every `fox start`, `fox blackbox upgrade`). Rules live in `bb.policy_rules` (rule id, command tag, optional case-insensitive regex over the statement, warn|block, reason, hint, enabled); changes go to the append-only `bb.policy_rule_history` with who made them. The event trigger `bb_policy_start` fires on `ddl_command_start` after the guardrail `bb_guard_start` (name order). **Warn:** an evaluation row and a NOTICE with SQLSTATE `BBX02`. **Block:** a `BLOCKED` Blackbox entry (risk `policy`) and an evaluation written through dblink so they survive the rollback, then an ERROR with `BBX01`; blocks win over warnings. DETAIL is one-line JSON (`v, rule_id, action, command, matched, reason, hint, override, evaluation_id, blackbox_id, impact`). **Override:** superusers and `db_admin` members may `SET bb.policy_allow = '<rule ids>'`; it is recorded as an `allowed` evaluation and in the capture row's `override_used`; `bb.allow_destructive` does not override policy. **Fail-safe:** any error while matching or recording becomes a WARNING and never stops a statement a warn rule matched; a block still refuses (with `evaluation_id` null). The gate ignores the client-settable `bb.v2`; a superuser turns it off with `ALTER EVENT TRIGGER bb_policy_start DISABLE`. Rule changes through REST need `db_admin` on that branch (`IsAdmin`); clients can read rules and evaluations but not write them. The existing guardrail (`bb.policy`, DROP TABLE/SCHEMA, SQLSTATE 42501) is unchanged.
HOW TO USE	`fox policy` · `fox policy check "ALTER TABLE orders DROP COLUMN note"` (exit 1 if a rule would block) · `fox policy block drop-column` · `fox policy add no-orders-drop --command "ALTER TABLE" --pattern 'orders' --block --reason "orders is shared"` · `fox policy evaluations` · REST `GET|POST /api/branches/{n}/policies`, `PUT|DELETE …/policies/{rule}`, `POST …/policies/check`, `GET …/policies/evaluations` · MCP `policy_check` · web Policies page. In an application: catch SQLSTATE `BBX01` and parse the error's DETAIL as JSON.
HOW TO TEST	integration-v2 §6b (statement matching, previews, blocks in multi-statement runs, `DO` blocks, clients can't move the position). Unit: `internal/ledger/statement_test.go`, `internal/ledger/policy_test.go` (TestSchemaPolicy — both SQLSTATEs, fires after the guardrail, built-ins ship as warn, ignores `bb.v2`, touches no existing object; TestParsePolicyDetail; TestPolicyContractExample — the contract's own example parses; TestCommandTag; TestValidRuleID) and `internal/branch/policy_test.go` (validation, rule SQL quoting, error messages, formatting). integration-v2 §6: warn (runs, BBX02, contract JSON, evaluation) for all four built-ins; block (BBX01, statement didn't run, JSON with ids, BLOCKED entry and evaluation survive the rollback, HINT names the override); non-admin override, `bb.v2=off` and `bb.allow_destructive` don't bypass; admin override runs and is recorded; REST list / 200 / 403 / 404 / 409 / check / evaluations; CLI check and exit code; MCP `policy_check`; clients can't write rules; fail-safe; kill switch; custom and disabled rules; rule history; invalid pattern; built-in can't be removed; append-only evaluations; guardrail error unchanged; reinstall keeps rule changes; hash chain still verifies.
STATUS & LIMITS	Verified in the VM (integration-v2 §6, 14 Sep 2026; existing suite 28 / 28). Patterns match the statement being run, with comments removed and string literals emptied (see H5 and docs/policy-errors.md); where that statement can't be established — DDL inside a function is matched by the statement Postgres reports, a query over 256 KB or a run that falls out of step — the whole query is used, as before. A `DO` block that builds DDL from strings is now caught. Rules are a guard against mistakes, not a security boundary against a client determined to evade a text pattern. `fox policy check` guesses the command tag from the statement's first words; the gate itself uses the real tag. TRUNCATE is not covered (it doesn't fire DDL event triggers). Imports and pipelines run through the gate too (with the built-in warn rules that only adds notices). Rules are per branch; new branches copy main's. The `impact` field is always null until impact analysis (Phase 7).
ADDED IN	Blackbox 2.0 Phase 5 (`feat/blackbox-phase5-policy`).

`H16` Agent provenance & execute_change

Working

WHAT IT DOES

Answers "which agent run, for which task, spawned by what, made this change?" for every change an agent makes — and gives agents one tool that runs a change with that provenance attached, checks policy first, and says exactly what happened.

HOW IT WORKS

`internal/ledger/provenance.sql` (installed by `EnsureLedgerV2`) adds `bb.agent_sessions`: session id, agent, parent session, task, tool, start time; append-only, written only by the engine, readable by clients. A session's changes carry its id in the Blackbox entry's `session` and the task, parent session and call hash in the capture row (H9), which checkpoints anchor (H10). **Agent API:** `POST /agents/{id}/branch` with `{"task_id", "parent_session_id", "session_id"?}` creates the branch, records the session (tool `agent-api`) and sets `bb.session`, `bb.task` and `bb.parent_session` as the branch database's defaults, so the agent's own connections are attributed without it doing anything; if recording fails the branch is removed. Invalid ids (not 1–200 printable characters) get 400 before any branch is made. Without those fields the original code path and response are used. **MCP** **`execute_change`** (`branch.ExecuteChange`): one session per MCP server process (agent = the client's name from `initialize`); previews the policy rules (H15); unless `dry_run`, records the session, connects as the non-superuser `db_client`, sets the actor, `actor_kind = agent`, tool `mcp`, session, task, parent session and `bb.call_hash` (sha256 over the tool, branch, sql, task and parent session), runs the SQL, and returns JSON: `status` (`applied` | `blocked` | `error` | `preview`), `command`, `session_id`, `call_hash`, `policy_preview`, `notices` (with the parsed `BBX02` detail), `policy` (the `BBX01` rule), `error` and `blackbox_entries`. Blocked and error results are also flagged `isError`. `run_sql` is unchanged. Its result also carries `impact`, the impact report for the statement when its target is recognised (H17).

HOW TO USE

`curl -k -H "Authorization: Bearer $KEY" -X POST -d '{"task_id":"JIRA-42","parent_session_id":"planner-1"}' https://localhost:8088/agents/alice/branch` · MCP `execute_change {"sql": "ALTER TABLE orders ADD COLUMN note text", "task_id": "JIRA-42"}` (add `"dry_run": true` to preview) · `fox blackbox sessions [branch]` · REST `GET /api/branches/{n}/blackbox/sessions` · SQL: `SELECT s.id, s.actor, a.task_id, a.parent_session_id FROM bb.schema_ledger s JOIN bb.agent_sessions a ON a.session_id = s.session`

HOW TO TEST

Unit: `internal/ledger/provenance_test.go` (TestSchemaProvenance) and `internal/branch/provenance_test.go` (TestValidProvenanceID, TestNewSessionID, TestCallHash, TestProvenanceSQL, TestClassifyExecError, TestFormatAgentSessions). integration-v2 §7: sessions table installed, clients can't write it; Agent API without provenance keeps its response keys and sets no session; with provenance returns and records session / task / parent, and the agent's own DDL over its DSN carries them; an invalid task id is a 400 with no branch made; `execute_change` applies and returns the entry, which carries agent, tool, session, task, parent and call hash; the MCP session is recorded; runs without superuser rights; a warn rule gives `BBX02` in `notices`; a block gives status `blocked` with the rule and its BLOCKED entry, and the change didn't run; `dry_run` runs nothing; the call hash is stable for the same arguments and differs otherwise; a database error is reported; an invalid task id is refused; CLI and REST sessions; the hash chain verifies.

STATUS & LIMITS

Verified in the VM (integration-v2 §7, 14 Sep 2026; existing suite 28 / 28). Provenance is supplied by the caller and not verified — it records what the agent said its task and parent were; the agent's identity itself stays non-forgeable (login role, H3). Agent API provenance is set per branch, so all connections to that agent branch share one session; start a new branch for a new session. An MCP session lasts as long as the MCP server process. A session recorded on an agent branch is removed together with that branch. `bb.agent_sessions` itself isn't anchored by checkpoints (the ids it explains are). No web view yet. Fixed 17 Sep 2026: since agents connect through the Gateway (K5) the Gateway gave every connection a session of its own, which overrode the branch's — an agent's changes carried a random session instead of the one it was created with. An agent's branch-scoped key now gets no Gateway session, so the branch's applies (integration-v2 §7 caught it).

ADDED IN

Blackbox 2.0 Phase 6 (`feat/blackbox-phase6-provenance`).

H17 Impact analysis

Working

WHAT IT DOES	Before a schema change runs, shows what it would affect: what depends on the object, which other branches have it, what the policy gate would say, and how risky it is overall — for people (`fox impact`) and for agents (MCP `impact` , and inside every `execute_change` result).
HOW IT WORKS	**Target:** `ledger.ParseTarget` reads the statement to find the table, view, materialized view, index or sequence it changes and, where relevant, the column and kind of change (`drop` , `drop-column` , `alter-column-type` , `rename` , `rename-column` , `truncate` are destructive; `alter` , `alter-column` , `create-index` , `grant` , `revoke` are not). Quoted and schema-qualified names work; `--object` / `--column` name it directly. **Dependents:** the read-only SQL function `bb.blast_radius(target, column, depth)` (`internal/ledger/impact.sql` , installed by `EnsureLedgerV2` , runs with the caller's rights and search path, so an unqualified name resolves as in the caller's own statement) walks `pg_depend` for normal and automatic dependencies and reports each dependent's type, identity, owning relation, depth and the view it was reached through: views and materialized views (and views on those, up to depth 5, capped at 10), constraints including foreign keys from other tables, indexes, triggers, row-level security policies, defaults, owned sequences and SQL-standard functions. For a column, only what uses that column. Every reported name is schema-qualified, and an index is reported against the table it indexes. **Branches:** every other running branch is checked for the object (and column) without waking suspended ones, with the branch it was cloned from (ZFS `origin` , btrfs parent UUID). **Policy:** the rules the statement would trigger (H15). **Score** (`scoreImpact`): destructive 5, otherwise 1; each dependent view, function or foreign key from another table +3, trigger or policy +2, index or other +1; deeper than checked +3; ≥10k rows +1, ≥1M +3; each other running branch with it +1 (max 5); a blocking rule +5, a warning +2. 15+ high, 5–14 medium, below low; every point is listed as a reason.
HOW TO USE	`fox impact "ALTER TABLE orders DROP COLUMN note"` · `fox impact --object orders_summary --json` · `fox impact "DROP TABLE orders" --branch qa` · REST `POST /api/branches/{n}/impact` with `{"sql": …}` or `{"object": …, "column": …}` · MCP `impact` · SQL: `SELECT bb.blast_radius('public.orders', 'note')`
HOW TO TEST	Unit: `internal/ledger/impact_test.go` (TestSchemaImpact — read-only, capped depth, caller's rights; TestParseTarget — 27 statements; TestTargetDestructive) and `internal/branch/impact_test.go` (TestScoreImpact, TestBranchParentParsing — ZFS and btrfs output, TestFormatImpact). integration-v2 §8: a table's views (depth 1 and 2, via), the foreign key from another table and its index; a column's dependents only; quoted names; non-destructive changes; a missing object; a statement without a target asks for `--object` ; a view's dependents; the depth limit; the object on a branch cloned after it existed (with its parent) and not on one without it; the policy verdict in the score; REST and MCP agree with the CLI; `execute_change` includes the impact.
STATUS & LIMITS	Verified in the VM (integration-v2 §8, 15 Sep 2026; existing suite 28 / 28). The target is read from the statement text: one object per statement (the first of a multi-object DROP), and statements on functions, types, schemas or whole-schema grants aren't recognised — name the object with `--object` . Dependencies PostgreSQL doesn't record aren't seen: application queries, functions with plain-string bodies (only SQL-standard `BEGIN ATOMIC` bodies are tracked), dynamic SQL, other databases. Row counts are the planner's estimate. The score is a guide, not a guarantee. The policy error DETAIL's `impact` field stays null (the gate doesn't run this inside the database); use `fox impact` or `execute_change` . No web view yet.
ADDED IN	Blackbox 2.0 Phase 7 (`feat/blackbox-phase7-impact-diff`).

H18 Blackbox diff, and promoting a branch's changes

Working

WHAT IT DOES

Shows which schema changes distinguish two branches: where they split, what was changed only on each since, and which objects were changed on both (likely conflicts). Diff only — nothing is merged or changed.

HOW IT WORKS

`branch.DiffLedgers` (`internal/branch/diff.go`) reads both branches' Blackbox histories in id order (starting a suspended branch first, as other Blackbox commands do). A branch is a copy-on-write clone, so it begins with its parent's history entry for entry; the fork point is the longest run of entries with the same id and the same recomputed hash. Everything after it is listed for each side (id, time, command, object, status, actor). Objects changed on both sides are listed with their entry ids; BLOCKED entries don't count, since those changes never happened. Each branch's parent is read from the storage layer.

HOW TO USE

`fox blackbox diff main qa` · `fox branch diff main qa --json` · REST `GET /api/blackbox/diff?a=main&b=qa` (also `/api/ledger/diff`) · MCP `blackbox_diff`

HOW TO TEST

Unit: `internal/branch/impact_test.go` TestBuildDiff (fork point, same id with different content, both-changed objects, BLOCKED ignored, self diff, no shared history, text output). integration-v2 §8: a branch cloned from main, changes on each side — the parent, shared history, changes only on main, only on the branch, the object both changed; `fox branch diff` matches; text output; a branch against itself; REST on both paths; unknown branch 404; MCP.

STATUS & LIMITS

Verified in the VM (integration-v2 §8, 15 Sep 2026; existing suite 28 / 28). It compares recorded schema changes, not the schemas themselves: changes made while Blackbox wasn't recording (before it was installed, or with the capture turned off by a superuser) don't appear, and data changes are out of scope. Branches that weren't cloned from each other (e.g. two imports) have no shared history, so everything shows on both sides. Rewritten history on one branch moves the fork point earlier — check `fox blackbox integrity` . Reads both histories in full.

ADDED IN

Blackbox 2.0 Phase 7 (`feat/blackbox-phase7-impact-diff`).

PROMOTION — WHAT IT DOES

Brings a branch's schema changes back. Branching was half a workflow: you could copy the database, change it and prove what changed, and then there was no way to apply it anywhere. `fox branch request <source> [--to main]` records the statements the source's Blackbox holds that the target's does not, and someone who can manage the target approves or rejects. Schema only — no data is moved.

PROMOTION — HOW IT WORKS

`BuildRequest` (`internal/branch/promote.go`) takes the fork point from the diff above, collects every entry after it in the order it ran, and leaves out entries the guardrail refused (a blocked change never happened) and any with no statement recorded. It refuses a request outright when both branches changed the same object since they split, or when the target's own policy gate would block a statement — checked against the target, because a rule an admin turned on for main is exactly what a merge must not walk around. The statements are snapshotted into the request, so a reviewer decides on what they were shown rather than on whatever the branch looks like later. `ApplyRequest` runs them in one transaction with `SET LOCAL bb.actor` per statement, so the target's Blackbox names whoever wrote each one, with the approver in the entry's tool and `request-<id>` in its session. One transaction means the target is never half-merged. The record lives in the account store (`change_requests`), and a decision is claimed with a conditional update so two approvals cannot apply the same statements twice.

PROMOTION — HOW TO USE

`fox branch request <source> [--to <target>]` · `fox request list [--open] [--target b]` · `fox request show|approve|reject <id> [--note "…"]` · REST `POST /api/branches/{name}/request` (409 conflict, 422 blocked by policy, 400 nothing to promote), `GET /api/requests[/{id}]` , `POST /api/requests/{id}/approve|reject` · MCP `request_changes` and `list_change_requests` — deliberately no tool that decides one · the web **Requests** page.

PROMOTION — STATUS & LIMITS

Working. Schema only. What is applied is what ran on the source, so a column added and then dropped is applied and then dropped again rather than being reduced to nothing — the target's record ends up holding the same statements the source's does, which is the property worth keeping. A statement that cannot run inside a transaction (`CREATE INDEX CONCURRENTLY`) fails the whole request. Approving needs the right to manage the target (`internal/access`): an admin for main, the owner otherwise — so an agent can propose a change and cannot approve its own, which is the point of the review step.

AREA I

Import & migration

Every import lands in a **new branch** (a copy-on-write clone of `main` whose `public` schema is wiped first), so a failed import never touches `main`. An import with 0 tables fails loudly. Code: `internal/branch/import.go`. Only the MongoDB path has end-to-end automated coverage (integration §9–10).

I1 From Postgres

Working

WHAT IT DOES	Copies a Postgres (or compatible) database, schema and data, into a new branch.
HOW IT WORKS	`loadPostgres`: `pg_dump <dsn> --no-owner --no-acl --no-comments | psql` inside the target container. psql's errors are counted: if any statement failed, the import fails listing them (see I5).
HOW TO USE	`fox import --from postgres://u:pw@host:5432/db [--as name]`; REST `POST /api/import`; web Import.
HOW TO TEST	Manual: start `postgres:18` (or any older major) on network `dbnet` with a 2-row table, import, `SELECT count(*)` = 2.
STATUS & LIMITS	Working; no automated test of the Postgres source itself (the error handling is shared with I5 and tested there); depends on pg_dump version compatibility — a statement the target can't run now fails the import instead of being skipped silently. `pg_dump` runs in the engine image, and an older `pg_dump` cannot dump a newer server, so the newest source this accepts is the engine's own major: moving a fresh install to PostgreSQL 18 (21 Sep 2026) raised that from 16 to 18. (The test note named the network `foxbyte`; it has been `dbnet` since the names were frozen.) **The claim was narrowed to what is true** (5 Oct 2026, from an outside review): the console's Import page offered “Postgres/Postgres-wire (RDS, Neon, Supabase, CockroachDB) with full fidelity”. RDS, Neon and Supabase are PostgreSQL, so `pg_dump` into a fresh instance is exactly that. CockroachDB is not PostgreSQL — it speaks the wire protocol — and `pg_dump` support there varies, so promising fidelity for it was a claim nothing stood behind. Worth being blunt about the evidence: the suites exercise the import *refusals* (admin-only, SSRF and metadata addresses) and no source's fidelity at all, so every word of that sentence rested on reasoning rather than a test. The page now names the three that are PostgreSQL and says plainly that wire-compatible engines are untested.
ADDED IN	`44cc05c`.

I2 From MySQL ≤5.7 / MariaDB

Working

WHAT IT DOES	Migrates MySQL/MariaDB with type mapping.
HOW IT WORKS	`loadMySQL` builds a local pgloader image on first use and runs `pgloader <dsn> postgresql://…@pg-<target>`; success judged by table count.
HOW TO USE	`fox import --from mysql://u:pw@host:3306/db --as legacy` (or `mariadb://`)
HOW TO TEST	Manual: `mariadb:11` container with a table, import, query rows.
STATUS & LIMITS	Working; no automated test.
ADDED IN	`b842a28`; hardening `c929eb5`.

I3 From MySQL 8.x

Working

WHAT IT DOES	Imports MySQL 8+ (whose auth pgloader can't handle).
HOW IT WORKS	`mysqlNeedsNative` detects ≥8; `loadMySQLNative` generates CREATE TABLE from `information_schema` (type map: json→jsonb, decimal kept, else text) and streams `mysqldump --no-create-info` INSERTs; per-table progress.
HOW TO USE	`fox import --from mysql://root:pw@host:3306/db --as m8`
HOW TO TEST	Manual: `mysql:8` with json + decimal columns; check `information_schema.columns`.
STATUS & LIMITS	Working; no automated test. Only columns, NOT NULL and PK are carried over (no other indexes, FKs or defaults).
ADDED IN	`c929eb5`.

`I4` From MongoDB

Working

WHAT IT DOES	Turns each collection into a typed Postgres table with the exact collection name.
HOW IT WORKS	`loadMongo` : list collections with `mongosh` , export each as NDJSON (ObjectId → hex, Date → ISO), then load via the JSON path (I6).
HOW TO USE	`fox import --from mongodb://host/db --as events`
HOW TO TEST	Integration §9, §10. Manual: `mongo:7` with a nested document; `\d` shows jsonb for objects, numeric for numbers.
STATUS & LIMITS	Working (the only import path with end-to-end tests).
ADDED IN	`b842a28` ; relational tables `f2c964e` .

`I5` From a .sql dump

Working

WHAT IT DOES	Runs a SQL dump into a new branch.
HOW IT WORKS	`loadSQL` pipes the file into `psql` in the container; branch named `import-<file>` . The whole dump runs; psql's `ERROR` / `FATAL` lines are collected (`psqlErrorCollector`) and, if there were any, the import fails with the count and the first 20 — the new instance is kept with everything that did load.
HOW TO USE	`fox import --from ./dump.sql` ; `POST /api/import/file` ; web Import.
HOW TO TEST	Unit: `TestPsqlErrorCollector` . Integration §11: a dump with one failing statement exits 1, lists it, and the rest loaded; a clean dump exits 0.
STATUS & LIMITS	Working. Any failed statement fails the import, including harmless ones (e.g. a setting only a newer Postgres knows) — they are listed so you can decide. Until 15 Sep 2026 failing statements were ignored and the import reported success.
ADDED IN	`44cc05c` .

`I6` From JSON / NDJSON

Working

WHAT IT DOES	Loads a JSON array or NDJSON into a table with columns inferred from top-level keys.
HOW IT WORKS	`loadJSON` stages documents as jsonb, then `relationalizeJSON` picks a type per key (`_id` text; string→text; number→numeric; boolean; nested/mixed→jsonb), column order `_id` then alphabetical. `streamJSONDocs` reads the array or lines; an invalid document stops the load, rolls back everything from that file, and fails the import naming the array element or line number.
HOW TO USE	`fox import --from ./people.ndjson`
HOW TO TEST	Unit: TestJSONColumn, TestSQLIdent, TestStreamJSONDocs. Integration §10; §11 (an array with an invalid 2nd element exits 1 naming it; a valid array imports).
STATUS & LIMITS	Working. Until 15 Sep 2026 a JSON array import stopped silently at the first element it couldn't decode and reported success.
ADDED IN	`44cc05c` ; relationalization `f2c964e` .

`I7` From CSV

Working

WHAT IT DOES	Loads a CSV into a table (one text column per header).
HOW IT WORKS	`loadCSV` : header → `CREATE TABLE` , rows → `\copy … FORMAT csv` .
HOW TO USE	`fox import --from ./people.csv`
HOW TO TEST	Manual: `printf 'id,Name\n1,a\n' > p.csv` , import, `SELECT "Name" FROM p` .
STATUS & LIMITS	All columns are text; a header containing a newline won't parse.
ADDED IN	`44cc05c` .

I8 Continuous replication & cutover

Working

WHAT IT DOES	Copies a Postgres source's schema and keeps streaming changes for a zero-downtime switch; cutover stops replication.
HOW IT WORKS	`ImportContinuousTo` : schema-only dump, `CREATE PUBLICATION bb_pub` on source (best effort), `CREATE SUBSCRIPTION bb_sub` on target. `ImportCutover` / `ImportCutoverTo` drop the subscription. `replication.go` reads the progress back — `pg_subscription` , `pg_subscription_rel` (`srsubstate='r'` = initial copy done) and `pg_stat_subscription` — as `ReplicationStatus` for one branch and `Replicating` for all running ones; `CutoverReplication` refuses a branch with no subscription (`ErrNotReplicating`) so an API cutover can't silently succeed against the wrong branch. The Gateway's reaper leaves a replicating branch running (`canSuspend`).
HOW TO USE	CLI: `fox import --from postgres://… --continuous --as live` then `fox import-cutover live` . Web: tick **Continuous replication** on the Import page, then watch the **Continuous imports** panel (tables copied, last message from the source) and press **Cut over**. REST: `GET /api/replication` , `GET /api/branches/{name}/replication` , `POST …/replication/cutover` .
HOW TO TEST	Integration §5b: a real source with `wal_level=logical` , initial copy 1 of 1 tables, a source insert streaming across, the branch surviving a short-idle reaper while a plain branch is suspended, cutover returning `standalone` , later source changes not arriving, and a second cutover (and a cutover of a branch that never replicated) refused 409. Unit: `TestParseReplication` , `TestCanSuspend` .
STATUS & LIMITS	Working, and verified in the browser end to end (import, panel, streamed row, cutover). Publication and subscription names are fixed, so one source per branch. Progress is what Postgres reports: table counts and the last message time, not a byte-level lag.
ADDED IN	`b842a28` ; status, cutover API and panel `feat/blackbox-page-gaps` .

I9 Guard handling during import

Working

WHAT IT DOES	Lets bulk loads run DDL the guardrail would block, while the ledger keeps recording — and can't leave a branch unguarded after a crash.
HOW IT WORKS	`setGuard` disables/enables `bb_guard_start` around the load (deferred); loaders set `bb.allow_destructive=on` ; `reconcileGuard` re-enables the guard whenever `EnsureRunning` restarts a branch.
AND THE TARGET'S GRANTS	The target is prepared with `DROP SCHEMA IF EXISTS public CASCADE; CREATE SCHEMA public` , so the instance holds the source's schema rather than a copy of `main` 's — and that took `public` 's grants with it. The gateway logs every client in as a role inheriting `db_client` , so **imported data could be read by the superuser the import itself runs as and by no client at all**: `db_client` had lost not just the table grants but `USAGE` on the schema. `prepareTarget` re-applies the three statements `ledger.sql` makes (schema `USAGE, CREATE` , and the two `ALTER DEFAULT PRIVILEGES` that cover tables the superuser creates later) — whose own comment says they are meant to cover imported tables. Found by a realtime check: a feed on an imported table was refused because `db_client` could not `SELECT` it, which was the preflight telling the truth about something else. Every assertion about importing had used the superuser, which is how it survived this long.
HOW TO USE	Automatic.
HOW TO TEST	`scripts/integration_test.sh` §5b: a **gateway client** (not the superuser) reads the imported table, and `scripts/integration_realtime.sh` §11 asserts the schema and table privileges directly. Manual: stop the engine mid-import, `fox branch resume <target>` , `SELECT evtenabled FROM pg_event_trigger WHERE evtname='bb_guard_start'` → `0` .
STATUS & LIMITS	Working.
ADDED IN	`b842a28` ; crash-safe `1884445` (PR #12).

`I10` Live progress (CLI & SSE)

Working

WHAT IT DOES	Shows a live migration log and, where the source supports it, a progress bar.
HOW IT WORKS	`Progress{Log, Step}`; the control plane streams Server-Sent Events `log`, `progress`, `done`, `error`. Step events from MySQL 8 (per table), MongoDB (per collection), pipeline models.
HOW TO USE	`curl -N -k -H "Authorization: Bearer $KEY" -H 'Content-Type: application/json' -d '{"source":"mongodb://mongo-src/shop"}' https://localhost:8080/api/import`
HOW TO TEST	Manual curl above; watch `event: log` / `event: progress`.
STATUS & LIMITS	psql subprocess output goes to engine logs, not SSE.
ADDED IN	`f2c964e`.

`I11` Import REST endpoints

Working

WHAT IT DOES	API entry points for imports (auth required).
HOW IT WORKS	`POST /api/import` body `{source, target, continuous}` — connection strings only (postgres, mysql, mariadb, mongodb). `POST /api/import/file` multipart `file` + `target`, kind from extension. Both stream SSE. Anyone may import from a database with a public address; loopback, private networks and names only the engine resolves (a container) are for admins; link-local and cloud-metadata addresses are refused for everyone (`checkImportSource`, resolved when the import starts). The branch an import makes belongs to the importer.
HOW TO USE	`curl -N -k -H "Authorization: Bearer $KEY" -F file=@people.csv -F target=ppl https://localhost:8080/api/import/file`
HOW TO TEST	Manual curl; without a key → 401. Unit: TestImportHosts, TestImportSourceRule. Integration §1d (an account cannot import from this machine or a container name; nobody from a metadata address).
STATUS & LIMITS	Working. Target name not validated against the branch-name rule; OpenAPI documents the wrong field names (K2). Until 22 Sep 2026 any account could make the engine connect anywhere its network reached (audit v2 G12). A name that resolves differently after the check is not caught.
ADDED IN	`44cc05c`, `b842a28`.

`I12` Web Import page

Working

WHAT IT DOES	"Migrate a database" form with a live migration console.
HOW IT WORKS	`web/src/pages/Import.tsx`: connection-string panel (continuous checkbox for postgres), file upload (.sql/.csv/.json/.ndjson/.jsonl), SSE console with progress, result with links to Console and Ledger.
HOW TO USE	https://localhost:8080/import
HOW TO TEST	Manual: upload a CSV and check the result panel.
STATUS & LIMITS	Working. Fixed 21 Sep 2026: the footer said SQLite files could be imported with the CLI, but no SQLite loader exists; the claim is gone.
ADDED IN	`44cc05c`; console `f2c964e` (PR #3).

`I13` Exact identifier fidelity

Working

WHAT IT DOES	Source names like `leaseAiChats` or `userId` keep their exact case.
HOW IT WORKS	`sqlIdent` quotes and truncates to 63 bytes (tables, columns); `sanitizeIdent` only for branch names.
HOW TO USE	Automatic; quote camelCase in SQL: `SELECT "userId" …`
HOW TO TEST	Unit: TestSQLIdent. Integration §10.
STATUS & LIMITS	Working.
ADDED IN	`f2c964e`.

AREA J

ETL pipelines (experimental)

J1 Pipeline engine

Working

WHAT IT DOES	dbt-style extract → land raw → transform with SQL models → quality tests, into a fresh branch.
HOW IT WORKS	`RunPipeline` (`pipeline.go`): delete + re-create the target branch, import the source, rename `public` → `raw` , run each model in order as `CREATE TABLE|VIEW public."name" AS …` with `{{ ref('m') }}` / `{{ source('t') }}` templating, then tests.
HOW TO USE	`fox pipeline run spec.json [--as name]` (default `pl-<file>` ; exit 1 on test failure).
HOW TO TEST	Unit: TestRenderSQL, TestTestQuery. Integration §9.
STATUS & LIMITS	No dependency graph (listed order), no incremental models, no scheduling. A run replaces its target branch — but only one this pipeline made: the control plane records which pipeline made which branch (`pipeline_targets`) and refuses to run into any other existing branch. Until 22 Sep 2026 naming a pipeline after someone else's branch and running it deleted that branch, and a source could be a path, read on the server with the engine's permissions (secrets file included) and loaded into a queryable branch; through the API a source must now be a connection string (`checkPipelineSource` , also applied to specs saved before). Unit: TestPipelineTargets, TestPipelineSourceMustBeAConnectionString; integration §1c.
ADDED IN	`f2c964e` .

J2 Data-quality tests

Working

WHAT IT DOES	Assertions after transform: `not_null` , `unique` , `accepted_values` , `row_count_min` , `custom` .
HOW IT WORKS	`testQuery` builds a "count violating rows" query; a test passes when the count is 0. Failures mark the run failed but keep the data.
HOW TO USE	`"tests": [{"type":"unique","model":"stg","column":"city"}]`
HOW TO TEST	Unit: TestTestQuery. Integration §9 (failing `row_count_min` → exit 1, data kept).
STATUS & LIMITS	Working.
ADDED IN	`f2c964e` .

J3 Stored pipelines & run history

Working

WHAT IT DOES	Save pipelines per user, run them over the API, keep run history.
HOW IT WORKS	SQLite tables `pipelines` , `pipeline_runs` ; routes `GET/POST /api/pipelines` , `GET/PUT/DELETE /api/pipelines/{id}` , `GET …/runs` (last 50), `POST …/run` (SSE).
HOW TO USE	`curl -k -H "Authorization: Bearer $KEY" -d '{"name":"shop etl","spec":{…}}' https://localhost:8080/api/pipelines`
HOW TO TEST	Manual: create, run, `GET …/runs` shows `success` .
STATUS & LIMITS	No endpoint to read a run's log; a run whose process dies stays `running` .
ADDED IN	`f2c964e` .

J4 Pipeline editor (web)

Working

WHAT IT DOES	Create, edit, run and inspect pipelines in the browser.
HOW IT WORKS	`Pipelines.tsx` (list) and `PipelineEditor.tsx` (source, model cards, test cards, Load example, Save, Run with streaming console, run history).
HOW TO USE	https://localhost:8080/pipelines → New pipeline → Load example → Save → Run.
HOW TO TEST	Manual with the integration §9 Mongo data.
STATUS & LIMITS	Delete on the list has no confirmation.
ADDED IN	`f2c964e` .

AREA K

APIs, SDKs & agents

K1 Control-plane REST API

Working

WHAT IT DOES JSON management API over HTTPS on `:8080` (also serves the web UI).

HOW IT WORKS `internal/controlplane/server.go`; `store.Authn` on `/api/*`; CORS for the configured UI origin (and the same scheme and port on localhost/127.0.0.1/[::1]); any other local port only with `FOX_DEV_CORS=1`, for the UI dev server. Branch names in `/api/branches/{name}` are checked before any handler (400), request bodies capped at 8 MiB and file uploads at `FOX_MAX_UPLOAD_MB` (4096); header timeout 10 s.

Routes	Purpose
`POST /auth/register|login|logout`, `GET /auth/me|providers`, `GET /auth/oauth/{p}[/callback]`	Accounts (public)
`GET /api/openapi.yaml`	Spec (public)
`GET /api/status`	Health, branches, agents, HA, storage
`GET|POST /api/branches`, `DELETE /api/branches/{n}`, `POST …/suspend|resume`	Branches
`POST /api/branches/{n}/query`	SQL as `db_client`, 10s, ≤1000 rows, attributed to the user
`GET /api/branches/{n}/ledger`, `…/ledger/verify`	Blackbox
`GET …/ledger/integrity`, `POST …/ledger/checkpoint`, `GET …/ledger/export`	Ledger integrity (H10, H12)
`GET /api/branches/{n}/ledger/entries`, `POST …/ledger/{id}/branch`	Entries with ids; branch from before a ledger entry (H13)
`GET /api/branches/{n}/ledger/sessions`	Agent sessions (H16)
`POST /api/branches/{n}/impact`, `GET /api/ledger/diff`, `GET /api/blackbox/diff`	Impact analysis and diff (H17, H18)
`GET|POST /api/branches/{n}/policies`, `PUT|DELETE …/policies/{rule}`, `POST …/policies/check`, `GET …/policies/evaluations`	Policy gate; changes need `db_admin` (H15)
`/api/branches/{n}/blackbox[/…]`	Every `…/ledger[/…]` route under its Blackbox name (H14)
`POST /api/import`, `/api/import/file`	Import (SSE)
`/api/pipelines…`	Pipelines
`GET|POST /api/keys`, `DELETE /api/keys/{id}`	API keys

HOW TO USE `curl -k -H "Authorization: Bearer $KEY" https://localhost:8080/api/status`

HOW TO TEST Unit: TestNameValidation. Integration §1; integration-v2 §0 (ledger/verify columns).

STATUS & LIMITS Working. Not versioned (`/api/`, not `/api/v1/`). **Ownership (22 Sep 2026, audit v2 G02):** every branch made through the API records its maker (`branch_owners`). `internal/access` is the rule every surface applies: an admin (superuser or `db_admin` on main, remembered 30 s) may do anything; the owner may use and manage their branch; every account may use main; anything else answers 404 and is left out of lists. Deleting, cutting an import over and checkpointing need owner or admin; the rest need use. A branch with no owner (made at the CLI, before owners were recorded, or whose owner was deleted) is an admin's. Unit: TestLevels, TestAuthorizeBranchRoutes; integration §1d. An admin hands a CLI-made branch to an account with `fox branch owner <name> <email>` (without an email it shows the owner). **Versioned paths** (audit v2 G35): `/api/v1/…` reaches the same handlers, rewritten to `/api/…` before authorization or the Blackbox alias see the path (`internal/controlplane/apiversion.go`), so there is one implementation of each endpoint and no chance of two drifting. `/api/…` keeps working — it is what the SDKs, the MCP server and other people's code are written against. `/api/v1` with nothing after it redirects to the status endpoint. A v2 would

	need routes of its own; until then "v1" is a promise about what these paths mean rather than a second copy of them.
ADDED IN	`a7580f6` ; TLS `0bd52f3` .

K2 OpenAPI spec

Working

WHAT IT DOES	Machine-readable API description for client generators.
HOW IT WORKS	`openapi.yaml` (OpenAPI 3.1) embedded and served at `/api/openapi.yaml` ; also documents agent routes.
HOW TO USE	`curl -k https://localhost:8080/api/openapi.yaml`
HOW TO TEST	`go test ./internal/controlplane` : `TestOpenAPIMatchesRoutes` compares the spec's paths with the routes the code registers (it reads the `HandleFunc` literals, because `net/http.ServeMux` cannot list what was registered), and `TestOpenAPIAliasRefsResolve` checks every alias pointer. Manual: `curl -k https://localhost:8080/api/openapi.yaml` .
STATUS & LIMITS	Matches the code: `/api/import` takes `{source, target, continuous}` and answers with an SSE stream, `/api/import/file` documents its multipart fields, branch and API-key creation answer 201, the ledger's `since` / `until` filters are described, and `/api/openapi.yaml` plus the seven `/auth/*` routes are documented. The `/blackbox` aliases are declared as path-item `$ref` s, so a generated client gets both spellings with no second copy to keep in step. Adding a route without documenting it now fails the build. Not checked automatically: whether a described body or response still matches its handler.
ADDED IN	`52b1fc5` / `cc5cbd7` (PR #15); corrected on `docs/api-accuracy` .

K3 Python SDK `foxbyte`

Working

WHAT IT DOES	Dependency-free client for the control plane (Apache-2.0).
HOW IT WORKS	`clients/python/foxbyte` : `FoxByte(api_key, base_url, verify_tls)` ; `status` , `branches` , `create_branch` , `delete_branch` , `suspend` , `resume` , `query` , `ledger` , `verify_ledger` . Blackbox names `blackbox()` and `verify_blackbox()` call the same routes as `ledger()` and `verify_ledger()` (H14).
HOW TO USE	`pip install ./clients/python` ; `db = FoxByte(api_key="key_…", verify_tls=False); db.query("qa","select 1")`
HOW TO TEST	`make integration-sdks` (scripts/integration_sdks.sh) drives the published client against a live engine: status, create, list, SQL, the Blackbox's entries and verify, suspend, resume, the error a call on a missing branch raises, and delete. 10 checks.
STATUS & LIMITS	Not on PyPI; no import/pipeline/key/agent methods. Tested against a live engine since 23 Sep 2026 (audit v2 G26); no packaging to PyPI yet.
ADDED IN	`52b1fc5` .

K4 TypeScript SDK `@foxbyte/client`

Working

WHAT IT DOES	The same client for Node and browsers (Apache-2.0).
HOW IT WORKS	`clients/typescript/src/index.ts` : `new FoxByte(apiKey, baseUrl)` ; same methods (`createBranch` , `verifyLedger` , …) using `fetch` . Blackbox names `blackbox()` and `verifyBlackbox()` call the same routes as `ledger()` and `verifyLedger()` (H14).
HOW TO USE	`cd clients/typescript && npm install && npm run build` ; local self-signed cert needs `NODE_TLS_REJECT_UNAUTHORIZED=0` .
HOW TO TEST	`make integration-sdks` : it compiles with its own tsconfig, then the compiled client is driven against a live engine — status, create, list, SQL, the Blackbox's entries and verify, suspend, resume, the error a call on a missing branch throws, and delete. 11 checks.
STATUS & LIMITS	Not published to npm. Tested against a live engine since 23 Sep 2026 (audit v2 G26); no packaging to npm yet.
ADDED IN	`52b1fc5` .

K5 Agent Branch API

Working

WHAT IT DOES	A disposable database per AI agent over HTTPS on `:8088`.
HOW IT WORKS	`internal/agentapi`: `GET /healthz` (public), `GET /agents`, `POST /agents/{id}/branch` (creates `agent-<id>`, returns DSN), `DELETE /agents/{id}/branch`. Cap `FOX_AGENT_MAX` (50); TTL reaping with `FOX_AGENT_TTL`. Optional body `{"task_id", "parent_session_id", "session_id"}` records Blackbox provenance for the agent's changes and adds `session_id` to the response; without it nothing changes (H16).
HOW TO USE	`curl -k -H "Authorization: Bearer $KEY" -X POST https://localhost:8088/agents/alice/branch`
HOW TO TEST	Integration §5 (usable via its DSN; the key dies with the branch); integration-v2 §1c (own non-superuser role, can't disable triggers or override, gateway-routed DSN, key refused for another branch and by both HTTP APIs). Unit: `TestAgentRolePassword`, `TestAgentGatewayDSN`, `TestGatewayHostPort`, `TestScopeAllows`, `TestScopedKeys`, `TestScopeColumnAddedToExistingStore`.
STATUS & LIMITS	The DSN is routed through the Gateway, so an agent running on the host can open it — the branch's Postgres still publishes no host port, because a direct connection would bypass the Gateway, its key, TLS and the attribution. Its password is an API key scoped to that one branch (`api_keys.scope`): the control plane and the Agent API refuse it, and the Gateway opens only the named branch with it, so a leaked agent credential reaches neither another agent's data nor the account's branches. The Gateway logs the agent in as the branch's own `agent-<id>` role, whose password is derived from the install secret rather than stored, so `session_user` stays the agent and attribution remains non-forgeable. The key is shown once and revoked when the branch is deleted (a reset keeps it). The cap, the listing and the TTL reaper now count suspended branches too. `FOX_AGENT_SUPERUSER=1` still restores the old container-IP superuser DSN. Agent branches created before this change have no agent role (recreate them), and `FOX_GATEWAY_HOSTPORT` is needed if the Gateway is not on `localhost:6432`. `FOX_AGENT_SUPERUSER` is honoured only by a build made with `-tags insecure` since 22 Sep 2026 (audit v2 G05): in a release build no environment variable brings back the DSN carrying the master password. Agent ids that would not make a valid branch name are refused (400). An agent's branch and its key belong to the account that asked for it (`CreateAgentBranchFor`); until 22 Sep 2026 the key went to the install's first account. `GET /agents` lists the caller's agents (an admin's: all).
ADDED IN	`a7580f6`; cap/TTL `e720c4e` (PR #12); reachability on `fix/agent-reachability`.

K6 MCP server

Working

WHAT IT DOES	Lets agent frameworks manage agent branches, run SQL and read/verify the ledger via the Model Context Protocol.
HOW IT WORKS	`internal/mcp` : JSON-RPC 2.0 over stdio (`initialize` , `tools/list` , `tools/call`). Tools: `create_branch` , `list_branches` , `delete_branch` , `run_sql` , `changes` , `verify_ledger` , `ledger_integrity` , `ledger_entries` , `branch_before_change` (H13). Blackbox names `verify_blackbox` , `blackbox_integrity` and `blackbox_entries` are listed too and run the same tools (H14). `policy_check` previews policy rules for a statement (H15). `execute_change` runs SQL with agent provenance and a policy preview and returns structured JSON (H16). `impact` and `blackbox_diff` expose impact analysis and branch diff (H17, H18).
HOW TO USE	Configure an MCP client to launch command `fox` with args `["mcp"]` and an `env` block carrying `FOX_API_KEY` (`--key <key_…>` also works, but a key on the command line is visible in the process list). Make one with `fox apikey create you@example.com mcp` .
HOW TO TEST	Unit: `TestAuthenticateWithoutAKey` , `TestApplyScope` . integration-v2 §1c (run_sql role, list_branches without passwords, compat switch; no key exits 1 saying how to make one; an invalid key is refused; run_sql records the key's account as the actor; a scoped key's call lands in its own branch and not main, another branch is refused, and the tools that reach further are refused), §3 (`ledger_integrity`), §4 (`ledger_entries` lists the entry id; `branch_before_change` listed, refuses a BLOCKED entry, restores by timestamp). Manual: pipe `initialize` , `tools/list` , `tools/call run_sql` JSON lines into `fox mcp` .
STATUS & LIMITS	The server requires an API key and acts as that account: `run_sql` runs as the non-superuser `db_client` role but is recorded with the key's account as the actor (`ClientQueryTextAs`), and a key scoped to one branch pins every call to it, refuses another branch by name, and refuses `create_branch` , `delete_branch` , `list_branches` , `blackbox_diff` and `branch_before_change` outright. There is no opt-out: without a valid key the server prints how to make one and exits. `FOX_MCP_SUPERUSER=1` — honoured only by a `-tags insecure` build since 22 Sep 2026 — still restores the superuser role for compatibility. Beyond the key there is no per-call authorization or rate limiting, and anyone who can read the client config can read the key — it is still meant for a local agent runtime. `list_branches` no longer shows passwords. Client setup and the full 16-tool reference are in `docs/mcp.md` , linked from the README and the Docs page. `create_branch` returns the same Gateway-routed, branch-scoped DSN as the Agent API (K5), so an agent driven over MCP can reach its database from the host. Tool calls follow the same ownership rule as the REST API (TestMCPAppliesTheAccessRule).
ADDED IN	`d62c676` / `ed5a379` (PR #14); authentication `feat/attribution-mcp-auth` .

AREA L

Web console

L1 Embedded single-origin UI

Working

WHAT IT DOES	The engine serves the React app itself at https://localhost:8080 — no separate web server.
HOW IT WORKS	`web/embed.go` (`//go:build embedui`) embeds `web/dist` ; `serveUI` falls back to `index.html` . `make web-build` builds with same-origin API URLs; Linux release binaries use `-tags embedui` . `make web-dev` is deprecated (hot-reload only).
HOW TO USE	`fox start` → open https://localhost:8080 (accept the self-signed cert).
HOW TO TEST	Manual: open `/dashboard` directly and refresh.
STATUS & LIMITS	Working. `make web-dev` defaults to `http://localhost:8080` and needs `VITE_API_URL=https://localhost:8080` . Checked against a published release on 24 Sep 2026: the released Linux engine carries the console, and its smoke tests pass against it. **The branch list every selector is built from is loaded through one hook** (`web/src/useBranches.ts` , used by the Blackbox, Integrity, Policies and Requests pages), which retries a dropped request and says so when it finally cannot load, instead of the `getBranches().then(setBranches).catch(() => {})` each page used to carry: one lost request left the page with empty selectors, buttons that did nothing and no explanation, recoverable only by reloading. It is lost requests rather than a rare one that made this matter — the engine runs short-lived containers on a Docker bridge (the backup health check behind `/api/status` is one), and Chromium aborts everything in flight when the host's network interfaces change, which is how the console's own promotion test failed in CI on 29 Sep 2026 while the engine logged the same request as a success.
ADDED IN	`4927095` ; React app `84741df` .

L2 Login & signup

Working

WHAT IT DOES	Log in / sign up tabs (sign-up only when open), OAuth buttons when configured, redirect to Dashboard. On a fresh install it shows only "Create the first account", with a setup-token field.
HOW IT WORKS	`/login` ; `RequireAuth` guards private routes; API client redirects to /login on 401.
HOW TO USE	https://localhost:8080/login
HOW TO TEST	Manual sign-up and logout.
STATUS & LIMITS	Working (OAuth caveat G5). Type-checked in CI since 22 Sep 2026.
ADDED IN	`235ff41` .

L3 Dashboard

Working

WHAT IT DOES	Live tiles (primary, branches, agent DBs, gateway, replica, storage), a branch table with create (from any parent), suspend, resume, delete and a copyable connection string, and a **Base backups** panel: how far back this install can go, with a copy-ready restore command per backup.
HOW IT WORKS	Polls `/api/status` and `/api/branches` every 3s; offline banner when the API is unreachable. The backups panel reads `/api/backups` once when the page opens (backups change when someone takes one, not every three seconds) and shows the newest 6, with the rest behind "Show N older backups".
HOW TO USE	https://localhost:8080/dashboard
HOW TO TEST	Manual: create a branch and see it appear; read the backups panel against `fox backup list` . Checked in the browser against a real install's 61 backups.
STATUS & LIMITS	Working. The panel is read-only: taking a backup and restoring stay CLI commands (a restore needs a host port and leaves a container to query).
ADDED IN	`84741df` ; backups panel `feat/durability-gaps` .

L4 SQL console & browser

Working

WHAT IT DOES	Schema sidebar, row browser (100/page), structure and index tabs, SQL editor (Cmd/Ctrl+Enter), row-as-JSON. FoxByte's own tables (the `bb` schema: Blackbox, policies, agent sessions) are hidden whenever the console opens; a **Show system tables** button at the end of the list shows or hides them, and the filter never reveals them on its own. An **Allow destructive changes** switch lets an admin run one blocked change.
HOW IT WORKS	`Console.tsx` → `POST /api/branches/{name}/query`, which logs in as the signed-in user's own role (a member of `db_client` acting as it, as the Gateway does), so the guardrail sees who is asking. `allow_destructive` applies `bb.allow_destructive=on` to that run's session only, and the switch turns itself off afterwards. `GET /api/branches/{name}/admins` tells the page whether the user may override. Reload refreshes the list and the open table and shows errors instead of an empty schema; the list also refreshes after DDL. Each value is rendered by `cell`: scalars, timestamps, jsonb and arrays of scalars as they were, and for the types pgx decodes into a byte array or a struct — uuid, bytea, interval, time of day, inet, macaddr, the geometric types, ranges, hstore — Postgres's own text form, asked of pgx's type map (`Map.Encode` with `TextFormatCode`), which is what psql would print. Array members are rendered by their own element type, so a `uuid[]` is a list of uuids.
LAYOUT	**Rebuilt as a workbench, 3 Oct 2026.** The page fills the window and nothing scrolls but the panes that should. The arithmetic drove it: on a 1440×900 window the console spent 458px of width on navigation before any work — 244 sidebar plus a 214 schema rail — and about 140px of height on a title and a sentence read once, while `.app-content` capped the whole page at 1180px and centred it, so a wide monitor left hundreds of pixels empty on both sides. Three things buy that back, and only the last is cosmetic: the **schema rail folds to a drawer** (its 214px go to both panes at once, and the state is remembered), a script's **results are one tab per statement** rather than a stack, so the open one gets the whole pane and a ten-statement migration is ten tabs instead of ten tables competing, and **either pane expands to the window** with Esc to return. The divider between them is draggable and its position is remembered. A status bar carries branch, rows and command tag. The console route is the only one that escapes the centred column (`.app-content.full-bleed`), because every other page is for reading and the measure limit is right there. Before this: a page header with the branch picker, then three cards — the schema rail, the query card (title bar, editor, toolbar with the destructive switch and **Run query**), and the result (a status bar with the command tag, row and column counts above the grid). The editor follows the theme rather than being dark in both. A browsed table keeps its own header, tabs and a pager aligned to the right of the row count. Replacing the title block took the page's only heading with it, which a screen reader uses to announce where it has landed, so the heading stays — off-screen (`.sr-only`), costing none of the window it was removed to reclaim.
HOW TO USE	https://localhost:8080/console
HOW TO TEST	Unit: `TestCellRendersPostgresTypes` (uuid, uuid[], bytea, interval, time of day, inet, macaddr, and the types that must not change), `TestCellTimestamp`, `TestCellUnknownType`. integration-v2 §6: the value rendering above over the REST API; the console runs as the signed-in user; a non-admin's override is refused; an admin is blocked without the switch and allowed with it. Run live on 17 Sep 2026 against a spare control plane on a throwaway branch: `session_user` was the account, `DROP TABLE` was blocked without the switch and went through with it. Pages type-checked and built; not reviewed in a browser after this change.
STATUS & LIMITS	Working. **A script runs, not only one statement** (2 Oct 2026): pgx speaks the extended protocol, where Postgres allows one command, so anything with a semicolon in it used to come back as `cannot insert multiple commands into a prepared statement` — the one thing people most wanted to paste in. The query falls back to the simple protocol (`pgconn.Exec`) when it sees that refusal, which is safe because the refusal happens while parsing, before anything runs. Each statement gets its own entry in `results`, numbered in the console, and the single-statement fields stay beside it so the SDKs and the browse tabs are untouched. **A script is one transaction** unless it opens its own, so one that fails at the seventh statement leaves nothing behind — which is what a migration wants, and is why the SQL is not split up and run statement by statement: semicolons live inside strings and dollar-quoted function bodies, and cutting on them is wrong in a way that corrupts quietly. Values come back as text on that protocol and are decoded through the same type map, so a uuid or an interval reads the same either way. The guardrails are triggers, so they fire inside a script exactly as they do on a statement sent alone. **Which statement produced which result** is answered in the console: each block carries its position, its command tag and the SQL it came from. The engine cannot supply that text — it sends the script in one message and Postgres returns results in order with a tag and no statement — so the console splits the SQL for display only and *checks its own work*: unless the number of pieces is exactly the number of results it labels nothing and shows positions alone, because a grid under the wrong statement is worse than a grid under none. A semicolon inside a string literal or a dollar-quoted function body makes the counts disagree, which is precisely when it gives up. **The editor is read-only while a query is in flight** — read-only rather than disabled, so the text stays selectable and the caret stays put — which keeps it from drifting out of step with the labelled results beneath it, and fixed a second thing found while doing it: `⌘/Ctrl+↵` had no

busy guard, so it started another query against the same branch while the first was still running, which the Run button had been refusing all along. **Line numbers, and where the error is** (3 Oct 2026): the gutter is a sibling of the textarea drawn with the same line height and scrolled in step, because a textarea cannot hold children. That alignment holds only while the editor does not wrap — a wrapped line is two rows against one number and every number below it would be wrong — so the editor scrolls sideways instead, as a code editor does, and the numbers can be trusted. The more useful half is the location: Postgres reports the character offset of a syntax error (`PgError.Position`) and it was being thrown away, leaving `syntax error at or near "FORM"` and forty lines to search. The offset now reaches the console, which turns it into a line and column, marks that line in the gutter, and puts the caret there. Postgres's own `hint` and `detail` come with it, since a hint like "Perhaps you meant to reference the column t.name" is usually the whole answer. For a script the offset is into the whole script, which is what was sent, so it lands on the right line of a long migration. 10s timeout, 1000-row cap. Each run is its own connection, so a `SET` typed in one run does not reach the next — the switch is the way to override. Only the `bb` schema is treated as system: tables in `public` left by test runs still show as ordinary tables. Fixed 17 Sep 2026 (found testing Blackbox precision in the browser): for a Blackbox policy block (BBX01) the **Allow & run again** button sent `allow_destructive`, which only overrides the guardrail, so an admin got the same error back; the panel now names the rule and its button sends `allow_rules` (`bb.policy_allow` for that rule), and commands with no result rows no longer print their success line twice. Fixed 18 Sep 2026 (reported from the UI): a uuid column — every `id` in the user's own tables — was displayed as a list of 16 numbers, because pgx decodes `uuid` to `[16]byte` and that is what JSON makes of it; the same pass fixed intervals showing as `{"Microseconds":…}`, inets carrying their quotes and bytea arriving as mangled text. Only a real `bytea` column is hex-encoded: pgx also hands back `[]byte` for a type it has no codec for, such as a custom enum, and those stay readable. One statement per run: the endpoint uses the extended protocol, so a pasted script of several statements is refused with `cannot insert multiple commands into a prepared statement`. Found by the SDK contract tests on 23 Sep 2026; accepting several would mean the simple protocol, which changes how every value is rendered, so it is left as it is. **The console's UI tests caught up with the redesign** (3 Oct 2026): the first nightly to cover it failed four tests, all of them looking for a page that no longer exists — the `SQL Console` heading the toolbar replaced, a button labelled **Run query** before it became **Run**, and the first cell of a result row, which has been the JSON expander since the pipeline work rather than a value, so the assertion was reading `{ }` and comparing it to a number. Two were missed renames; the third had been latent in tests written against a table that had already grown a column. The class of mistake is worth naming: Playwright runs only in the nightly, against a live stack, so a selector that stops matching is invisible to `tsc`, to the web build and to every host-side check. **That gap is closed** (5 Oct 2026): the console has component tests that run in `ci.yml`'s web job, in jsdom, needing no stack and taking about a second (`web/src/pages/Console.test.tsx`, vitest and Testing Library). They are not a second copy of the Playwright suite — they assert the handful of things it reaches for, which is what actually broke: the editor and the Run button by the accessible name the suite uses, one gutter row per line, a script's tab per statement, the editor going read-only in flight, the error line marked from Postgres's position, and that `.res-body td` first is the JSON expander while `td:not(.expand-col)` first is the value — the exact trap that failed two tests in a way that read like a product bug. Each was checked by putting the break back: removing the rollback note from the single-result path fails the note test, hard-coding the error position to zero fails the gutter test, and renaming the Run button fails all six, since every test goes through it. **The runners moved to Node 22** with them: jsdom 30 declares `engines ^22.22.2`, and on Node 20 it fails at import with `webidl.util.markAsUncloneable is not a function` before a single test runs — which says nothing about the Node version that caused it. The first attempt passed locally on 22 and failed in CI on 20 for exactly that reason, so `web/package.json` now declares the requirement and `web/.npmrc` sets `engine-strict`, which turns it into npm refusing with a clear message instead of installing happily and exploding later. GitHub had already begun warning that Node 20 is deprecated on their runners, so the bump was owed rather than forced. Running that test's second half for the first time found a real one: **a script that fails says nothing was applied only when it has two or more completed results**, because the note was rendered on the tabbed path alone — and a two-statement script that fails on the second comes back with one. That is the case where it matters most, since the first statement may well have been an `INSERT`; the note now renders on both paths. **A known limit it also exposed:** a statement that fails in parse analysis — an unknown table, say — is never described by Postgres, so it does not appear in `results` at all and has no tab of its own. The labels see that the statement count and the result count disagree and step aside rather than put SQL under the wrong grid, which is the behaviour that was asked for, and the error block names the line. Giving the failing statement its own tab would mean the engine synthesising an entry for it, which is a change to what a failing script returns and is left for its own batch.

ADDED IN

`84741df`; override, system group and reload on `fix/console-permissions`; value rendering on `fix/console-uuid-cells`.

L5 Blackbox page

Working

WHAT IT DOES Browse, filter and verify the Blackbox per branch, and read the statement behind any entry.

HOW IT WORKS `Ledger.tsx` : branch, status (APPLIED/FLAGGED/BLOCKED), kind (human/agent), actor, table, risk (drop, drop-column, type-change, security-definer, policy) and since/until date filters — every filter the API has — with "Clear filters"; infinite scroll 25 rows at a time; columns Time, Actor, Tool, Change, Status. A row expands to its entry id, session, branch, full statement and **the chain** — the entry's own row hash and the previous entry's, each 12 characters with the whole value on click (the list is fetched `with=session,chain` , a column set that is opt-in so the SDK contract is untouched). The page keeps itself up to date: every 3 s it asks for the first page and prepends whatever is new, marking it for two seconds, and it stops doing so while the tab is hidden, while a request is in flight, or once infinite scroll has moved past the first page — a reader is not interrupted by rows appearing under them. A **Live** toggle turns it off. An expanded entry on `main` offers **Branch from before this change** (H13) with a dialog naming the entry and stating that `main` is not modified; on another branch the action is absent and the page says why. **Verify** calls `…/ledger/verify` and reports the chain intact or the first broken entry, linking to Integrity. A request sequence number (`seqRef`) means a filter change always reloads and a stale response is dropped. Nav label and page title "Blackbox"; reachable at `/blackbox` and, as before, `/ledger` (H14).

HOW TO USE https://localhost:8080/blackbox

HOW TO TEST Integration-v2 §5b (the `with=session` and `with=chain` column sets, default columns unchanged, an unknown `with` value ignored, that a fetched entry's `prev_hash` really is the previous entry's `row_hash` , table and table+risk filters, and a date range). Unit: `ledgersql_test.go` . Console e2e: a change made elsewhere arrives without a reload, the chain is shown, and the rewind dialog names the entry. Manual: in the browser, filter by table and risk, set a To date, expand a row, and press Verify — checked against the same counts from SQL.

STATUS & LIMITS Working. Dates are whole days in the browser's locale, sent as `since` / `until` (the To date covers the day to 23:59:59). Filtering is server-side; the visible rows are whatever the filters return.

ADDED IN `e42e792` ; filters, verify and row detail `feat/blackbox-page-gaps` .

L6 API keys page

Working

WHAT IT DOES Create (shown once, copy button), list and revoke API keys. Since E3 this is where an install's first key comes from, so a new key is shown twice: on its own, and inside the connection string it belongs in. Also changes your password (every other session is signed out).

HOW IT WORKS `/keys` → `/api/keys` . The connection string is assembled in the page from the host the console was opened on and the gateway's port, so it is correct for a remote install as well as localhost.

HOW TO USE https://localhost:8080/keys

HOW TO TEST Manual create + revoke.

STATUS & LIMITS Working.

ADDED IN `235ff41` .

L7 Landing, Guide & Docs pages

Working

WHAT IT DOES Public landing page, a developer guide (install, connect, CRUD, branches, agents, PDF download) and a docs page (CLI, REST, env tables with live status buttons).

HOW IT WORKS `Landing.tsx` , `Guide.tsx` , `Docs.tsx` .

HOW TO USE https://localhost:8080/, `/guide` , `/docs`

HOW TO TEST Type-check and build (`npm --prefix web run build`); read line by line against the code, 16 Sep 2026.

STATUS & LIMITS Refreshed: the Guide drops `make web-dev` for `https://localhost:8080` , every connection string carries `sslmode=require` , and the agent-API examples use `https://localhost:8088` with a note on `-k` . The Docs page lists the whole REST surface (Blackbox and its aliases, policies, impact, diff, import, pipelines, keys, auth, agents), points at `GET /api/openapi.yaml` , and gains an "Agents over MCP" section linking `docs/mcp.md` . The `FOX_WEB_ORIGIN` default it shows (`http://localhost:5173`) was checked against `internal/auth/auth.go` and is correct. The landing page is unchanged.

ADDED IN `84741df` , `e9210fb` ; refreshed on `docs/api-accuracy` .

L8 Blackbox integrity page

Working

WHAT IT DOES	Verify a branch against its anchors, create a checkpoint, download the ledger export, and branch main from just before a ledger entry — from the browser.
HOW IT WORKS	`web/src/pages/Integrity.tsx` (route `/integrity`, nav "Integrity") calls the integrity, checkpoint and export endpoints and shows INTACT / TAMPERED, entry and checkpoint counts, problems and notes. The "Branch from before a change" panel lists main's 20 newest entries (`GET …/ledger/entries`) with a Select button (disabled for BLOCKED), takes an entry id and optional name and calls `POST …/ledger/{id}/branch` (H13), showing progress while the restore runs.
HOW TO USE	https://localhost:8080/integrity
HOW TO TEST	TypeScript type-check and production build pass. Reviewed signed in on 16 Sep 2026 (Chrome, macOS): on a test branch the report matched `fox blackbox integrity` field for field (INTACT, 61 entries, 61 hash-chained, 2 checkpoints, 2 anchored, 59 unanchored, same notes); **Create checkpoint** anchored a new entry (checkpoints 1 → 2, confirmed on disk) and was correctly a no-op when nothing new existed; **Export (JSONL)** answered 200 `application/x-ndjson` with one JSON line per entry; **Branch from before a change** restored `main-before-3649` in 4 s from the right base backup — the branch lacked the table that entry created, its Blackbox stopped at the previous entry, and main was untouched. No automated UI test.
STATUS & LIMITS	Working — exercised in a browser against a running stack (16 Sep 2026): verify, checkpoint, export and branch-before all matched the CLI. Note: the entry list is loaded once, so a change recorded after the page opened appears only after a refresh.
ADDED IN	Blackbox 2.0 Phase 3 (`feat/ledger2-phase3-checkpoints`); branch-before panel in Phase 4 (`feat/ledger2-phase4-branch-before`).

L9 Policies page

Working

WHAT IT DOES	Shows a branch's policy rules, lets a `db_admin` switch each between warn and block, enable or disable it, and add or remove custom rules; previews which rules a statement would trigger; lists recent warnings, blocks and overrides. **Who can override** lists a branch's `db_admin` members and lets an admin grant or revoke by email.
HOW IT WORKS	`web/src/pages/Policies.tsx` (route `/policies`, nav "Policies") over the `/api/branches/{n}/policies…` endpoints (H15). A non-admin sees the server's 403 message on any change.
HOW TO USE	https://localhost:8080/policies
HOW TO TEST	TypeScript type-check and production build pass; the endpoints it calls are covered by integration-v2 §6. Reviewed signed in on 16 Sep 2026 on a test branch: the four built-in rules matched `fox policy list`; warn → block → warn, disable → enable, and add → remove of a custom rule each matched the CLI afterwards and never touched `main`; **Check a statement** returned the same verdict as `fox policy check` (ALTER TABLE, WARN alter-column-type, same reason); on `main`, where the account holds no `db_admin`, a change was refused with a readable banner naming the grant command, and the rule stayed warn. No automated UI test.
STATUS & LIMITS	Working — exercised in a browser against a running stack (16 Sep 2026): rule changes, the statement check and the non-admin refusal all matched the CLI, and writes reached only the branch selected. Minor: the new-rule form pre-fills Command with `ALTER TABLE` as a real value, which reads like a placeholder and is easy to type after by mistake.
ADDED IN	Blackbox 2.0 Phase 5 (`feat/blackbox-phase5-policy`).

`L10` App sidebar layout

Partial

WHAT IT DOES	Gives the signed-in app a left sidebar with its pages grouped into sections, so adding pages doesn't overflow a single navbar; keeps the public navbar for the marketing pages.
HOW IT WORKS	`web/src/components/AppLayout.tsx`: `APP_NAV` lists the sections and pages (a new page is one entry), each with an inline SVG icon from `components/icons.tsx` (no icon library). The active page is highlighted (including the old `/ledger` address and nested pages such as `/pipelines/{id}`), and a top bar shows "section / page". The footer has GitHub, the light/dark switch, the account, Log out and Collapse; the collapsed state is stored in `localStorage` (`bb.sidebar.collapsed`, ignored if storage is unavailable). Under 860px the sidebar is an off-canvas drawer, opened from the top bar and closed by choosing a page, tapping outside or Escape. `web/src/main.tsx` routes Home and Log in through `PublicLayout`, the app pages through `SignedInLayout` (redirects to Log in when signed out), and Guide/Docs through `HelpLayout` (sidebar when signed in, public navbar otherwise). The logo and theme toggle moved to `components/brand.tsx` (still exported from `main.tsx`). Styles use the existing light/dark design tokens.
HOW TO USE	Sign in at https://localhost:8080 — the app opens with the sidebar. Collapse with the button at the bottom; on a phone use the menu button top left.
HOW TO TEST	TypeScript type-check and production build pass. A preview control plane built with `-tags embedui` served `/`, `/dashboard`, `/login` and `/blackbox` (200) with the sidebar code in the bundle. Reviewed signed in on 16 Sep 2026 (Chrome, macOS): Dashboard, Console, Import, Pipelines, Changes, Integrity, Policies, API keys, Guide and Docs each opened with the right section highlighted and breadcrumb, `/ledger` highlighted Changes, collapse survived a reload (`app-shell collapsed`, `bb.sidebar.collapsed=1`, applied before first paint), the theme toggle set and kept `data-theme`, and no console errors appeared on any page. Still manual: narrow the window below 860px and open/close the drawer (menu button, tapping outside, Escape, choosing a page). No automated UI test.
STATUS & LIMITS	Working — reviewed in a browser while signed in (16 Sep 2026): every sidebar link opened its page with the right section highlighted and breadcrumb (including `/ledger` → Changes), collapse persisted across a reload with no flicker on first paint, and the theme toggle applied and persisted. Not checked: the under-860px drawer — the automation could not change the page viewport, so it needs a manual look. Visual only: routes, addresses, APIs and page contents are unchanged. The breadcrumb shows the section and page name, not deeper context such as the pipeline being edited.
ADDED IN	`feat/web-sidebar-layout`.

L11 Design system & FoxByte theme

Working

WHAT IT DOES

Gives the whole web console one set of colours, type sizes, spacing steps, radii and motion, and one mark — so a page is laid out from the same vocabulary as every other page, and the product can be reskinned from a single block.

HOW IT WORKS

`web/src/styles.css` is two layers. The top is tokens: a 1.2 type scale (`--t-xs` … `--t-3xl`), a 4px space scale (`--s-1` … `--s-8`), `--radius-sm/--radius/--radius-lg/--radius-pill` , elevation, `--speed` , a focus `--ring` , and the palette per theme. Everything below refers to tokens — no raw hexes, no picked pixel values. The palette is the mark, and takes both halves of it (4 Oct 2026). The cube has a full warm ramp and a full cool one, so the product divides the same way: **warm for what you press, cool for what you are looking at**. Buttons are the mark's orange `#fb6300` into its amber `#fec42d` — one hue family, so a filled button reads as one colour with depth rather than as a second logo; the cube's orange-to-blue sweep belongs to the mark and to the wordmark's x, not to a control someone clicks forty times a day. Links, focus rings, selection and the active nav item take the mark's blue, lightened to `#2f8bff` for graphite and darkened to `#0a57d6` for paper, because a brand colour and a legible colour are different problems: the artwork's own `#006bff` reaches 3.81:1 on the dark background and fails as link text. The light theme's orange it replaces managed 3.81 too, so links got more legible rather than less. `--info` moved to the mark's cyan (`#04d1fe` , `#0e7490` on paper) so that an informational blue is not the accent blue. `--accent-ink` is unchanged and was re-checked against both fills it now lands on: 6.08:1 on the orange, 5.53:1 on the blue, and 6.27:1 for white on the light theme's blue. The light theme's warm ramp stays darkened, because white ink sits on those fills and the mark's own orange is too light to carry it; `--accent-ink` is the text colour on an accent fill (dark on orange in the dark theme, white in the light one), which is why the filled buttons are legible in both. `--editor-bg` lets the SQL editor follow the theme while a terminal transcript stays dark. **The mark is the supplied logo** (4 Oct 2026): an orange-to-blue cube, shipped as artwork rather than drawn by this repository. `Mark()` in `web/src/components/brand.tsx` renders `web/public/mark.png` at 128px, which covers a 3x display for something never drawn above 30px and costs 19K; it carries the product name as its `alt` rather than being marked decorative, because the sidebar collapses to the mark alone. What this gives up is named rather than glossed over: the mark it replaced was an SVG fox head filled from `var(--grad-a)` / `var(--grad-b)` , so it followed the theme, and a fixed image cannot — which is the right way round, since the logo is the logo on either page, and the supplied mark is the one asset with real transparency, so it needs no plate behind it. `Wordmark()` still derives from `brand.json` rather than markup, but now picks out the letter before the second capital — the **x** the logo itself colours — in `--grad-mark` , the mark's own ember `#ff7f02` to blue `#006bff` sampled from the artwork. Those two are deliberately kept apart from `--grad-a` / `--grad-b` , which tint every button and focus ring: taking the mark's blue across the whole palette is a design decision rather than a logo one, and has not been taken. `web/public/favicon.png` (48px) and `apple-touch-icon.png` (180px, on the brand's dark ground, because iOS ignores a touch icon's alpha and composites it on black) replace the old `favicon.svg` . **The logo is generated, not cut by hand** (5 Oct 2026). It has now been replaced three times, and each time the work was the same — trim, scale to a handful of sizes, key the background out, and remember every place that hard-codes a size or a colour. `brand.json` gained a `logo` block naming four source files and three colours, and `cmd/brandgen` derives everything else: two app marks at 128px, a 48px tab icon and a 180px iOS icon on an opaque plate, two 512px marks and two 720px lockups for the documents, and `web/src/brand.gen.css` carrying the colours as custom properties. Changing the logo is now: replace the files in `docs/brand/source/` , set the colours, run `make brand` . Because the output goes through the same `gen.Files` map as every other generated file, `make brand-check` and `TestGeneratedFilesAreInStep` cover it for free — forgetting to regenerate is a red build rather than a favicon a version behind the mark beside it. The generator keys the artwork's background out by distance from the corner pixel with a feathered edge, so every derived asset is transparent; it scales by averaging the source pixels under each destination pixel, which is the right filter for a reduction and needs no image dependency. The mark ships in two inks because its cool half is white on a dark page and navy on a light one, and CSS picks between them, so the right one is painted on the first frame rather than after a script decides — the wordmark's accented letter can follow the same rule, and no longer does: the artwork that introduced it drew its own letter in a gradient, this one draws the name in a single colour, so `logo.wordmark_accent` is off and the app matches the logo rather than the one before it. The per-theme colours are kept behind the flag, because a single cool colour vanished into the dark background when the accent was last on. The tab icon is transparent — two of them, chosen by `prefers-color-scheme` , since no one transparent icon reads on both a pale and a dark tab bar; the iOS icon is the single exception that keeps an opaque ground, because iOS ignores a touch icon's alpha and composites it on black. **One thing only Windows found:** `.gitattributes` pinned `*.go` , `*.sh` , `*.ts` and the rest to LF but said nothing about `*.css` , so the first generated stylesheet was checked out with CRLF on windows-ci while the generator wrote LF — `TestGeneratedFilesAreInStep` failed there and only there, with a message about brand.json that said nothing about line endings. `*.css` is pinned now, and `TestGeneratedTextFilesHaveALineEndingRule` reads `.gitattributes` and fails on any platform if a

	generated text file's extension is not covered — so the next one is caught before it reaches a runner rather than by it. `docs/brand/` now holds the artwork itself rather than a page that drew it — `logo.html` and its two render commands are gone — with the mark, both lockups and both wordmarks; only the mark is transparent, because the dark lockups' backgrounds are a gradient and cannot be keyed out without a halo, which is why the README keeps a light/dark pair for GitHub to choose between.
HOW TO USE	Style a new page with the tokens: `var(--s-*)` for spacing, `var(--t-*)` for type, `.page-head` for the title and its controls, `.filterbar` for filters, `.table-wrap` for a table, `.empty` for nothing-to-show. Reskinning is the token block; renaming is `brand.json` (see docs/branding.md).
HOW TO TEST	TypeScript check and production build. Driven in a browser against a canned control plane on 21 Sep 2026: Dashboard, Console (query, result, browsed table), Blackbox, Policies, Integrity, API keys (including a freshly minted key), Import, Pipelines, Login and the landing page, each in both themes, and the console at 375px wide.
STATUS & LIMITS	Working. Visual only — no route, API or behaviour change. Not yet reviewed against a running engine with real data (the canned control plane stands in for one), and the emoji icons on the landing page's feature cards are still emoji rather than drawn marks.
ADDED IN	`feat/foxbyte-ui`; the palette and mark replace the purple theme the product carried through both earlier names.

AREA M

Build, CI & releases

M1 Makefile targets

Working

WHAT IT DOES	One place to build, test and release.
HOW IT WORKS	`build`, `vet`, `fmt`, `test`, `vm-build` (Lima, embedui), `integration`, `integration-v2`, `integration-update` and `integration-pg-upgrade` (in the throwaway `fox-test` VM; `test-vm`, `test-vm-stop`, `test-vm-delete` manage it), `web-build`, `web-dev` (deprecated), `release` (5 `fox` + 5 `fox-verify` binaries + docker context), `release-linux`, `wsl-distro`, `wsl-zfs`, `feature-doc` (renders this PDF and `FOX_Checklist.pdf`, after checking both list the same features).
HOW TO USE	`make test` · `make integration` · `make release VERSION=x.y.z` · `make feature-doc` `make integration-sdks` (the published clients) and `make integration-ui` (the console, in a browser; the first run downloads Chromium into the VM, and the target builds the UI on the host because the repository is read-only in the VM).
HOW TO TEST	CI runs build/vet/test/gofmt. On 17 Sep 2026 all three integration suites ran in a new `fox-test` VM: `integration` 53 / 53, `integration-v2` 227 / 227, `integration-update` 51 / 51, after two bugs they found were fixed (G1, H16). The guard was checked on the real VM without running a suite: each stops with exit 2 before its first action, and a missing guard stops it too.
STATUS & LIMITS	Working. `release` prints an outdated ZFS-bundle note. The integration suites are destructive by design (they wipe Blackbox history, restore main to an earlier point, fail HA over, create accounts and keys), so they run only in a VM that `scripts/test_vm.sh` has marked (`/etc/fox-test-instance`): `scripts/lib/test_guard.sh` refuses everywhere else. The test VM never uses the names `foxbyte` or `default` (fox would take it for the real install) and forwards no ports to the Mac. It needs about 4 GiB of memory while running; `make test-vm-stop` frees it.
ADDED IN	`a7580f6`; `feature-doc` added with this document.

M2 Linux CI

Working

WHAT IT DOES

Every push to main and every PR is built and tested on Linux. Nightly, the integration suites run too (audit v2 G24).

HOW IT WORKS

`.github/workflows/ci.yml` : `go build` , `go vet` , `go test` , gofmt check. Also runs `scripts/test_release_checksums.sh` , which checks the release checksum merge never drops another job's entries, and `scripts/test_check_release_assets.sh` , which checks the release job's test that every file `fox update` needs was published, and `scripts/test_feature_checklist.sh` , which checks `FOX_Checklist.html` lists exactly the features in this document. **Nightly (.github/workflows/nightly.yml, 23 Sep 2026):** one runner per suite — integration, ledger-v2, update, pg-upgrade, sdks, ui — each installing ZFS and psql, building the engine (with the UI embedded for the console's tests) and running that suite with `FOX_TEST_ALLOW_REAL_INSTALL` , which the guard allows because the runner is thrown away. A failure keeps the log, and the console's screenshots and traces, as artefacts for 14 days. `workflow_dispatch` runs one suite on demand. **A seventh suite, `upgrade` (3 Oct 2026):** `scripts/integration_upgrade.sh` asks the question none of the others do — can a *previously released* fox update to this build? Every other suite has the current build do the updating, which is the wrong way round, because `fox update` validates a staged engine by parsing its `version` output and the parser doing that work belongs to the binary already installed. It builds each recent tag from its own source (selected by comparing `go.mod` , so tags from before the FoxByte rename are skipped as a different module rather than kept in a list that would go stale), serves this tree as a fake newer release, and checks each old binary accepts the staged engine. It needs no stack at all — on Linux staging is a file copy and the check is a `version` call — so it runs with a throwaway `HOME` , touches no install, skips the ZFS, engine and image setup the others need, and takes about a minute. The checkout gained `fetch-depth: 0` for it, since the default depth carries no tags and the suite would have had nothing to test; it says so and fails rather than passing empty, and fails too if every version skipped.

HOW TO USE

Automatic on PRs.

HOW TO TEST

Open a PR and check the Actions tab; the nightly job also runs on demand (Actions → nightly → Run workflow).

STATUS & LIMITS

The suites run nightly rather than on every pull request: they take an hour of runner time and are destructive. They run on GitHub's runners, whose kernel does carry the ZFS module. **Release tags are gated on them** (since 2 Oct 2026): `release.yml` 's `nightly-gate` refuses a tag unless the newest completed nightly concluded green, is under 48 hours old and skipped no job — a one-suite run concludes "success" while having tested almost nothing, which is worse than a red gate because it looks like evidence. Adding the `upgrade` suite to the matrix therefore put it behind that gate with no change to the gate itself. One hole remains by design: whether the tagged commit is the one the nightly ran is a *warning*, not a refusal, so a tag can ship code no nightly has seen.

ADDED IN

`797bc14` (PR #11).

M3 Windows CI

Working

WHAT IT DOES

Tests the Windows build and installer on every push/PR.

HOW IT WORKS

`windows-ci.yml`: build/vet/test, cross-compile, Pester (both files), install.ps1 encoding + PS 5.1 parse; Linux job checks `make release` assets.

HOW TO USE

Automatic.

HOW TO TEST

Actions tab.

STATUS & LIMITS

No end-to-end WSL install test (needs nested virtualisation). Fixed 21 Sep 2026: this job is the only one that runs the repository's guard tests on backslash paths and a CRLF checkout, and both failed there while every other job was green — a path exemption that matched nothing, and a generated file that looked like drift because Git had rewritten its line endings. Worth

M4 Releases & GHCR image

Working

WHAT IT DOES

A `v*` tag publishes binaries, checksums and a multi-arch Postgres image.

HOW IT WORKS

`release.yml`: `make release`, verify assets (including `fox-verify`), create (pre)release, `SHA256SUMS`, upload; image job builds `docker/postgres` for amd64+arm64 once per supported PostgreSQL major (a matrix over `16` and `18`, passed as `PG_MAJOR`) and pushes `ghcr.io/thefoxbyte/postgres-walg:<major>` and `:<major>-<tag>`, labelled with `org.opencontainers.image.source` so the package links itself to the repository. Checksums go through `scripts/release-checksums.sh`, which merges into the release's existing `SHA256SUMS` instead of replacing it, so the distro image's entry from `wsl-distro.yml` is kept whichever job finishes first. A last step, `scripts/check-release-assets.sh`, fails the job unless every file `fox update` needs (A9) is attached and listed in `SHA256SUMS`; a Go test keeps its list in step with the updater's.

HOW TO USE

Push a tag `vX.Y.Z` (or run manually with a tag).

HOW TO TEST

Check the release page assets, and `docker pull ghcr.io/thefoxbyte/postgres-walg:18` and `:16` anonymously — the package must be public, or every install fails with `unauthorized` (it did on 21 Sep 2026, on `v0.1`).

STATUS & LIMITS

Working. GHCR package must be made public manually the first time. **Supply chain (22 Sep 2026, audit v2 G22, G23):** every Action is pinned by commit SHA (Dependabot proposes updates; its groups are minor-and-patch only since 3 Oct 2026, because a group matching everything swallows majors too and a framework upgrade then arrives wearing the same clothes as a patch bump — "bump the web group across 1 directory with 8 updates" was React 18 to 19 and React Router 6 to 7. An update outside its group gets its own pull request, so majors arrive alone and named. Go modules are grouped for the first time, which turns five open pull requests a week into one, with `pgx` deliberately excluded: it is the driver every query goes through, the only tests that would notice a regression are the nightly suites, and "which of these five did it" is a question worth not having to ask). SHA256SUMS is signed with the release key from the `FOX_RELEASE_SIGNING_KEY` secret (`cmd/releasesign`; a missing secret fails the release; the merge with wsl-distro.yml re-signs when the two race). Every file gets a GitHub build attestation (`gh attestation verify <file> --repo thefoxbyte/foxbyte`), and the release carries an SPDX SBOM (`foxbyte-sbom.spdx.json`). Images are built with provenance and an SBOM and attested by digest. First run on the next tag; not yet exercised on GitHub.

ADDED IN

`8cafdb6` (PR #2); image job `aae9e20` (PR #12).

	reading a Windows-only failure as a question about the platform before the code.
ADDED IN	`4fc8afb` (PR #1); path and line-ending fixes on `fix/windows-ci`.

M5 WSL distro & ZFS builds

Partial

WHAT IT DOES

Builds a prebuilt WSL distro image; legacy ZFS module bundle for WSL kernels.

HOW IT WORKS

`wsl-distro.yml` / `make wsl-distro` (Ubuntu + Docker + btrfs + engine + saved images), published as `foxbyte-distro.tar.zst`: `zstd -19 --long=27` since 21 Sep 2026, gzip before. The build compresses the same bytes with gzip as well and prints both sizes, so the saving is a measured number in every build log. `wsl --import` and Windows' `tar.exe` are only dependable with gzip, so `fox.exe` unpacks the image itself (`internal/host/distro_image.go`, the pure-Go `klauspost/compress` decoder) into a plain tar beside the download, imports it, and removes it; the `.zst` is kept so a re-run does not download again. It writes to `.partial` and renames at the end, so an interrupted unpack never leaves a truncated tar for the import to take as whole. The installer tries the zstd image, then the gzip one older releases published, then the Ubuntu rootfs; `wsl-zfs.yml` / `make wsl-zfs` (OpenZFS for the WSL kernel). It creates the release if needed, then

	publishes its checksum through the same `scripts/release-checksums.sh` as `release.yml`.
HOW TO USE	Tag push (distro) or manual dispatch.
HOW TO TEST	Actions artifacts and the size comparison in the build log; a fresh Windows install from v0.8.6 imported the image and loaded its preinstalled containers. Unit: `TestUnpackZstdRoundTrip`, `TestUnpackZstdLeavesNothingOnDamage`, `TestImportableDistroPassesGzipThrough`, `TestDistroBuildWindowFitsTheDecoder`, `TestDistroImagesPreferZstd`; Pester checks the installer prefers zstd and skips an

	image the release does not list.
STATUS & LIMITS	Confirmed end to end on Windows (16 Sep 2026): the installer downloaded and verified the distro image, and a fresh install imported it — setup skipped the apt work entirely and loaded the preinstalled images. It now does that on every install, and the build tags its Postgres image with the name a fresh install runs (`ghcr.io/thefoxbyte/postgres-walg:18` since 21 Sep 2026; a test keeps it in step with `PGMajor`) so the preload is used. The setup-side "already loaded" probe checked the pre-16-Sep legacy name, which a newer distro never carries, so every re-run of setup loaded the whole tarball again; it now probes the image the preload actually holds. Still open: the image was 685 MB as gzip; the zstd version's size will be in the first build log after 21 Sep 2026, and **the unpack has not yet run on a real Windows machine** — unit-tested only (round trip with a 128 MiB window, a truncated download refused with nothing left behind, gzip passed through, and the build's `--long` held within the decoder's limit by `TestDistroBuildWindowFitsTheDecoder`). It needs about 2 GB free while it imports (a fallback to the Ubuntu rootfs covers slow or failed downloads), and the ZFS bundle is legacy (Windows uses btrfs). **A release is held until something has actually installed it** (3 Oct 2026). Until now every check was about presence — the files exist, SHA256SUMS lists them, the signature is ours, the nightly is green — and none of them ran the published binary. v1.0 passed all of them and was then refused by every install that tried to take it. Two changes. **The prerelease hold is real:** the workflow already created releases with `--prerelease` and a comment saying to promote by hand once smoke-tested, but that only ran when the workflow created the release — publishing by hand (`gh release create` or the web UI) makes the tag and the release together, so the `gh release view` test succeeded and the flag never applied. Neither v1.0 nor v1.0.1 was ever a prerelease: the guard had never once taken effect. It

is now forced with `gh release edit --prerelease` whichever way the release was made, and that matters because `fox update` does not offer a prerelease — an unverified release cannot reach anybody. **And `release-verify` earns the promotion:** a new job that installs from the *published* assets with `install.sh` and checks the binary reports the version it was published as, both as field two and as the last field (which is what installs older than v1.0.1 read); then installs the greatest earlier full release and updates that to this one, over the real assets, asserting the staged engine is accepted. `--version` is what makes this possible while the release is still a prerelease: `Candidates()` admits a pinned prerelease while an ordinary update never offers one. Only on success does it run `gh release edit --prerelease=false --latest`. If it fails, the release stays unreachable instead of reaching users broken. It stops at the staged-engine check rather than completing the update, which would need a pool and containers; what happens after that check is the nightly `update` suite's job against a real stack, and what only this job covers is the published artifact. The Windows install path is still not covered by CI (A9) and is named in the job summary as a manual step.

ADDED IN	`d0c1f26` (PR #7); `8305e5b` (PR #1).

M6 Postgres image

Working

WHAT IT DOES

Stock PostgreSQL plus wal-g for archiving, backups and PITR. A fresh install runs **PostgreSQL 18**; an install created on 16 keeps running 16.

HOW IT WORKS

`docker/postgres/Dockerfile`: `ARG PG_MAJOR=18`, `FROM postgres:${PG_MAJOR}-bookworm`, wal-g `v3.0.9`, `restore-entrypoint.sh`. One Dockerfile serves every supported major; bookworm is kept for all of them because the wal-g release binary is built against an older glibc. The majors live in one place, `internal/branch/images.go` (`PGMajor` for a fresh install, `SupportedPGMajors` for what fox can still run), and `TestPostg

M7 Product name in one file

Working

WHAT IT DOES

Names the product in one place, `brand.json`, so a rename is an edit, a regenerate and a sweep — and no install ever has to be migrated or reinstalled because of one. The product has been renamed twice (VectoraDB, then OxynDB — renamed from those to FoxByte), and each time before this the name had been written into the data.

HOW IT WORKS

Two layers. **Names people read** — product, command, environment prefix, state directory, repository, module, image registry, test VM, document prefix — come from `brand.json`; `make brand` (`cmd/brandgen`) writes them into

`resMajorIsInStepEverywhere` fails if the Dockerfile default, the release workflow's matrix or the Windows preload disagrees with them. **The image follows the data, not the binary** (`pgImage`, `internal/branch/pgversion.go`): the engine reads `PG_VERSION` from main's data directory and runs the image for that major, so `fox update` onto a binary that ships 18 leaves a 16 install on 16 — moving a cluster between majors is a migration, not an update. `FOX_PG_IMAGE` overrides it. Before every `docker run`, `checkDataMajor` compares the branch's `PG_VERSION` with the image's major and refuses a mismatch	`internal/brand/brand_gen.go`, `scripts/lib/brand.sh`, `web/src/brand.ts` and a marked block in each installer (fetched standalone from GitHub, so they carry literals). **Names written into data** — schema `bb`, roles `db_client` / `db_admin`, superuser `dbadmin`, database `appdb`, SQLSTATEs `BBX01` / `BBX02`, key prefix `key_`, cookies, pool `dbpool`, network `dbnet`, containers `pg-*`, object store, bucket, label, anchor format — are brand-free and frozen, defined once in the package that creates them. Retired names keep working where they matter: an

with both versions named and the way out, instead of leaving Postgres to fail inside the container with "database files are incompatible with server". **Pinned by content (22 Sep 2026, audit v2 G23):** the base image by digest per major (`PostgresBaseDigests`; `FROM postgres:${PG_MAJOR}-bookworm@${PG_DIGEST}`), and wal-g by the SHA-256 wal-g publishes for each asset, checked with `sha256sum -c` before unpacking (both confirmed against the release's own .sha256 files). MinIO images are pinned by tag and digest too; the old unpinned `minio/minio:latest` is no longer accepted. `scripts/check-`

environment variable under a retired prefix is still read, a retired state directory is moved, a VM or distro from a retired name is still found, and `fox uninstall` removes installs of retired names. `scripts/rebrand.sh --product X --cli x --repo owner/repo` edits `brand.json`, regenerates, sweeps prose and file names, and renames the module path. `docs/branding.md` is the procedure.

HOW TO USE

Rename: `scripts/rebrand.sh …`, then `go test ./...`, `make feature-doc`, the suites. Day to day: `make brand` after editing `brand.json`; `make brand-check` checks without writing. CI enforces it `image-digests.sh` reports a pin whose tag has moved on.

HOW TO USE

Pulled automatically (C8). `FOX_PG_IMAGE=<ref>` for a registry mirror.

HOW TO TEST

Unit: `TestResolvePGImageFollowsTheData`, `TestCheckDataMajor`, `TestImageMajor`, `TestParsePGVersion`, `TestBuildImageArgsPassTheMajor`, `TestPostgresMajorIsInStepEverywhere` (checked to fail when the Dockerfile default is set back to 16). `docker run --rm ghcr.io/thefoxbyte/postgres-walg:18 wal-g --version`. The three suites in the throwaway VM on 18.

STATUS & LIMITS

Working on PostgreSQL 18 (21 Sep 2026): all

through `go test` (`TestGeneratedFilesAreInStep`), not a separate step.

HOW TO TEST

Unit: `TestGeneratedFilesAreInStep` (every generated file matches `brand.json`), `TestInstallerBlocksKeepTheirVariables` (a PowerShell `$` once vanished in generation), `TestBrandJSONIsUsable`, `TestLoadRejectsIncompleteBrand`, `TestGetenvFallsBackToRetiredPrefixes`, `TestStateDirMovesFromARetiredName`, `TestConstantsAreSane`; repository-wide guards in `cmd/fox/oldnames_test.go`: `TestNoRetiredProductNames` (a retired name anywhere fails, except lines marked as history or `legacy:`), `TestStore` three suites passed in the throwaway VM on a cluster reporting `PostgreSQL 18.6` — `make integration` 108/108 (wal-g archiving and base backups, point-in-time restore, HA failover and failback including the `pg_controldata` parse, uninstall), `integration-v2` 277/277 (Blackbox attribution through `PG_CONTEXT`, event-trigger coverage, the guardrail and `dblink` BLOCKED recording, policies, the hash chain), `integration-update` 51/51. The image in that run was built locally by the `ensureImage` fallback (the `:18` tag was not yet published), so the `--build-arg PG_MAJOR`

`dNamesAreBrandFree` (the SQL the engine installs names no product), `TestDurableNamesAreNotSpeltOutTwice` and `TestDurableNameExemptionsReadWindowsPaths`.

STATUS & LIMITS

Working. The sweep is text substitution, so it can over-reach: one rule for the SQL schema also caught the Windows launcher's file name (A2, fixed 21 Sep 2026), and the WSL distro's name (`vm_instance`) was left as the old slug in prose (A4, fixed). Review a sweep's diff; the guard tests catch retired names, not wrong new ones. A self-update cannot cross a rename, because release assets are named for the command.

ADDED IN

`a5ca38f`; generated shell names

path is proven too. The cluster has data checksums on and `uuidv7()`. wal-g stays at v3.0.9, which handled 18 throughout. Rechecked on 18: `pg_settings` still does not list a custom placeholder setting, so the reasoning in the 18 Sep change-log row holds. Moving an existing install from 16 to 18 is `fox pg upgrade` (D6). No extensions added (pgvector is not included). **pgvector** is in the image for both supported majors (audit v2 G34), installed from the PGDG repository the official base already trusts and pinned to a version like everything else here. Nothing is enabled by its presence: `CREATE`

`7f1b9a7`; `fox uninstall` for retired installs `e95820a`.

`EXTENSION vector` is the user's to run, per database. It is there because the product is aimed at agent workloads, and an agent that needs embeddings should not have to leave the database it was given.

ADDED IN	`a7580f6`; GHCR name `aae9e20`; PostgreSQL 18 and the per-major images on `feat/postgres-18`.

M9 Licensing

Partial

WHAT IT DOES

A signed file unlocks the paid features of an Enterprise build. Without one an Enterprise binary behaves exactly like Standard, which is the property that stops a build unlocking itself.

HOW IT WORKS

`internal/license`. A licence is JSON plus an Ed25519 signature over a canonical rendering of its own fields — domain-separated by `foxbyte-license/1`, newline-joined, fixed order, features sorted, times RFC3339 in UTC — the same shape an anchor's payload has (`internal/ledger/sign.go`) and for the same reason: the

M8 Two editions

Working

WHAT IT DOES

Builds two binaries from one tree. `fox` is **Standard**, free and AGPL. `fox-enterprise` is the same engine with the code under `enterprise/` compiled in; its features need a licence, and without one it behaves exactly like Standard. The command is `fox` either way, so nothing downstream of the install has to know which one is present.

HOW IT WORKS

One repository, two licences — the mechanism the tree already used for `clients/LICENSE` (Apache-2.0 under an AGPL root), applied again: `enterprise/LICENSE` is commercial, and every file there carries `LicenseRef-`

signer and every verifier must agree on the bytes. It carries a `key_id`, so the signing key can be rotated without invalidating every licence already issued; that field is inside the payload, because one a forger could edit would name whichever key they liked. The public half is pinned in a `var` exactly as the release key is, so a test build can be given a test key with `-ldflags -X` and nothing else can. `cmd/licensesign` mints them: `generate`, `issue`, `verify`, with the private key read from `FOX_LICENSE_SIGNING_KEY` and never from a file

`FoxByte-Enterprise-1.0` instead of the AGPL header. Everything under it is behind `//go:build enterprise`, following `internal/branch/insecure_on.go` / `insecure_off.go`, with one untagged file per package so `go build ./...` and `go vet ./...` stay green without the tag (the reason `web/embed_stub.go` exists). **The seam is `internal/edition`**, and it answers two different questions: `Enterprise` is a build-time constant, `Has(Feature)` a run-time one the licence layer supplies. They are separate on purpose — conflating them would make "enterprise build" mean "licensed", and nothing would stop a build unlocking

HOW TO USE	in the repository. `make license-key` once · `go run ./cmd/licensesign issue --customer "Acme Ltd" --features anchors --months 12` · `verify` to read one back. `make license-samples` mints a batch of evaluation licences (`docs/evaluation-licences.md`). **Mint before putting the key away**: signing reads `license-signing.key` and there is no second copy, so the order `generate` prints is commit the public half, mint, then put the private half in a password manager and delete the file — it	itself. The core never imports `enterprise/`; only the `_on.go` halves of tag pairs may (`internal/controlplane/enterprise_on.go`, `cmd/fox/enterprise_on.go`), and those are compiled out of Standard. Asset names carry the edition, and **both binaries on a machine must match**: on macOS and Windows the launcher is on the host and the engine in the VM, so `host.EngineAsset` names the engine after the launcher's own edition, and `update.binName` keeps a self-update inside its edition — fetching the Standard asset into an Enterprise install would silently remove the paid features and read to the

used to say delete second, which would have left every binary from that commit trusting a key nothing could sign for.

HOW TO TEST

`internal/license/license_test.go`: a signed licence verifies and survives a round trip through JSON; **each signed field in turn is edited and the signature must break**, which is what proves the payload covers all of them; a licence signed by another key is refused by name; the expiry states are walked hour by hour across the grace boundary; a fingerprint mismatch warns and still unlocks;

user as their licence having stopped working.

HOW TO USE

`curl -fsSL …/install.sh | FOX_EDITION=enterprise sh`, or `sh install.sh --edition enterprise`; on Windows `$env:FOX_EDITION = 'enterprise'`. Locally: `make build` and `make build-enterprise`. `fox version` and `fox check` both name the edition and whether a licence is active. `GET /api/status` reports `edition` and `features`, so the console can show a paid feature locked rather than hiding it — a hidden feature sells nothing.

HOW TO TEST

Unit: `TestHasNeedsBothTheBuildAndALicence`, `TestALicenceGrantsOnlyWhatI` an unreadable machine id is not a mismatch; a site licence runs anywhere; the signing payload's exact bytes are pinned, because every licence already issued is signed over them; and the private key must be in `.gitignore`.

ACTIVATING FROM THE CONSOLE

`POST /api/license` and an **Account → Licence** page: paste the licence, press Activate. **Why this exists:** somebody meets a locked feature in the console, and until now the way out was a terminal, a file saved to disk and a command they had to be told about — the person who needs to activate is rarely the person

STATUS & LIMITS

`tNames`, `TestNameMatchesTheBuild`, `TestEveryFeatureDescribesItself`; the boundary guards `TestLicenceHeadersMatchTheDirectory` (both directions), `TestOnlyTaggedFilesImportTheEnterpriseCode`, `TestEveryEnterprisePackageHasAnUntaggedFile` — each checked to fail against a deliberately misplaced file. `bash scripts/test_editions.sh` builds both editions and inspects the linked binaries with `go tool nm`, since only the binary can show the code is absent. CI runs `go build`, `go vet` and `go test` a second time with `-tags enterprise`; without it the paid code is never compiled at all.

Working. **Source-**

at a shell. **Admin only**, because it changes what the whole engine serves. **A forgery is refused outright and nothing is written**, while expired or bound-elsewhere installs with the state saying so — the same rule as `fox license activate`, and it has to be: two doors that disagreed about what is acceptable would be a way around one of them. **It takes effect at once** for this process, because the entitlement is otherwise read only at startup and an activation that needed a restart before the console would show it working is most of the friction the

available is a legal deterrent, not a technical one: the code can be read and built, and what changes is that removing a check is a licence breach rather than something the AGPL permits. Forging a licence is a different matter and is not possible — only the public key ships. **An AGPL grant cannot be revoked:** anything already published under it stays free at that version, so paid value has to be new capability rather than a removal — which is why the Blackbox record and the agent guardrails stay in Standard. **`go.mod` is not build-tag aware:** a dependency only the paid code uses still appears in the AGPL page exists to remove. **What it cannot finish is said, not hidden:** the Gateway and the Agent API are separate processes that read the entitlement when they start, so they need a restart; and on macOS or Windows the control plane is inside the VM while `fox license` deliberately runs on the host — there is no channel back so only the VM’s copy was written and rebuilding the VM would lose it. `host.InGuest()` decides whether that second sentence appears at all. **A Standard build is offered no paste box**, only the reason and the install command: a licence

module's manifest. The guarantee that holds is about the binary, not the manifest, and fixing it would need a second module — the thing one repository exists to avoid. While `enterprise/` holds no compiled code the symbol check is vacuous, and `scripts/test_editions.sh` says so rather than reporting a pass that means nothing.

ADDED IN

`feat/editions`.

cannot unlock code that is not in the binary, and accepting one would invite somebody to paste it and wonder why nothing changed. The page also names an empty fingerprint as a *site licence* rather than leaving it blank, which is the distinction that decides what happens the first time a standby is promoted on another machine.

STATUS & LIMITS

Partial: the licence, the tool that mints it, the host fingerprint, `~/.fox/license.json` and `fox license activate|show|rebind|remove` exist and are tested. **A licence now entitles:** `license.Install()` runs once the launcher has handed over and this process is the engine, reads the installed licence and gives `internal/edition` the predicate behind `edition.Has` — the one place a licence becomes an entitlement. It is silent on every failure (none installed, no key in the build, lapsed, forged), because a refusal belongs where someone asks for a paid feature and can be told which one and what to do; announcing it on every command would be noise for the Standard installs that are the common case. `fox check` gained a licence line that reports the same `Reason` and `Action` the commands print, from the same `Status`, so the two cannot drift into describing one licence two ways. Demonstrated end to end with a throwaway key: an Enterprise build reports “no licence active”, then “2 features licensed” once one is activated, then “no licence active” again after `remove` — and **a Standard build with the same licence file is unchanged**, which is the property the edition split exists for. **The security log now carries it.** Four new event kinds — `license.activated`, `license.rebound`, `license.lapsed`, `license.removed` — appended to the same hash-chained, append-only `security_events` the sign-ins and refusals go to, so `fox audit verify` covers them and the checkpointer anchors them like everything else. **The engine records, not the command.** On macOS and Windows `fox license` runs on the host, which has no store to chain an event onto — but the better reason is that an auditor is asking which features *this engine honoured*, and only the engine can answer that. So the commands change the file and the engine records what it then honoured, which also means a rebind done on a laptop is recorded once, by the thing it affected.

And it records only at start-up, which is a decision rather than a shortcut. An entitlement is established once, when the process starts; nothing re-reads the licence while the server runs. A watcher that noticed a licence expiring at 3am and wrote “lapsed” would be putting an *untrue* event into a tamper-evident log, because the running engine is still unlocking those features and will until it restarts — a late true record beats a prompt false one. Making the entitlement itself live is the other half of the portal lease work, and the record follows it there. `license.Record` is therefore idempotent by construction: it compares against the last licence event’s **kind and subject** (two stored columns, so nothing parses a sentence written for a person — the subject is `<id>@<machine>`, which between them carry which licence, which machine, and whether it was unlocking), and a steady install records one event ever rather than one per restart. A *Warning* is recorded as an activation, because a warning unlocks — the reason goes in the detail, to be read rather than enforced — and a forgery is recorded as lapsed, because an auditor wants to know one was presented. An unreadable log writes nothing at all: treating “cannot read” as “nothing recorded” would duplicate the activation on every start. The actor column is deliberately empty: nobody did this, the engine observed it, and inventing a name in an audit log is worse than leaving it blank. `Status` gained `BoundTo` and `Rebinds` on the way, because “activated on a different machine” cannot be reported, recorded or renewed without saying which. **One gap this uncovered:** a licence changed under a running engine does nothing until a restart, and nothing said so. `activate`, `rebind` and `remove` now check whether the control plane is answering (`host.EngineRunning`, one ask with a short deadline) and say which commands apply the change. **`GET /api/license` and the console’s notice.** `/api/status` already says which edition this build is and which features are usable; this says what to *do* when they are not — whether a licence is installed at all, what is wrong with it, and what would fix it, from the same `Status` the CLI prints, so the console and `fox license show` cannot describe one licence two ways. It reports what *this process is honouring* (`license.Installed`) rather than re-reading the file: a console showing a licence activated five minutes ago as active, while the running engine still refused the features, would be worse than showing nothing. **Two tiers:** every signed-in user sees the state, the expiry and the wording, because any of them may hit a locked page and deserves to know why; who bought the licence and which machine it is tied to are the account’s details rather than the product’s, so those are for admins. The console’s notice (`web/src/components/LicenseBanner.tsx`) follows four rules, each a decision: it shows **nothing** for a working licence, because a strip that is always there is wallpaper and the next one that matters goes unread; **nothing in Standard**, because a Standard user is not a failed Enterprise user and a permanent strip about what they have not bought is an advert — paid features are shown locked on their own pages, where there is room to say what they do; a **warning can be dismissed and a lapse cannot**, since grace exists so that an expiry does not interrupt anyone and an unclosable strip interrupts them for a fortnight — the dismissal is keyed to what it said, so a *different* problem comes back; and it **never blocks the page**, so an engine too old to have the route leaves the console exactly as it was. Seven jsdom tests assert those rules, and the colours were measured rather than eyeballed: every part of both tones is at or above 4.5:1 in both themes (light 5.15–8.16, dark 6.15–8.8). **A bug found while building it, and the most consequential one in this stage.** `Install` compared the licence's binding against `host.MachineID()` — but on macOS and Windows the engine runs *in the VM*, whose machine id is a different number from the host's, and a licence is bound to the host's because that is where `fox license` runs. Every such install would therefore have sat permanently in “this licence was activated on a different machine”: a warning on every command, a permanent amber banner, and that sentence recorded in the security log as the reason a perfectly good licence was unlocking. The binding is now compared only where it means something — the machine a person sits at — and is *unknown* inside the VM, which the code already handles everywhere as “not a mismatch”. `FOX_IN_GUEST` was checked on the live VM first and does reach the daemons `fox start` spawns, which is what makes it answerable from a long-running server. **One more thing tightened on the way:** `State`’s zero value was `Active`, because it was written first in the obvious order — so a `Status` nobody had evaluated read as a working licence, which is exactly what an API returns before `Install` has run. `Invalid` is the zero value now. **Anchors are the first feature gated behind it**, and the line it draws is one sentence: **making anchors is paid, checking them is free.** `branch.Checkpoint` holds the gate, rather than only the CLI and the route — it is the one function that writes an anchor, the scheduler calls it directly, and anything added later would otherwise have to remember. `fox blackbox checkpoint` and `POST …/ledger/checkpoint` refuse before doing any work, so the message names the feature and the way out instead of surfacing as a failure; the REST refusal is **403, not 404 or 500**, because nothing is hidden (the edition and the licensed features are already on `/api/status`), a 404 would leave a console unable to tell “not available to you” from “your engine is too old”, and nothing failed. **Three things deliberately stay free,** and each is asserted by a test so it cannot drift. `fox blackbox integrity`, `fox-verify` and `blackbox anchor-key`: an install that anchored before upgrading still holds anchors, and taking away its ability to check them would punish a customer for the version they were on — and an independent verifier you have to buy is close to a contradiction, since what makes it worth anything is that it needs no trust in us. **The security log's own anchors** (`fox audit verify`, `branch.CheckpointSecurityLog`): that is the accountability trail, it now records the licence itself, and the rule the editions follow is that safety stays free — `StartCheckpointer` does both jobs in one loop, so the gate is inside the loop and says once, not every tick, why it is not anchoring branches. And **everything already written keeps verifying. The failure mode this change had to avoid** is an install that upgrades, silently stops anchoring, and looks broken: `blackbox integrity` now adds a note saying the unanchored rows will stay that way and that the chain is still checked in full, `fox check` gained a `blackbox anchors` line, and the console shows the checkpoint button *locked rather than missing* with a line saying verifying and exporting are unaffected — a control that vanished reads as a fault, and a hidden feature sells nothing. The console assumes the button works until `/api/status` says otherwise, so an engine too old to report features is not disabled by a newer console. **One wording bug found by running it:** the feature described itself as “signed Blackbox anchors and independent verification”, which this change makes false — it is “writing signed Blackbox anchors” now, and the refusals still tell a missing build from a missing licence, because one is a download and the other is a purchase. **The remaining five features followed**, on the same pattern and with the same line: **doing is paid, reading is free.** Gated: writing a Blackbox policy rule (`add` , `remove` , `block` , `warn` , `enable` , `disable`), impact analysis, asking for and applying a change request, Blackbox export, and ETL pipelines. Free, and each asserted by a test so it cannot drift: reading the rules, checking a statement against them and the evaluations log — a rule written before an install changed edition keeps enforcing, and being unable to read the rule that just blocked you would be worse than useless; the two default guardrails, which are enforced in SQL and are not rules at all; listing, showing and *rejecting* a change request, so one left open when an install changed edition can be closed rather than stranded; the plain `blackbox diff` , which reads two records; the `/admins` routes, which are access control and only share a file with the rule engine; and `fox import`, because charging for the way in is a bad trade for both sides. **Three judgements worth naming. Export is gated and free verification still works,** because `fox-verify` reads the live database with `--dsn` — had the export been the only way to produce the file it checks, gating it would have broken the free verifier by the back door. **Pipelines are gated whole, listing included,** because unlike the record or a policy rule there is nothing there that the paid feature did not create. And **the MCP tool list hides what it cannot serve**, which is the one surface where hiding beats showing locked: that list is read by a model, which would otherwise plan around a tool, call it and have to recover — `tools/call` still refuses by name, for a client working from a list it cached. **The gates sit on the functions that do the work** (`AddPolicyRule`, `Impact`, `ApplyRequest`, `ExportLedger`, `RunPipeline`…) rather than only at the edges, so a new caller cannot use a paid feature by not knowing about it; the CLI and the routes keep their own check only so the refusal can name the feature and the way out. One existing test had to move to the Enterprise build as a result — `TestApplyRequestRefusesNothing` — because the licence is now checked before the request is looked at, which is the right order. **These five are gated in**

place, not moved into `enterprise/`: the code is still compiled into both binaries, so the entitlement is the whole of the gate (a test asserts a Standard build refuses even when handed a licence that says yes to everything). Moving them behind the build tag, with the SQL schema split that goes with it, is now done. **All five are split**: `enterprise/{pipeline,impact,promote,anchor,policy}`, about 950 lines of engine, and `go tool nm` finds none of it in the Standard binary. **The line is not the file**, which is why each was a split rather than a move: `ImpactReport` is embedded in the free agent-provenance record and `BranchParent` feeds the free `blackbox diff`; a change request's entry type, encoding and rendering are what listing and *rejecting* one need; `StartCheckpointer` anchors the security log and the branch ledgers in one loop, so the loop stays and dispatches; and `IsAdmin` lives in `policy.go`, where `internal/access` resolves authorization through it — moving that file wholesale would have taken authorization out of the Standard build. **Twelve primitives** are what the paid edition needs from `internal/branch` (ExecSQL, ExecScript, QuerySQL, LedgerQuery, LedgerRows, LedgerV2Tables, PolicyQuery, ResolveBranch, CommonPrefix, QuoteIdent, QuoteLiteral, QuoteLiteralOrNull, SetGuard, TruthyEnv, Progress.Report) — all wrappers, with the unexported originals and every existing caller untouched. **CommonPrefix is shared rather than copied, and that is not tidiness:** a four-line reimplementation written during the split was wrong, comparing a field called Hash instead of the entry id and `ledger.RowHash` — two answers to “where did these branches split” would eventually disagree, and one of them decides what gets applied to main. **The gating tests became a test of the seam:** every dispatcher is checked to reach the engine that registered with it and to refuse rather than panic when none did, which is exactly what a Standard binary is. And one test was found to have been quietly broken by the earlier gating — `TestBuildRequestChecksNames` asserted only `err != nil`, so once the licence was checked first it passed on the licence error while appearing to test name validation. **The SQL schema split is done too.** `ledger_v2.sql` divided at one line into `ledger_ext.sql` (free: richer *recording*) and `checkpoints.sql` (paid: *proof*), and `policy.sql` and `impact.sql` moved whole. All three live in `enterprise/schema`, embedded there, so the Standard binary contains not one byte of them — checked directly: `bb.ledger_checkpoints`, `bb.policy_rules` and `bb.blast_radius` appear nowhere in its strings. They are installed **per feature, not all-or-nothing**: a licence covering anchors but not the rule engine creates the checkpoint table and not `bb.policy_rules`, the same line the commands draw. **Two of the plan's rows were wrong against decisions taken since it was written.** `provenance.sql` was listed as Enterprise, but agent provenance is free — the MCP server records a session for every change an agent makes, `fox blackbox sessions` reads them back, and there is no licence that could unlock it, since provenance is not a feature `internal/edition` knows about. And the plan did not say what a Standard install should do when the rule engine's tables are absent: the existing message said "run `fox blackbox upgrade`", which on that install **cannot fix it** — a licensed boundary reading as a broken install, which is the failure this whole stage has been avoiding. The readers now answer truthfully per edition: no rule engine is *no rules*, and the two default guardrails are unaffected and are not rules. **And that was not the only one.** The same message appeared a second time, in free code and on a command a Standard user actually runs: `fox blackbox integrity` noted "ledger checkpoints are not installed — run `fox ledger upgrade`" on every install without them, so a fresh Standard install printed that *and* the correct note saying they will never arrive — two contradictory sentences, one of them pointing at a command that cannot install what it names. Both sites now go through one function, `branch.UpgradeHint`, which returns nothing when the install cannot act on it; a test walks every feature and fails if any unlicensed one is told to upgrade, and it was checked by removing the guard and watching it fail. `fox policy list` also says which case it is in rather than a bare “No policy rules”. The rule underneath: **a remedy nobody can act on is worse than no remedy**, because a licensed boundary then reads as a broken install and the next step is a support ticket. **The invariants came through intact.** “Nothing added later may redefine what wrote a chain” is one list, `ledger.ForbiddenInAdditiveSchema`, applied to the free half here and the paid half in `enterprise/schema` — two packages under two licences that cannot share a test helper, and so could otherwise drift into enforcing different rules about the same base. The trigger arithmetic followed the files: the raising guard the count allowed for turned out to be in `checkpoints.sql`, so each file now counts its own. And `checkpoints.sql` reuses `bb.deny_ext_change`, which `ledger_ext.sql` defines — a dependency across the licence boundary that is now asserted, because nothing else would notice the install order being changed. **The boundary test earned its keep again:** it caught `checkpoints.sql` carrying the commercial header while still in `internal/ledger`, which is what prompted moving all three rather than relicensing files in place. **Stage 3 has begun: the Realtime foundation.** Everything that has to be right before a decoder exists, and nothing that needs one. **The opt-in is a file and the default is off** — a Standard install should not pay extra WAL volume for a feature it does not have, and an upgrade must never restart somebody's database on its own. `fox realtime setup` is the one way in, it prints what it will do and changes nothing without `--yes`, and it refuses after a failover (naming `fox ha failback`) because then `main` *is* the standby and restarting it is not a 15-second blip. **The container arguments are provably additive:** `postgresArgs` was extracted so the feed's settings have one place to live, and a test writes out the primary's five settings in order and fails if they change — with the feed off a primary starts exactly as it always did and a branch gets no command line at all. `wal_level` is named once rather than overridden by a repeated `-c`: Postgres does take the last, but a setting that decides whether the WAL can be decoded is not a thing to leave resting on precedence. The standby gets whatever the primary gets, or a failover would silently end the feed for good. **The budget is the design:** `max_slot_wal_keep_size` (1GB, tunable) is what makes durable replay safe on a pool shared with `main` — past it Postgres invalidates an abandoned slot instead of holding WAL forever, so a closed laptop costs a resync rather than a read-only database. Slots are swept on `fox up`, listed by `fox realtime slots` with the WAL each holds, and `canSuspend` gained a third probe that asks *Postgres* — the reaper runs in the gateway and a decoder runs in the control plane, so process-local state would suspend a branch out from under its subscribers. **The preflight is the safety core**, and a pure function over a struct so every refusal is table-tested without a database: row-level security is refused with **no override** (decoded WAL has had no policy evaluated against it, so every subscriber would receive every row — an override there is a data breach with an audit trail); anything `db_client` cannot already read is refused, whole table or single column; views, other schemas and the Blackbox are refused; and **a table with no primary key is refused with both ways out named**, because adding one to a publication that publishes updates makes every later `UPDATE` and `DELETE` on it fail — enabling a feed would break queries that work today. **Checked against a live database before any of it was written:** a `LOGIN REPLICATION` non-superuser can open `replication=database` through the image's existing `pg_hba`, because the `replication` keyword there matches only *physical* replication and a logical connection falls through to the ordinary line. So no `pg_hba` change is needed, which was the plan's fallback and the thing nobody wants to widen. **The decoder is built.** `jackc/pglogrepl` is the seventh direct dependency, and the three pieces are separate because only one of them needs a database. **The wire format is the part that is expensive to get wrong,** since every client that ever reads the feed depends on it: values are Postgres *text* as JSON strings and never JSON numbers (a bigint does not survive a JavaScript number, and money in a numeric survives it less), NULL and the empty string stay distinguishable, and an unchanged TOASTed column is **named in `unchanged` rather than written as null** — Postgres does not send a large column an update did not touch, and a decoder that wrote null there would silently overwrite a good value in the subscriber's copy, which is the quietest data loss this design could produce. `identity` is always present; `old` appears only when Postgres actually sent one; `changed` only under `REPLICA IDENTITY FULL`, where it can be computed truthfully rather than guessed. A delete under the default identity carries the key and nothing else, because that is all there is. **The fan-out drops a slow subscriber loudly.** One decoder feeds every subscriber of a branch, so blocking on a slow reader would stall the feed for everyone and let the slot's WAL grow behind it; an unbounded buffer moves the failure to memory, and skipping events leaves a subscriber running with silent holes nobody downstream can detect. So: 256 events of slack, then that stream ends with a reason it can act on. **The position is acknowledged on commit, never on receipt** — acknowledge early and a subscriber that drops at the wrong moment returns to find the feed moved on without it, with no error anywhere. Tested under `-race`, including a concurrent close-while-publishing case that *caught a real send-on-closed-channel panic* in the first version of the hub; the fix was verified by reinstating the bug and watching the test fail. **And a hole this stage's own earlier work had opened:** `govulncheck ./...` runs without the build tag, so nothing under `enterprise/` had been scanned since the paid code moved there — not its dependencies either. CI now scans both builds. **The feed is wired end to end.** `GET /api/branches/{name}/realtime` streams it, mounted on the *outer* mux because a branch-scoped API key has to reach it and `Authn` refuses those outright — a more specific pattern beats the `/api/` gate, the same trick `GET /api/openapi.yaml` already relies on. The risk of that approach is forgetting a check the bypassed middleware would have made, so the handler writes all four out in order: authenticate (including the narrow scoped-key exception), authorize with **404 for a branch the caller may**

not reach so names are not confirmed, then the licence, then whether the engine is set up at all. The licence is checked *after* authorization, so an unlicensed install cannot be used to ask which branches exist. **`UserForBranchStream` is the exception, and it is narrow:** a scoped key authenticates only when its scope equals the branch in the path. Safe because the reach shrinks rather than grows — that key already opens all SQL on that branch through the Gateway as `db_client`, so a read-only stream of tables somebody with Manage explicitly enabled is a strict subset. `Authn` still refuses it everywhere else, which is asserted. **The decoder's lifetime is the hub's:** started on the first subscriber, stopped after the last, because a branch nobody is listening to should not hold a replication slot open. A supervisor reconnects across ordinary failures — the slot is what makes that resume rather than skip — and gives up after ten, telling subscribers rather than leaving a client retrying into a wall. A slot Postgres invalidated for exceeding the WAL budget is reported as `resync`, never as silence. **The decode role is `db_realtime`:** LOGIN REPLICATION, no table grants, and deliberately *not* a member of `db_client` — it can decode the log and cannot run a query. `fox realtime enable` asks the catalog for every fact the preflight needs **in one query**, so the answers describe the same moment; a table that gained RLS between two queries would otherwise pass a check that was true a second ago. The publication is never `FOR ALL TABLES`. **The console page watches rather than configures:** choosing tables has refusals attached, and a button would have to reproduce every one. The frame splitter is extracted and unit-tested — including a frame split across reads, which is what a chunked response does — while `consumeSSE` is left alone on purpose: it throws when a stream ends without a result, which is right for a migration that has to finish and wrong for a feed that runs until you close it. **And the suite that proves it against a real database.** Everything before this was unit-tested; none of it had decoded an actual write-ahead log. `make integration-realtime` runs in the throwaway VM and is in the nightly matrix. **It mints its own licence**, because the signing key is deliberately not in the repository or in CI and a suite needing the real one could never run: it generates a throwaway key, pins the public half into the binary under test with `-ldflags -X` exactly as a test build is given the release key, and issues a licence against it. The key lives and dies with the run. **The assertions that matter are the ones only a live database can make.** That the no-primary-key refusal prevents the breakage it exists for — the table is refused, and `UPDATE` on it still works afterwards, which is the regression the whole refusal is for. That an unchanged TOASTed column really does arrive *unsent*: a 12 kB value is written, an update touches a different column, and the event has to name it in `unchanged` and leave it out of `new`. That an empty string and a NULL survive as different things. That a bigint arrives as a string. That a branch-scoped key streams its own branch, gets **404 rather than 403** for another, and is still refused on every other `/api` route. And that teardown leaves the database no worse off than it found it — `UPDATE` still works on a formerly-streamed table and on the one that was set to `REPLICA IDENTITY FULL`. **It found three bugs in the feature, none of which a unit test could reach.** `RealtimeDSN` used the container's *name* as the host, which does not resolve from the control plane — it runs as a process on the VM, not in docker — and it also got `main` wrong after a failover, when `main` *is* the promoted standby; it uses `BackendAddr` now, which knows both. **`db_realtime` was never created:** `EnsureRealtimeRole` was written and called from nowhere, so no decoder could ever authenticate and the first live run produced no events at all. It is created by `realtime setup` and again before each connection, because a branch is a clone of main and one taken before setup would not have it. And the one that matters most: **`--events` was publication-wide while the API presented it as per-table.** Postgres's `publish` option belongs to the publication, so the first table anybody enabled silently decided which events every later table would get — a table enabled with `--events=insert` made another table's updates and deletes disappear, with nothing anywhere to say why. There is one publication per event set now, the decoder subscribes to all of them, and `disable` finds whichever holds the table. **And five bugs in the suite itself, two of them worse than a plain failure.** One assertion passed when *no events arrived at all*, because an empty result matched an empty expectation; it now also says how many changes it saw. Another failed against correct code: the TOAST test wrote `repeat('x', 12000)`, which compresses to 147 bytes and is kept inline, so Postgres sent it with the update like any ordinary column — the unchanged-TOAST path only exists for values stored out-of-line. The column is `STORAGE EXTERNAL` with incompressible data now, and the suite asserts the value really is out-of-line so a future failure says which of the two things went wrong. The suite also was not re-runnable, hung on a stream the server holds open for an hour by design, and — pleasingly — had its own cleanup refused by the Blackbox guardrail that blocks `DROP TABLE`, which is a free feature working exactly as advertised. It asks for the override now, the same way a person would. **Then replay and resync were added, and found**

four more. `SweepRealtimeSlots` dropped every idle slot, and `Up()` runs on every engine start — so a subscriber that disconnected and reconnected across a restart lost its position entirely and resumed from “now”, with a silent gap where the missed changes should have been. That removed the one feature durable replay was chosen for, invisibly. `max_slot_wal_keep_size` already bounds what an idle slot can cost, so only slots Postgres has marked `lost` are swept now — and the old assertion had been pinning the bug, so it asserts the opposite. **`teardown` refused correctly and said nothing useful:** Postgres will not start with `wal_level` below logical while a logical slot exists, so refusing is right — but it reported “could not clean up on: main/fox_rt_main_stream” and left the feed on. It now says why, distinguishes a branch that is not running from a subscriber still attached, and names the way out. **A subscriber could attach after the decoder had already given up** and wait an hour in silence: the route starts a decoder and then subscribes, so one that failed at once closed the hub before anybody was on it. The hub remembers that it closed and why, and `Subscribe` hands back a finished stream carrying the reason. **And the one that mattered most: an invalidated slot was not recognised.** `isSlotLost` matched the message “can no longer get changes from replication slot”; Postgres says “can no longer *access*”. So the budget worked exactly as designed, the slot was invalidated, and the decoder treated it as a passing failure — ten retries over nearly a minute, then “could not be read after repeated attempts” instead of the resync that tells a subscriber to refetch. A client retrying into a wall with no idea it has to start again is the one outcome this design promises not to produce, and it had been staked on prose. It matches **SQLSTATE 55000** now, with the wording only narrowing it, and two unit tests pin both directions — because mistaking a blink for a lost slot would throw away a position that could have been resumed from. **Found by reading the engine’s own log rather than guessing a fifth time. Two consecutive clean runs: 46 passed, 0 failed. Writing it found four things the code was right about and the script was wrong about:** `go run` cannot work from outside the module, which is what minting the key in a temp directory did; API keys are `key_` and `apikey create` takes an email; and a branch-scoped key has no CLI at all — it comes from the Agent API, which is itself the point, since it is a credential for a program rather than a person. **Still to come:** and `internal/ledger`’s Go is deliberately untouched — it is the anchor *format*, which the free `fox-verify` reads. **The engine is now given the licence.** On macOS and Windows the engine runs in the VM with a state directory of its own, so a licence activated on the host is not the file the engine opens — which was the worst of the three possible outcomes, because `fox license show` would call it active and it would unlock nothing. `host.PushState` (`internal/host/gueststate*.go`) lands a copy there, using the transport the updater already uses to stage a binary in; on Linux the host is the engine and the copy is nothing. Four decisions in it. **The path is resolved by the guest's own shell, not computed here:** `$HOME` there belongs to whichever user the engine runs as, and on Lima that is the guest user — `/root/.fox` does not exist, so a path worked out on the host would have been the wrong one. **The content arrives on stdin** rather than in the command, so nothing is quoted and a licence never appears in the guest's process list; it is written to a temporary name and renamed, 0600 in a 0700 directory, the same rule `internal/secrets` follows on this side. **It will not create `~/.fox` where a retired state directory is still waiting:** `brand.resolveStateDir` only moves the old directory while the new one does not exist, so doing that would have stranded that install's accounts, keys and anchors — handing the engine a licence must not cost it its data. The old directory is written to instead, and the engine moves it on its next run with the licence inside. **And `fox start` brings the VM up before**

copying, not after: the entitlement is read once, when the engine starts, so a copy made afterwards would be a run too late — which is what makes the notice `activate` prints when the VM is down ("`fox start` passes it on") true rather than reassuring. `activate` and `rebind` push, `remove` takes the copy away too, and all three report a stopped VM rather than failing: the licence is installed on this machine either way, and a licence is never a reason a database fails to come up. Verified against the running VM rather than argued: a file pushed from the host arrived at `/home/<user>.guest/.fox` as `-rw-------` owned by the guest user, a build of this branch staged into the VM reported *that exact path* back in `fox check` when the file was there, and said the licence was not needed again once `RemoveState` had taken it away. The shell the scripts generate is unit-tested by running it against a throwaway home, including the retired-directory case. **A bug that only running it found:** a licence's fingerprint is signed, so `rebind` cannot change it — the first version wrote a new value beside the licence and then compared against the signed one, so it reported success and moved nothing. The binding that is compared now lives beside the licence (`bound_to`), starting as the issued one and moved by `rebind`; the licence still records which machine it was issued for, which is what a portal reconciles against. That a user could edit that file is not an objection, because the lock is soft by decision and it was never the thing stopping anybody — it is the record, not the lock. `activate` binds to what the licence was issued for rather than to this machine, so activating one meant for another says so instead of silently adopting it. **Three decisions worth knowing, taken deliberately. The machine lock is soft:** a fingerprint that does not match warns and keeps working, because offline enforcement is unenforceable anyway — the binary is source-available — and a hard lock would only ever fire on a rebuilt VM or a replaced disk. The cloud plan makes per-machine meaningless in any case: one VM per customer instance means a new machine id on every provision. **A lapsed licence stops new paid work and nothing else:** it never stops a database, and never stops reading or verifying something already recorded. If the verifiability of a tamper-evident record depended on a subscription being current it would not be evidence, and a billing event must not become an outage. **The signing key is not in CI,** unlike the release key: that one must be, because every tag is signed there, and its compromise is loud — a forged release gets noticed. A leaked licence key mints Enterprise silently, and rotating it invalidates every licence in the field, because the public half is compiled into every binary. **The host fingerprint** (`internal/host/machineid.go`) is

`IOPlatformUUID` on macOS, `MachineGuid` on Windows, and `/etc/machine-id` on Linux with `/var/lib/dbus/machine-id` and the DMI product UUID behind it. Four properties, each a decision: it is **read on the host and never in the VM**, because recreating the Lima or WSL VM is an ordinary repair step and fingerprinting the guest would make a routine fix look like licence evasion; it is **hashed with a domain separator** and only the hash ever leaves the machine, so it cannot be lined up against anything else that hashes the same operating-system identifier; it is **allowed to fail**, and an empty value means unknown rather than wrong; and it identifies a machine rather than a person. A container inherits its image's machine-id, so every container from one image reports the same fingerprint — which is a reason the lock is soft rather than a bug, since no reading of a machine id is meaningful under immutable infrastructure. **Where this is going** (decided 5 Oct 2026): a portal will sell Enterprise and activate a machine against an account. It will issue a **short-lived signed lease** rather than being asked live on every start — a database that needs permission before serving queries makes the vendor's portal a dependency of the customer's uptime, and air-gapped sites could not reach it at all. The engine works offline against the lease and renews in the background; an unreachable portal with a valid lease is silent, and revocation is simply declining to renew. Nothing in the licence format has to change for that: a lease is a licence with a short `not_after`, which is why the payload was worth settling before anything consumed it. **Air-gapped customers get a long licence issued by hand**, so the offline path stays a supported choice rather than a workaround. **The known limit:** the rebind counter would live in `~/.fox`, which `fox uninstall` removes, so it is a deterrent rather than a proof. It is therefore recorded for the audit trail and *not* enforced locally: the portal counts machines per account, which is the only place that can.

ADDED IN

`feat/licensing`.

Test matrix

Every automated test and the feature it covers. Run: `make test` (unit, host) · `make integration` (Lima VM) · Pester via windows-ci.yml.

Test file	Tests	Covers
`cmd/fox/main_test.go`	TestAddrFlag, TestDurFlag, TestRestoreArg, TestUsageDescribesRevertAndFailback	B1, H7
`internal/branch/ha_test.go`	TestSuspendRefusal, TestHAGuard, TestPrimaryPointerGuards, TestLSNAtLeast, TestParseControlCheckpoint	C3, D4
`internal/branch/import_errors_test.go`	TestPsqlErrorCollector, TestStreamJSONDocs	I1, I5, I6
`internal/auth/auth_test.go`	TestPasswordLogin, TestAPIKeys, TestVerifyKeyConcurrent, TestSessions, TestScopedKeys, TestScopeColumnAddedToExistingStore, TestConcurrentFirstOpen	G1–G4, K5
`internal/branch/agent_access_test.go` , `internal/proxy/scope_test.go`	TestAgentRolePassword, TestAgentGatewayDSN, TestGatewayHostPort, TestScopeAllows, TestLedgerOptionsLeavesAgentSessionToTheBranch	K5, K6, H16
`internal/branch/branch_test.go`	TestParsePublishedPort, TestDSN, TestAgentBranchName, TestContainerName	C2, C4, K5
`internal/branch/branch_before_test.go`	TestParseEntryJSON, TestRecoveryTarget, TestParseBackups, TestPickBaseBackup, TestWalArchived, TestValidateBeforeName, TestEntryQuery, TestRestoreBeforeScript	H13
`internal/branch/ledger_entries_test.go`	TestEntriesQuery, TestFormatLedgerEntries	H13
`internal/ledger/impact_test.go` , `internal/branch/impact_test.go`	TestSchemaImpact, TestParseTarget, TestTargetDestructive, TestScoreImpact, TestBranchParentParsing, TestFormatImpact, TestBuildDiff	H17, H18
`internal/ledger/provenance_test.go` , `internal/branch/provenance_test.go`	TestSchemaProvenance, TestValidProvenanceID, TestNewSessionID, TestCallHash, TestProvenanceSQL, TestClassifyExecError, TestFormatAgentSessions	H16
`internal/ledger/statement_test.go`	TestSchemaStatementMatching	H5, H15, H4, H2
`internal/ledger/policy_test.go` , `internal/branch/policy_test.go`	TestSchemaPolicy, TestParsePolicyDetail, TestPolicyContractExample, TestCommandTag, TestValidRuleID, TestValidatePolicyRule, TestPolicyRuleSQL, TestFriendlyPolicyErr, TestFormatPolicy	H15
`internal/controlplane/blackbox_test.go` , `internal/mcp/mcp_test.go`	TestBlackboxToLedgerPath, TestBlackboxAlias, TestBlackboxToolAliases	H14
`internal/controlplane/openapi_test.go`	TestOpenAPIMatchesRoutes, TestOpenAPIAliasRefsResolve	K2
`internal/branch/security_test.go`	TestTruthyEnv, TestSuperuserSwitchesDefaultOff	K5, K6
`internal/auth/setup_test.go`	TestSignupClosedUnlessOpen, TestFirstAccountNeedsTheSetupToken, TestOnlyOneFirstAccount, TestOAuthRules, TestThrottle, TestLoginIsThrottled, TestAuthBodyIsCapped, TestStateCookieIsSecure, TestPipelineTargets	E3, G4, G5, J1

`internal/httpx/httpx_test.go`	TestIsLoopback, TestAllowedOrigin, TestCORSHeaders, TestLimitBodies	A5, E2, K1
`internal/branch/names_test.go`	TestCheckName, TestLifecycleRefusesBadNames, TestPublishedPublicly, TestPublishedPortsAreLoopback	C1, C4, K5
`internal/proxy/tls_required_test.go`	TestNeedsTLS, TestMaxConns	E2, F2
`internal/controlplane/safe_defaults_test.go`	TestCheckBranchName, TestPipelineSourceMustBeAConnectionString	K1, J1
`cmd/fox/listen_test.go`	TestServicesListenOnLoopbackByDefault	A5
`internal/access/access_test.go`	TestLevels, TestAdminIsCachedBriefly	K1, K5, F2
`internal/auth/accounts_test.go`	TestEmailsAreBareAndFitARoleName, TestChangePasswordEndsOtherSessions, TestDeleteAccount, TestBranchOwners	G2, K1
`internal/controlplane/authz_test.go`	TestAuthorizeBranchRoutes, TestImportHosts, TestImportSourceRule	K1, I11
`internal/branch/rolepass_test.go`	TestRolePasswordsAreDistinct, TestNoPasswordOnCommandLines	E1, F4
`internal/mcp/access_test.go`	TestMCPAppliesTheAccessRule	K6
`internal/ledger/sign_test.go`	TestAnchorSignatures, TestVerifySignaturesSequence	H10, H11
`internal/auth/audit_test.go`	TestSecurityLogChains	G6
`internal/branch/seclog_test.go`	TestVerifySecurityLog	G6
`internal/update/update_test.go` (22 Sep)	TestResolveRequiresTheReleaseSignature, TestNoReleaseKeyNoUpdate	A9
`internal/update/releasekey_test.go`	TestReleaseKeyIsSetAndTheSameEverywhere	A1, A9
`internal/branch/images_test.go` (22 Sep)	TestPickImage (no unpinned fallback), TestMinioImagesArePinnedOffDockerHub (digest), TestPostgresMajorIsInStepEverywhere (base digests in the Dockerfile, the release matrix and the distro build; wal-g checked)	M6, M4
`scripts/test_install_sh.sh`, `test_release_checksums.sh`, `test_check_release_assets.sh` (22 Sep)	a signed release installs; another key's, a replaced binary with its checksum, an unsigned one are refused; LibreSSL warns and FOX_REQUIRE_SIGNATURE refuses; published signatures match the merged SHA256SUMS, even after a race; an unsigned release fails the asset check	A1, M4
`internal/branch/target_test.go`	TestParseTargetURL, TestTargetValidate, TestTargetEnvironment, TestSaveTarget, TestBackupSchedule	D1, D2
`internal/ledger/integrity_test.go`, `internal/ledger/ledger_v2_test.go`	TestRowHashMatchesSQLFormula, TestFormatAt, TestMerkleRoot, TestVerifyIntactAndUnanchoredTail, TestVerifyDetectsTampering (6 cases), TestVerifyDetectsAnchorGap, TestCheckChain, TestBuildAnchorRejectsBadRanges, TestAnchorFiles, TestDecodeRowJSON, TestQueries, TestSchemaV2IsAdditive, TestSchemaV2Safety	H9–H12
`internal/branch/pipeline_test.go`	TestSQLIdent, TestRenderSQL, TestTestQuery, TestJSONColumn	I6, I13, J1, J2
`internal/controlplane/server_test.go`	TestNameValidation	K1

`internal/daemon/daemon_test.go`	TestStopTerminatesProcess	B2
`internal/host/host_test.go`	TestIsELF, TestBundledLinuxBinaryPrefersCache, TestRefreshEngineBinaryDisabled, TestDownloadFile	A7
`internal/update/update_test.go`	TestParseVersion, TestCompareVersions, TestRequiredAssets, TestCandidates, TestChecksums, TestResolveAndDownload, TestReleaseListCache, TestBackgroundCheck, TestShouldCheck, TestInstallBinary, TestSwapExecutable, TestReplaceDirFromTarGz, TestReleaseWorkflowPublishesUpdateAssets	A8, A9, M4
`internal/host/host_update_test.go`, `cmd/fox/update_test.go`	TestUpdateFlow, TestUpdateChangesNothingUnlessReady (8 cases), TestUpdateInstallFailureRestartsOldEngine, TestParseUpdateArgs	A9
`scripts/test_feature_checklist.sh`	same feature ids in both documents; no duplicates; valid statuses; badge matches status	M1, M2
`scripts/test_check_release_assets.sh`	complete release; missing attachment; unlisted checksum; binary-mode and CRLF lines; no SHA256SUMS	M4
`internal/host/host_wsl_test.go`	TestResolveWSLDistro, TestWSLArgs, TestDecodeWSLList, TestWinPathToMnt, TestParseKernelRelease, TestImportLocalFile	A4, A6
`internal/proxy/proxy_test.go`	TestBuildParseStartupRoundtrip, TestReadStartupSkipsSSL, TestReadStartupUpgradesTLS, TestReadStartupHandlesCancel, TestSendError	F1, E2, F6, F7
`deploy/install.Tests.ps1`, `deploy/windows-install.Tests.ps1`	Arch/asset URLs, PATH, encoding, WSL independence, elevation/reboot probes, checksum verification, E2E (opt-in)	A2

Integration section (`scripts/integration_test.sh`)	**Assertions**	**Covers**
§1 Fresh start + auth	unauthenticated API → 401; main ready; gateway rejects bad key; accepts API key	B2, G2, G3, F2, K1
§1 Setup token (22 Sep)	start prints the token and `fox setup-token` the same one, stored 0600; providers reports the first account; sign-up without the token or with a wrong one is 403; the token is gone after the first account; sign-up is then closed, old token or not	E3, G4
§1c Safe defaults	control plane, Gateway and Agent API listen on 127.0.0.1 only; the object store is published on loopback only; a branch name or agent id that decodes to a path is 400 and main survives; a pipeline sourcing a server file is refused; a pipeline named after a branch it did not make refuses to run and the branch keeps its data	A5, C4, K1, K5, J1
§1d Accounts own their branches	alice's branch is not in bob's list and answers 404 to his read, query, suspend, delete and copy, and "no branch" through the Gateway; alice and the admin reach it; bob uses main but cannot checkpoint it; bob does not see or delete alice's agent, whose key is alice's; bob cannot import from this machine or a container name, nobody from a metadata address; the install secret no longer logs in as db_client and `pg.env` is 0600; bob cannot list accounts, changes his password (the old one stops working), and the admin deletes him (his key stops working); all of it is in the security log, which admins read over the API, anchors signed, verifies, and catches an edited event	K1, K5, F2, I11, E1, G2, G6
§11c Remote backup target (22 Sep)	a plain-HTTP target is refused without `--allow-http`; setting one reports the bucket's Object Lock and moves backups there; the first base backup lands there; the key is in 0600 files and not in main's arguments; WAL archives there; point-in-time restore reads from it; Blackbox and security-log anchors are copied there under Object Lock	D1, D2, D3, H10

	and one cannot be removed; `fox status` and the status API report the target and newest backup; going back to the local store works	
sdks (23 Sep)	each published client against a live engine: status, create and list a branch, SQL, the Blackbox's entries and its verify, suspend, resume, the error a call on a missing branch gives, delete; the TypeScript client also has to compile with its own tsconfig	K3, K4
ui (23 Sep)	the console in Chromium: sign in reaches the dashboard and lists main; a wrong password is refused; a branch made in the dashboard appears; the console runs SQL on it and the Blackbox page shows that change, attributed to the signed-in account; an API key is shown once and the account's password form is there	L2, L3, L4, L5, L6
§2 Branch isolation	branch sees its row; main doesn't have the table	C1, C2
§1b Hosting & login URLs	`fox vm` on Linux says there is no VM, its shell fails with a reason, an unknown subcommand is rejected; a password login sets a `Secure`, `SameSite=Lax` cookie; with no URLs set the GitHub authorize link calls back to `https://localhost:8080`	B1, G3, G5
§2b Branch from a branch	CLI `--from` copies the parent's data and leaves main alone; a missing parent is named; REST 404 for a missing parent and 201 naming the parent, with the copied row readable	C1, C2, K1
§5b Continuous import	replication starts into its branch; initial copy 1 of 1 tables; the rows are there; a source insert streams across; the branch survives a short-idle reaper while a plain branch is suspended and is still streaming; the list includes it; cutover returns `standalone` and it stops replicating; later source changes don't arrive and the data stays; a second cutover and a cutover of a branch that never replicated are refused 409	I8, C3, K1
§3 Time-travel / PITR	backup + `restore --to latest` → 3 rows	D1–D3
§3b PITR target selection	the API lists backups newest-first with the newest flagged; a restore says which backup it starts from and it is not the newest; a point before the newest backup reaches 3 rows and not 9; a point before every backup is refused, names the oldest and leaves no container; `--to latest` still reaches the end of the archive	D2, D3, K1
§4 Suspend / resume	exited after suspend; running after resume; suspending main refused, main keeps running	C2, C3
§5 Agent branch API	POST returns DSN; table created and read	K5
§6 HA	1 streaming replica; write via gateway after failover; the promoted standby archives WAL and its archiver is shipping; a backup taken from it (named as such) adds one to the stored backups; `backup list` works with main stopped; `fox status` names the container serving main; a restore reaches a write made after the failover; `ha enable|disable|failover` and suspending the standby refused after it; `fox up` recreates the promoted standby and leaves the old main stopped; `ha failback`: exit 0, main primary and not in recovery, pointer back on main, post-failover write kept, standby removed, gateway writes to main, WAL archiving resumed	C3, D4, F1
§7 Reaper regression	standby still running; idle branch exited	C3, C5
§8 Blackbox	DDL attributed human; DROP blocked; BLOCKED recorded; DELETE blocked; chain intact	H1–H4
§9 ETL pipeline	exit 0; raw tables; staged values; failing test → exit 1 with data kept	I4, J1, J2
§11 Import errors	a .sql dump with a failing statement exits 1, lists it, and the rest loaded; a clean dump exits 0; a JSON array with an invalid 2nd element exits 1 naming it; a valid array imports	I5, I6
§10 Schema fidelity	exact `userId` column; nested → jsonb; model query returns 1 row	I4, I6, I13

Integration section (`scripts/integration_ledger_v2.sh`)	Assertions	Covers
§0 Behaviour unchanged	guardrail error text; REST ledger and verify columns; `row_hash` formula (SQL vs Python); CLI verify intact	H2, H4, H6, K1
§1a Restore credentials	restore returns 3 rows; no hardcoded password printed; no `minioadmin` in the restore container	D3
§10 Data changes (22 Sep)	every user table has both triggers and making them recorded nothing; TRUNCATE blocked like DROP TABLE, rows kept, recorded BLOCKED with table and user; a non-admin override refused; an admin's truncates and is recorded; a person's UPDATE/DELETE not recorded; the chain verifies; an agent's DELETE and UPDATE recorded as the agent with the statement; a rolled-back change leaves nothing; an agent's TRUNCATE blocked; it cannot drop, disable or rename the triggers on its own table; CREATE TABLE AS gets them	H1, H4
§3 Signed anchors (22 Sep)	anchors carry the install key's id; `fox-verify --pubkey` accepts them and refuses one whose signature was altered; `fox blackbox integrity` reports the key	H10, H11, H12
§1b Override permission	ledger upgrade; non-admin refused with HINT; admin allowed; revoke restores the block; superuser override still works	H4, H8, F4
§1c Least privilege, attribution & MCP auth	agent DSN uses its own non-superuser role; DB usable; attributed to the agent; can't disable triggers or override; MCP list hides passwords; MCP run_sql as `db_client`; compat switch; `fox mcp` without a key exits 1 saying how to make one and naming the env var; an invalid key is refused; run_sql records the key's account; a scoped key's call lands in its own branch and not main, another branch is refused, and the wider tools are refused; a human claiming to be an agent is recorded as human and an agent claiming to be human as an agent	K5, K6, H3
§2 Blackbox 2.0 capture	installed at start; upgrade idempotent; capture row per ledger row with xid/LSN; task and call hash; hash verifies; override recorded; failing capture doesn't block DDL; kill switch (database-wide, and a superuser session); a client's own `SET bb.v2 = 'off'` — also as `db_admin` — does not skip capture; append-only; clients can't write; base chain intact	H9, H2
§3 Checkpoints & verifier	anchor written read-only in the published format; no-op when nothing new; out-of-sequence checkpoint refused; append-only; untampered branch INTACT via CLI, REST, MCP, fox-verify (live and export); forged edit with rewritten chain, deleted newest anchored entry and wiped ledger all TAMPERED; anchors outrank the database's checkpoint rows	H10–H12
§3b Scheduled checkpoints	with a 15s interval the control plane anchors new entries on its own	H10
§8 Impact & diff	impact on a table (views at depth 1 and 2 via the first, the foreign key from another table, its index, score), a column (only what uses it), quoted names, non-destructive, missing object, no target → `--object`, a view, depth limit, branch presence with parent, policy in the score, REST and MCP match the CLI, `execute_change` includes it; diff of a clone: parent, shared history, only-on-each-side, object both changed, `fox branch diff` matches, text output, self diff, REST on both paths, 404, MCP	H17, H18
§7 Agent provenance	sessions table installed, not client-writable; Agent API without provenance unchanged; with provenance: session / task / parent returned, recorded, and carried by the agent's own DDL; invalid id → 400, no branch; `execute_change`: applied with entry carrying agent / tool / session / task / parent / call hash, session recorded, no	H16, K5, K6

	superuser rights, BBX02 warning, BBX01 block with its BLOCKED entry, dry run, stable call hash, database error, invalid id; CLI and REST sessions; chain verifies	
§6 Policy gate & console values	a uuid, a uuid[], an interval, an inet and a bytea read as Postgres writes them, while numeric, jsonb, arrays and text are unchanged; built-ins installed as warn; fires after the guardrail; warn runs with BBX02 and contract JSON for all four built-ins; block refuses with BBX01, JSON ids, BLOCKED entry and evaluation surviving the rollback, HINT; non-admin `bb.policy_allow`, session `bb.v2=off` and `bb.allow_destructive` don't bypass; admin override runs and is recorded (evaluation + `override_used`); REST list / 200 / 403 / 404 / 409 / check / evaluations; CLI check + exit code; MCP `policy_check`; clients can't write rules; fail-safe; kill switch; custom / disabled rules; history; invalid pattern; built-in not removable; append-only; guardrail error unchanged; reinstall keeps changes; chain verifies	H15, H4, H9
§5b Blackbox page	`with=session` adds the session column and the default columns are unchanged; a table filter returns only that table's entries; table + risk narrows further; a date range covering today returns them and one ending in 2000 returns none	L5, K1
§5 Blackbox names	`fox blackbox verify|entries|integrity`, REST `…/blackbox/verify`, `…/blackbox` (same columns), `…/blackbox/entries`, 401 without a key, MCP listing and `blackbox_integrity` / `verify_blackbox` all match the original names	H14
§4 Branch before an entry	entry has a captured xid; its id is listed by `fox ledger entries`, REST and MCP; CLI branch-before by xid: rows before the change present, the change and later changes absent, ledger stops before the entry, no recovery settings left, reachable via the gateway, guardrail active, main untouched and its integrity result unchanged; refusals (existing name, BLOCKED, unknown, non-main source, bad id) leave nothing behind; REST 404 / 400 / 409 and success; MCP listed, refuses BLOCKED, succeeds by timestamp for an entry without capture; the branch deletes like any other	H13, K1, K6

Integration section (`scripts/integration_update.sh`)	Assertions	Covers
Setup	test releases 0.98.0 (installed) and 0.99.0 built and served by a local fake GitHub with a prerelease v1.0.0 and an incomplete v0.99.5; stack started on 0.98.0 with data on main and a branch	A8, A9
§1–§3 Notice	the notice is start's last line, once, with the banner still printed; `FOX_NO_UPDATE_CHECK=1` prints nothing; offline prints nothing and doesn't slow start	A8
§4–§6 Before updating	`--check` reports v0.99.0 and not the prerelease or incomplete release, changes nothing; without a terminal, or with stdin closed, `--yes` is required; a tampered download is refused ("Nothing was changed"), version and servers unchanged, no partial file left	A9
§7 Update	exits 0 and says done; Blackbox upgraded on the branch; `fox` is 0.99.0; the three servers restarted and run the new binary; main and branch containers not recreated; main and branch data unchanged; every Blackbox entry from before the update unchanged on main and the branch (hash of the entries), both chains intact, new Blackbox objects present; `secrets.json` and TLS certificates unchanged; previous engine kept; the gateway serves main with an API key made before the update; control plane answers	A9

§8 Afterwards	`--check` says up to date; `start` shows no notice	A8, A9

Not covered by any automated test: most MCP tools, OAuth against a real provider, REST pipeline handlers, SDKs, web UI, SCRAM, storage quotas, non-Mongo imports. `fox vm` is covered on Linux only (macOS and Windows were exercised by hand and compile-checked).

AREA N

Realtime change feed

N1 Realtime change feed

Working

WHAT IT DOES

Streams every insert, update and delete on chosen tables to a subscriber as it happens, with **replay**: a subscriber that disconnects reconnects with the position it last saw and receives everything it missed. Paid — an Enterprise build with a licence covering `realtime`; a Standard binary contains none of it, licence or no licence. Off until somebody asks: a database that nobody is streaming runs exactly as it always did.

HOW IT WORKS

Postgres's own logical decoding, read with `github.com/jackc/pglogrepl` (the `pgx` author's, pure Go) over the built-in `pgoutput` plugin. **The opt-in is a command, not an upgrade**: `fox realtime setup` writes a marker, says that `main` restarts and waits for `--yes`, and only then does `postgresArgs` add `wal_level=logical` and `max_slot_wal_keep_size` — with the marker off, a primary's arguments are byte-identical to the five it has always had and a branch gets none, which a test asserts. **The budget is the whole reason durable replay is safe here**: a replication slot nobody drains pins WAL for ever, and on a pool shared with `main` that is how a closed laptop takes a database read-only. Past `max_slot_wal_keep_size` (1GB by default, `FOX_REALTIME_WAL_KEEP`) Postgres invalidates the slot instead of holding more, and the subscriber is told to `resync` — a worse answer than replay and a far better one than a full disk. **One decoder per branch**, in the control-plane process, started on the first subscriber and stopped after the last; it wakes a suspended branch rather than failing against one. Named slots `fox_rt_<branch>_<subscription>`, swept by `fox up` only once Postgres has marked them lost. One publication per event set, because `publish` is a property of the publication rather than of a table in it. **The wire format** carries values as Postgres's text output in JSON strings (never numbers: `bigint` and `numeric` do not survive a JavaScript number), `identity` on every change, `old` only when Postgres sent one, `changed` only under `REPLICA IDENTITY FULL` where it can be computed truthfully, and `unchanged` for unchanged-TOAST columns — which are absent from `new`, and are the trap that silently overwrites a good value with null in a naive client. **Transport is SSE** on the outer mux, where a more specific pattern beats the `/api/` gate, so a branch-scoped key can reach it through `UserForBranchStream` while `Authn` stays untouched; the handler then repeats every check the bypassed middleware would have made, including answering `None` with **404 rather than 403** so a branch's name is not confirmed to somebody who may not use it. A connection is bounded (1h) and ends with a `resync`; a subscriber that fills its 256-event buffer is dropped loudly rather than stalling the decoder. **What it refuses**: a table with row-level security, with no override flag, because decoded WAL carries no policy evaluation and every subscriber would receive every row; a table with no primary key, because adding one to a publication that publishes updates makes every later `UPDATE` on it fail; anything outside `public`, the Blackbox's own tables, non-tables, and anything `db_client` may not already read. The reaper asks Postgres rather than its own memory whether a branch is being streamed (`probeRealtime`), because the reaper runs in the gateway and the decoder in the control plane; and the HA standby carries the same arguments, and is restarted by `setup` if it predates it, or a failover would promote a primary that cannot decode.

GETTING A TABLE READY

`fox realtime doctor` classifies every table on a branch — streaming, ready, needs changes, or cannot stream — and `fox realtime prepare <table>|--all` makes the changes, printing the exact SQL and changing nothing without `--yes`. **Why this exists:** `enable` was a gate rather than a path. Four of its six refusals were fixable and it made the user write the SQL, which for an application with a hundred tables is a project rather than a command. **The ladder matters more than the commands.** A table with no primary key used to mean `REPLICA IDENTITY FULL`, which writes the whole old row on every `UPDATE` — roughly double the WAL, for ever. Postgres will accept any unique, valid, non-partial index on `NOT NULL` columns as the replica identity at the same price as a key, with no schema change, so `prepare` looks for one first; then for an append-only table, which needs no identity at all; and only reaches `FULL` for a table that is genuinely keyless *and* updated. `--all` never applies a costly fix: it does the free ones and lists the rest for a person to decide one at a time, with the cost measured from `pg_stat_user_tables` rather than described. **Only application tables.** The old rule was “public only”, which refused the application’s own schemas and — because an extension may create tables anywhere, including in public — let an extension’s tables through. It is now any schema that is not `bb`, not a Postgres schema and not extension-owned, held both in the catalog query and again in Go over the schema name, so the rule protecting the Blackbox cannot be lost by a caller forgetting a field. Everything `prepare` runs is ordinary DDL and lands in the Blackbox, attributed — “turning on realtime granted SELECT on nine tables and set a replica identity on two” is a question an auditor asks and the answer already exists.

THE CLIENTS

`clients/typescript` and `clients/python` gain change-feed subscribers: `subscribe(dsn)` in both, plus `db.subscribe(branch)` for a program already holding an API key. Still dependency-free — `fetch` and Node’s own test runner on one side, `urllib` and `unittest` on the other. **Why this exists, and why it waited:** the plan held the SDKs back until the wire format had survived real use, and it changed once in that time — the `commit_lsn` correction — which would have been a breaking release had they shipped earlier. **What they carry is the knowledge a subscriber would otherwise have to rediscover, wrongly.** A column missing from `new` is *not* null: Postgres omits a large TOASTed value an update did not touch and names it in `unchanged`, so copying `new` over a stored row writes null across a value that never changed — and nothing reports it, because the feed did exactly what it promised. `applyChange` and `Change.apply` exist for that one trap. Resume moves only at a transaction boundary, never mid-transaction; `resync` drops the position rather than leaving a hole; a stream the server closes at its hourly limit is reconnected from the last transaction delivered; and values are strings, because a `bigint` survives neither a JavaScript number nor a float. **TLS is the caller's decision:** a default install serves a self-signed certificate, and neither client disables verification on anybody's behalf — Python takes `verify_tls=False`, TypeScript takes a `fetch` the caller has bound. The contract tests drive both against a live engine with a real realtime key, proving the wire format they were written against is the one the engine sends.

TRANSACTION FRAMING

`?transactions=1` puts a `begin` and a `commit` around each transaction's changes. **Why this exists:** an application moves money between two rows in one transaction; without frames a subscriber sees two independent changes and cannot tell they belong together, so anything derived from the feed passes through a state where one side moved and the other had not — a glitch in a cache, a wrong answer in a ledger or a search index. **The boundary is known in advance.** pgoutput announces a transaction's commit position at `BEGIN` (`FinalLSN`), so every change in it is stamped with the same `commit_lsn` and a subscriber can record the boundary before it applies anything. That also corrects a misnomer: `commit_lsn` held the individual record's position, which is a different number — resuming from one of those replays the whole transaction that contained it, losing nothing but duplicating. The record position is still there, as `lsn`, and every change now carries its `xid`. **Off by default**, because a client written against the unframed feed would otherwise start receiving event types it has no case for; one decoder serves every subscriber of a branch, so the frames are always produced and dropped per subscriber at the route. A value other than 1 or 0 is refused rather than read as 0 — a subscriber that asked for frames and silently did not get them would apply a transaction in pieces and never learn why. **An empty transaction produces no frame:** Postgres reports transactions that touched only unsubscribed tables, so `begin` is held until the first visible change and `commit` is sent only if a `begin` was. The console asks for frames and draws the boundary, with the rule on the commit rather than the begin because that list is newest-first.

WHAT STAYING WARM COSTS

`fox realtime activity [branch] [--window 24h]`, `GET /api/branches/{name}/realtime/activity`, and a panel in the console. **Why this exists:** the feed chose the warm model over a cold start, because a cold start defeats the point of realtime — so a subscribed branch is *never* suspended. That was the right trade and it is not a free one, and nothing in the product said for how long a branch had been warm or whether anything had used it. A forgotten tab and a busy application looked identical. **Measured, not estimated.** How long warm comes from the container's own `StartedAt`: it needs no new bookkeeping, survives the control plane restarting, and cannot drift from reality the way a timestamp we wrote down could. The work comes from `pg_stat_database`, sampled on a timer into the install's own store, because those counters are cumulative — one reading says nothing and two say everything. **Transactions, said as transactions.** Without `pg_stat_statements`, which is not loaded and costs a restart to load, that is what Postgres counts; calling them queries would be a word that is not true about a number somebody may bill from. **Five ways the subtraction could lie, each answered with a note rather than a figure:** fewer than two readings, a statistics reset between the ends, a counter that rewound, a branch suspended and resumed mid-period (measurable, but the rate is then averaged over time it was not running), and a zero-length period. **The bookmarks** are reported with the WAL each holds and how much room is left before Postgres drops it, which is the number that turns "a subscriber went away" into something to act on before the disk does it for you. **A cap** of 8 subscribers per branch (`FOX_REALTIME_MAX_SUBSCRIBERS`), refused with the count: without one a client that reconnects on error with a bug adds a stream per attempt until the process runs out of something. Readings are dropped when a branch is deleted, because a branch name can be reused and a new branch inheriting the old one's readings would report work it never did. The meter is the fourth scheduler in the control-plane process and shares that known limit — no leader election — which the report tolerates by subtracting the ends of a series rather than summing rows.

THE CONSOLE

A page rather than a viewer: **Tables** (what can stream, and what it would take), **Connecting an application** (a key and its connection string), and **Feed** (the old viewer, unchanged). **Why this exists:** the page used to say in its own comment that tables are chosen with `fox realtime enable` "because that is a decision with refusals attached — and a console button would have to reproduce every one of them". That was true of a gate; a verdict carries the exact statements and their cost, so a button reproduces nothing and runs what the verdict already says. **One button asks first, and it is the only one that should.** A free fix is a grant or an index-backed identity — running it is the whole decision. A costly one is `REPLICA IDENTITY FULL`, a bill rather than a step, so the dialog carries the figure measured from `pg_stat_user_tables` and cancelling is the default. **Shown once means shown once:** the key is hashed on the way in, so the response that carries the connection string is the only place it exists, and changing branch clears it from the screen — it is a secret, and after the change it is the wrong one. The minted DSN names the host the caller actually reached and the `sslmode` this install actually serves, because a connection string that says `127.0.0.1` to somebody on another machine is worse than none. Routes are on the inner `/api/` mux, so they inherit the gate the front door had to repeat, and `needFor` requires `Manage` for every `realtime/` tail by prefix — a route added later cannot default to `Use` by being forgotten. Reading verdicts is deliberately not among them: the console calls the same `/realtime/v1` endpoint an application does, so the two cannot drift.

THE FRONT DOOR

An application reaches the change feed at `/realtime/v1/branches/{name}/{hello,tables,stream}`, on the port the console already uses, with a connection string of its own shape: `fox-realtime://rtk_…@host:8080/branch?sslmode=require`. **A key kind that only streams.** Until now a subscriber used a branch-scoped gateway key, defended as a strict subset — that key already opened all SQL on the branch, so a read-only feed of explicitly enabled tables widened nothing. True, and a poor trade: an application that only wants to know when a row changed had to hold a credential that could read every table and drop them. `fox realtime key create <branch>` mints kind `realtime` (prefix `rtk_`), which the control plane refuses as every scoped key already is, **and which the Gateway refuses by kind** — that refusal is the whole claim, so it is asserted against a real psql connection rather than described. **The key is never in a URL.** It sits in the DSN’s userinfo, where a Postgres URL puts its password, and a client turns it into an `Authorization` header; a URL reaches access logs, browser history and `Referer`, so a key in one has to be treated as disclosed. `sslmode` carries libpq’s meanings because a default install serves a self-signed certificate, which is exactly `require`; `disable` has to be asked for by name. **Additive.** `GET /api/branches/{name}/realtime` is unchanged and a subscriber running today keeps working — this is a second door, and `kind` defaults to empty, which is what every key issued before it is and behaves exactly as it did. `hello` exists so a wrong DSN fails at start-up with a message instead of at 3am as silence. **Serving verdicts as JSON found two bugs the CLI had hidden.** `Assess` set `Status` in a `defer` on an unnamed result, so the write landed on a local the return value had already been copied from and all 229 verdicts went out as `"status":""` — invisible while only the CLI read them, since it renders the state itself. And the first version of this endpoint passed `Survey`’s output straight out, on a comment claiming system tables were already dropped; they are not — the CLI filtered them inline — so an application’s key was served the `bb` schema’s table names, row counts and column shapes. There is now one `realtime.Application()` that both surfaces call, returning the application’s tables and a `withheld` count, because two copies of a boundary drift and this one decides whether a subscriber can be offered the Blackbox.

SURVIVING A FAILOVER

A subscriber’s position lives in a replication slot on the primary, and slots are not copied to a standby — so until now a promotion lost every one of them. The feed degraded correctly rather than silently (subscribers received `resync`, which both published clients handle by dropping their position and refetching), but every subscriber re-read its whole table at the worst possible moment. PostgreSQL 17 added slots that follow a promotion, and they are now used: the decoder marks its slot with `ALTER_REPLICATION_SLOT … (FAILOVER)`, and both containers run with `sync_replication_slots` and `hot_standby_feedback` on. Phase 4 of the HA plan. **Four things, not one.** Slot synchronisation runs only when the slot is marked, both settings are on, the standby holds a physical slot on the primary (`primary_slot_name`, `fox_standby`, named on the standby’s own command line and created idempotently when it starts), *and* `primary_conninfo` names a database — and when one is missing it does not fail, it logs a line nobody reads and the slots are never copied. The last is the trap: `pg_basebackup -R` copies the parameters it connected with, and `-h host -U user` produce a conninfo with no database name at all, while the worker that reads the primary’s slots connects to a *database*. So the base backup passes `-d "host=… user=… dbname=…"`. **`synchronized_standby_slots` is deliberately not set** — it makes logical WAL senders *wait for* the slots it names, so a named standby that is not there stops decoding altogether. **Gated on the PostgreSQL major, which is the regression this nearly shipped.** An install created under PostgreSQL 16 keeps running 16 (`pgImage` follows the data, not the binary), `sync_replication_slots` does not exist there, and an unrecognised setting is fatal — `postgres -c sync_replication_slots=on` on 16 exits with `FATAL: unrecognized configuration parameter`, which inside docker is a database that never comes up. Checked against both images in the test VM. Everything here is behind `SlotSyncMajor`, including the catalog read behind `fox realtime slots`, which gets `failover` and `synced` through `to_jsonb` because naming them as columns is an error on 16. **And the base backup must not stream through that slot at all** — two reasons, one found by reading and one only by running it. `pg_basebackup -C -S` refuses a slot that already exists and then deletes the data directory it was building, and `fox ha enable` is the documented way to rebuild a standby. Creating the slot separately and passing only `-S` fixes that and introduces something worse, which the integration suite caught: a slot created with `immediately_reserve` is subject to `max_slot_wal_keep_size` from the moment it exists, so on a busy primary with a small budget it can be invalidated in the seconds before the backup’s WAL streamer connects — `could not send replication command "START_REPLICATION": can no longer access replication slot … invalidated due to "wal_removed"` — and then `ha enable` cannot build a standby *at all*: a resilience feature taking out the thing it was meant to protect. So the backup streams through no slot, exactly as it always did; the standby is told its slot on its own command line, where a setting outranks the `postgresql.auto.conf` that `-R` writes, and the slot reserves nothing until the standby connects. Nothing is lost by waiting — the WAL the standby needs next is already in its own `pg_wal`, put there by `-X stream`. A **failback** gets no slot: `main` is rebuilt as a temporary standby by `startContainer`, which names none, so one made there would reserve WAL nothing reads. **The same switch as the feed.** A physical slot holds WAL for a standby that may be gone, and with the feed off there is no `max_slot_wal_keep_size` to bound it — so it exists only while the feed is on and the major can synchronise it, a base backup on every other install is byte-for-byte the command it has always been, and `fox ha disable` drops it. **Degrades rather than refuses:** a slot that is not failover-aware still streams every change correctly, so a failed `ALTER` logs and carries on, and `fox realtime slots` says of each slot whether it is marked to survive a failover — a degradation nobody can see is the thing worth avoiding, not the degradation. **And the limit, which is PostgreSQL's rather than ours:** a slot is copied to the standby only *while it is being consumed*. PostgreSQL refuses to copy one whose position is behind the standby's catalog horizon — `Synchronization could lead to data loss, because the remote slot needs … catalog xmin N, but the standby has … catalog xmin N+1` — and the horizon of an otherwise idle database sits exactly one transaction ahead of a slot nobody is reading, so it never catches up; measured holding at N and N+1 for fifty seconds without moving. So **a subscriber that reads continuously keeps its place through a failover; one that read once and went away is told to resync, exactly as it did before.** That is why the CLI says "marked to survive" rather than "survives", and prints the reason once. It also took three runs of the integration suite to find, because `pg_replication_slots.synced` goes true long before the copy is usable — the copy is still `temporary` then, and a promotion drops a temporary copy. The condition that actually means a position will survive is a copy that is **not temporary and has a position of its own**, which is what the suite waits for. `synchronized_standby_slots` was tried against this and does not help, which is a second reason not to set it.

HOW TO USE

`fox realtime setup --yes` once · `fox realtime enable orders [--events insert,update,delete] [--replica-identity=full] [--branch b]` · `fox realtime tables` · `fox realtime slots main [--drop <name>]` · `fox realtime status` · `fox realtime teardown --yes`. Subscribe with `GET /api/branches/{name}/realtime`, resuming with `?since=<commit_lsn>`; the console's Realtime page watches it live. `setup`, `teardown`, `status` and `slots` are in the **free** build on purpose, so somebody who goes back to Standard can still see and undo what they turned on. `fox check` reports the feed, and warns on a slot nothing is draining.

HOW TO TEST

Unit: `enterprise/realtime/wire_test.go` (the format, including unchanged-TOAST and the delete that carries only a key), `decode_test.go`, `preflight_test.go` (every refusal, as a pure function over a struct), `hub_test.go` (overflow drops one subscriber and not the decoder), `run_test.go` (`isSlotLost`, both directions), `internal/branch/realtime_test.go` (slot and publication naming, the arguments with the feed off), `internal/proxy/suspend_test.go` (a streamed branch is not suspended, and a suspended one's idle clock is forgotten), `internal/auth` (`TestUserForBranchStream`), `web/src/sse.test.ts` (frame splitting). End to end: `scripts/integration_realtime.sh` (`make integration-realtime`, in the nightly matrix) — 13 sections against a real write-ahead log, covering the Standard boundary with a licence activated, every refusal and that `UPDATE` still works after one, the wire format under **both** replica-identity settings in one capture (a delete carrying the key alone, and the whole old row with a truthful `changed`), replay through `?since=`, the budget invalidating a slot, the reaper leaving a subscribed branch alone and resuming one for a new subscriber, `kill -9` on the control plane mid-stream, a continuous import decoded on the same branch, and backups and an HA failover and failback with the feed following `main` onto the promoted standby.

STATUS & LIMITS

Working, and proved against a live database rather than a mock. **The integration suite found fourteen bugs in the feature**, every one of them in a path no unit test reaches: the decoder's DSN named a container the control plane cannot resolve; the decode role was never created; `--events` was publication-wide while being presented per-table, so one table's choice silently removed another table's updates; every idle slot was swept on `fox up`, which removed the one thing durable replay exists for; a `teardown` that refused correctly explained nothing; a subscriber could attach after the decoder had given up; an invalidated slot was matched on prose Postgres does not use, so the budget doing its job looked like a transient fault; a suspended branch was not woken for a subscriber; and a branch resumed by anything but the gateway was suspended straight back. **Three of them failed silently** — the feature appearing to work while doing the wrong thing. **Limits, by decision.** The console and REST only this release: `clients/typescript` and `clients/python` get the feed once the wire format has survived real use, because a public API is worth committing to once. No per-subscriber row filters, which is why RLS is refused rather than filtered. A delete under the default replica identity carries the primary key and nothing else, so a client that needs the old row keeps its own copy or asks for `--replica-identity=full`. `TRUNCATE` emits one event per relation with no row data, and is blocked by the Blackbox guardrails by default anyway. `go.mod` is not build-tag aware, so `pglogrepl` appears in the AGPL module's manifest even though no Standard binary contains a byte of it: the testable guarantee is the binary, and `go tool nm` finds no `pglogrepl` symbol in `fox`. And an operator trap worth knowing: `max_slot_wal_keep_size` cannot be moved with `ALTER SYSTEM`, because the engine passes it on Postgres's command line and a command-line setting takes precedence — it is accepted and silently ignored. `FOX_REALTIME_WAL_KEEP` plus `fox realtime setup` is the way.

ADDED IN

`a5aa9a4` the foundation; `80c7a70` the decoder; `3c647d3` the stream and the console; `30b9831` the integration suite and the three bugs it found; `c0e825a` replay, resync and four more; `9d3812e` the shared licensed test build.

Known issues

Found while building this inventory. Recorded, not fixed — each fix changes existing behaviour and needs explicit approval. Struck-through items have been fixed.

1. ~~PITR restore broken on fresh installs (hardcoded `minioadmin`)~~ — **fixed** in Phase 1. (D3, H7)
2. ~~Destructive-DDL override ungated~~ — **fixed** in Phase 1 (superusers and `db_admin` only). Use of the override is now recorded (H9). (H4, H8)
3. ~~Agents and MCP run as superuser~~ — **fixed** in Phase 1 (own `agent-<id>` role; MCP as `db_client`). (K5, K6, H3)
4. ~~Integration §1 and §5 call http:// endpoints that now serve TLS~~ — **fixed** in `13734a7`.
5. ~~Integration "bad key" and "append-only" checks gave false failures~~ — **fixed** (test-only; both protections confirmed working in the VM).
6. ~~`fox branch suspend main` stops the primary and the gateway won't wake it~~ — **fixed**: suspend refuses main and the HA standby. (C3)
7. ~~`fox ha enable` after a failover deletes the live primary (`pg-standby`); post-failover `ha disable` routes main to a stopped container~~ — **fixed**: after a failover `ha enable|disable|failover` are refused, `fox up` brings the promoted standby back instead of the old main, and `fox ha failback` returns to main keeping every write. (D4)
8. ~~`backup` / `restore` always target `pg-main`, even after a failover; PITR always fetches the LATEST base backup~~ — **fixed**: `fox backup create` backs up whichever container serves main (and says when that is not `pg-main`), the listing is read from a throwaway container so it works with main stopped, and a restore starts from the newest base backup that precedes the target — a target no backup precedes is refused, naming the oldest. Verified on a real install: a restore to three weeks earlier chose the right backup and came up without the `fox` schema, which did not exist then. (D2, D3, H13)
9. ~~A promoted standby archived no WAL, so nothing written while main ran on it could be backed up or restored~~ — **fixed**: the standby runs with the primary's object-storage environment and `archive_mode=on`, which archives only once it is out of recovery. (D4)
10. ~~Ledger verify can't detect deletion of the newest rows or a wiped table~~ — **addressed** in Phase 3: checkpoint anchors kept outside the database catch it (H10–H12). Entries written since the last checkpoint are still only chain-checked. (H2)
11. ~~A release job could replace `SHA256SUMS` and drop the other job's entries (v0.8.1 lost `foxbyte-distro.tar.gz`, so the Windows installer skipped verifying it)~~ — **fixed** for future releases by the shared merge script. v0.8.1 itself needs `wsl-distro.yml` re-run for tag `v0.8.1` by a repository admin. (M4, M5)
12. ~~MinIO images are pulled from Docker Hub, where `minio/minio` and `minio/mc` no longer exist — the Windows distro release build and fresh installs failed~~ — **fixed**: pinned quay.io images, with cached old images still used. (D1, A4)
13. ~~The 2.0 capture kill switch `bb.v2 = 'off'` could be set by any client for its own session, skipping its capture rows~~ — **fixed**: it is now honoured only database-wide or in a superuser's session. (H9)
14. ~~The new-release notice never appeared against real GitHub: the check's `SHA256SUMS` request is throttled to 7–75 s, far past its budget~~ — **fixed**: the check is one request (the releases list) and its answer is remembered for 6 hours. (A8)
15. ~~Policy patterns match the whole query text a client sent (all statements of a multi-statement string, comments and literals included)~~ — **fixed**: rules and risk flags match the statement being run, without comments or literals, falling back to the whole query only when that statement can't be established. (H15, H5)
16. ~~A blocked command after a Blackbox write in the same transaction hung the client forever (the dblink connection recording the attempt waited on its own session's lock)~~ — **fixed**: the attempt goes unrecorded with a WARNING and the block stands. (H4, H15)
17. ~~`fox ledger revert --to` doesn't revert a branch~~ — **fixed**: its help and output now say it is a point-in-time restore of main; H13 branches from just before a change. (H7)
18. README documents ~~`fox branch diff` (didn't exist)~~ — **now added** (H18) — and ~~ledger UI filters by table/risk (not in the UI)~~ — **now added**: the Blackbox page filters by table, risk and date range, verifies the chain and expands a row to the statement and session. (L5)
19. ~~OpenAPI spec disagrees with the code (import fields, status codes, missing params/routes)~~ — **fixed**: the spec was corrected against the handlers, and `TestOpenAPIMatchesRoutes` fails the build if the two diverge again. (K2)
20. ~~`.sql` imports don't stop on error; JSON array imports stop silently at the first bad element~~ — **fixed**: both now fail, listing the failed statements or naming the invalid element. Web Import mentions SQLite (no loader). (I5, I6, I12)
21. ~~Continuous-import cutover is CLI-only~~ — **fixed**: `GET /api/replication`, `GET /api/branches/{name}/replication` and `POST …/replication/cutover`, with a **Continuous imports** panel on the Import page. (I8)
22. ~~The Gateway suspended a branch mid continuous import, so the import silently stopped about two minutes in~~ — **fixed**: found by watching the new panel — a subscription's apply worker is a background worker, so the branch reported zero client connections and looked idle. The reaper now leaves a replicating branch running. (I8, C3)
23. ~~Creating a branch from a non-`main` parent a second time failed: `zfs destroy` only removed the `main@for-<branch>` snapshot, leaving the parent's clone snapshot behind~~ — **fixed**: delete removes the clone's own origin snapshot too. (C1, C2)

24. ~~Agent cap and TTL reaper only count running containers; agent DSN only reachable inside the VM~~ — **fixed**: all three count suspended branches, and the DSN is Gateway-routed with a branch-scoped key, so it works from the host without widening what an agent may reach. (K5)
25. ~~OAuth defaults point at `http://localhost:8080` / `http://localhost:5173`~~ — **fixed**: the public URL defaults to `https://localhost:8080` — what the engine actually serves — and the web origin defaults to the public URL, so a default install's callbacks and post-login redirect land on the running console. Session cookies follow the origin: `Secure` when it is https, and `SameSite=None` only when the console is on a different origin than the API. (G5, G3)
26. ~~`fox vm` is listed as a command but not implemented~~ — **fixed**: `fox vm status` reports the engine VM's state and size and `fox vm shell` opens a shell in it (Lima on macOS, WSL on Windows); on Linux both say there is no VM. (B1)
27. ~~Windows installer never downloads the prebuilt distro; its preloaded image tag doesn't match the engine's image name; setup summary omits the API key~~ — **fixed**: the installer prefers the verified distro image, the build and the loader use the engine's image name, and the summary prints the connection string with the key. (A2, A4, M5)
28. ~~Web Guide/Docs pages are stale in places~~ — **fixed**: both were read against the code and refreshed (https, `sslmode=require`, the full REST surface, MCP). (L7)
29. ~~The SQL console could never override the guardrail — not even for an admin — so `DROP TABLE` failed with the same message however it was retried~~ — **fixed**: the console logged every user in as the shared `db_client` role, which is never in `db_admin`, and each run is a new connection so a typed `SET` was lost. It now logs in as the user, and the override travels with the run. (L4, H8)
30. ~~The integration suites run against the install's own `main` and leave tables there (`pit`, `led_a`, `repl`, `rlsdemo`, `t1`, …), which then show as user tables in the console~~ — **fixed**: worse than tables, they deleted main's Blackbox history, restored main to earlier points and left 8 test accounts with 70 working API keys. They now run only in a throwaway test VM and refuse to run anywhere else. On this install the tables were dropped and the keys revoked; the Blackbox history from before 16 Sep 2026 cannot be recovered. (M1)
31. ~~On a new install the first `fox start` could leave the control plane and the Agent API down (`SQLITE_BUSY` creating the auth store)~~ — **fixed**: store creation takes the write lock up front and retries. (G1)
32. ~~An agent's changes through its Gateway DSN lost the session its branch was created with~~ — **fixed**: the Gateway no longer gives an agent key a session of its own. (H16)
33. ~~Tool, kind and session are client-settable, so a client can record itself as the other kind of actor~~ — **fixed** for the kind, which is what attribution rests on: it is derived from the login role (an `agent-<id>` role is an agent, a per-user role is a person), so an agent can no longer be recorded as a human. Tool and session are declared by the client by design and are now labelled that way on the Blackbox page. (H3)
34. ~~The MCP server has no authentication of its own: any client that can start it gets all 16 tools, and its changes are recorded against the shared role~~ — **fixed**: `fox mcp` requires an API key and acts as that account, with no unauthenticated path; a key scoped to one branch is held to that branch. (K6)
35. ~~The console displayed a uuid column as a list of 16 numbers (and an interval as a struct, an inet with quotes, a bytea as mangled text)~~ — **fixed**: values pgx decodes into byte arrays or structs are shown in Postgres's own text form. Reported from the UI, where every `id` column looked like `[195,64,152,78,…]`. (L4)
36. ~~`setup` hung for ever when a container named `minio` already existed on the machine~~ — **fixed**: the object store is called `objstore`, a name of ours rather than the software's, and the bucket step gives up with a clear message instead of retrying invisibly. Found by a user whose machine had another project's `minio`. (A3, D1)
37. ~~Removing the product was a list of commands to run by hand, and missing one left containers, a pool or a stale binary behind~~ — **fixed**: `fox uninstall` lists what it will remove, asks, then removes it — including installs left by retired product names. (B1)
38. ~~Renaming the product broke behaviour, not just text: the database raised a message carrying the product's name, the SQL console matched that prose, and a test asserted it~~ — **fixed**: messages the database raises carry no product name, and the names written into data carry none either, so a rename cannot reach them. (H4, L4)
39. ~~A Linux install listened on every interface with open sign-up, published its backup store, restore and standby ports publicly, fell back to plaintext without a certificate, and let any account delete any branch through a pipeline name or read server files through a pipeline source~~ — **fixed** 22 Sep 2026 (audit v2 G03, G05–G09, G11, G13, G15; N1–N5). (A5, C4, E2, E3, G4, G5, J1, K1, K5)
40. ~~Any account could act on every branch and agent; import reached any host the engine could; one install secret was every role's password and was passed on command lines; accounts could not be listed, deleted or change their password~~ — **fixed** 22 Sep 2026 (audit v2 G02, G10, G12, G16, N6). (E1, F2, F4, G2, I11, K1, K5, K6)
41. ~~TRUNCATE was neither blocked nor recorded, and an agent's DELETE or UPDATE left no trace; anchors were unsigned; nothing recorded sign-ins, keys or refused requests~~ — **fixed** 22 Sep 2026 (audit v2 G04, G17 signing, G28). People's DML and INSERTs are still not recorded, and anchors are not yet copied off the machine. (H1, H4, H10, H11, G6)
42. ~~Every backup lived on the same disk as main; backups were manual only and a new install had none~~ — **fixed** 22 Sep 2026 (audit v2 G18, G19): a remote S3 target with Object Lock, a first backup at install, a schedule with retention, and the newest backup's age in `fox status`. Not yet tested against AWS S3 itself; no automated test restore. (D1, D2, H10)

43. ~~Releases were checksum-only; images were pinned by tag; wal-g was downloaded unchecked; Actions were pinned by tag~~ — **fixed** 22 Sep 2026 (audit v2 G22, G23): signed SHA256SUMS checked by fox update and install.sh, build attestations, an SBOM, digests and SHA-256 pins, Dependabot. The Windows installer checks checksums only (no Ed25519 in PowerShell). The first signed release has not been cut yet. (A1, A2, A9, M4, M6)
44. ~~The suites ran only by hand, and the clients and the console had no automated tests at all~~ — **fixed** 23 Sep 2026 (audit v2 G24, G26): a nightly job runs every suite on its own runner, the published clients are driven against a live engine, and the console is driven in a browser. The first nightly run on GitHub is still to come; release tags are not gated on it.
45. The SQL console (and its REST endpoint) runs one statement per call: a pasted script of several is refused by the extended protocol. (L4)
46. Storage quotas are ZFS-only and per-branch caps are opt-in. (C6)
47. ~~The Postgres image has no pgvector~~ — **fixed** 28 Sep 2026: the image installs pgvector from the PGDG repository the official image already trusts, and the build fails if `vector.control` is missing afterwards. The line outlived the fix by ten days, and an investor deck was corrected against it before anyone noticed the doc was the stale one. (M6)
48. ~~Every `fox start` appends nine unattributed entries to each running branch's Blackbox~~ — **fixed** 21 Sep 2026 on `feat/pg-upgrade`: the event triggers are created after the privilege block. The note below is kept as found.
    Every `fox start` appended nine unattributed entries to each running branch's Blackbox — `GRANT`, `REVOKE` and `ALTER DEFAULT PRIVILEGES` from the Blackbox's own install script (`ledger.sql`), which re-runs on every start and whose privilege statements are not on `bb` objects, so `bb._skip` does not filter them. Found 21 Sep 2026 testing D5/D6; the update suite had been comparing history by digest up to a known id, which hid it. The chain stays intact; the page shows maintenance as if it were a user's change. (H1, L5)